Trajectory Planning for Contour Following and Robotic Deburring

by

Wade MacMillan, B.Eng.

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs in partial fulfillment of the requirements for the degree of

Masters of Applied Science

in

Mechanical Engineering

Ottawa-Carleton Institute for Mechanical and Aerospace Engineering

Department of Mechanical and Aerospace Engineering

Carleton University

Ottawa, Ontario

December, 2022

©Copyright

Wade MacMillan 2022

Abstract

The work of this thesis is part of the RoboDeb project, aimed at investigating the application of industrial robots on deburring sheet metal parts. This thesis presents investigations, developments, and implementations of trajectory planning algorithms to automate trajectory design. This thesis first develops custom CAD/CAM software called the *Planar Computer Automated Trajectory* (PCAT) planning algorithm to establish baseline performance. Trajectory planning is next automated using the novel *Planar Image-Space Trajectory* (PIST) planning algorithm, which uses computer vision alone to generate machining trajectories. The PIST algorithm is completely automated, with no required CAD data of the workpiece, adapting to new workpieces or manufacturing imperfections. The feasibility of the PIST algorithm is investigated through robotic deburring experimentation, where it successfully deburred the sheet metal parts. The PIST algorithm proved to be the preferred candidate over the PCAT algorithm, as it provided an indistinguishable resulting surface finish while reducing the setup time.

To my parents Rick and Ivana, for their love and support.

Acknowledgements

I would like to acknowledge the feedback and support of Dr. Rishad Irani of the Multi Domain laboratory and Dr. Mojtaba Ahmadi of the Advanced Biomechatronics and Locomotion Laboratory, as well as Steffan Lloyd, Reza Dehghanitafti, Ryan McKenzie Grael Miller and Devon Wang for their contributions to the RoboDeb platform and assistance provided throughout my studies. The author also acknowledges the support of the Natural Sciences and Engineering Research Council of Canada (NSERC) grant CRDPJ 514258-17.

Contents

Abstract i

Acknowledgements iii

List of Figures x

List of Tables xi

Nomenclature **xii**

1 Introduction **1**

1.1 Motivation 1

1.2 Background of Thesis 4

1.3 Contributions 7

1.4 Organization 8

2 Mathematical Preliminaries **9**

2.1 Computer Vision 9

2.1.1 Perspective Projection 10

2.1.2 Planar Homography 12

2.1.3 Camera Calibration 15

2.1.4 Orthorectification 18

2.1.5 Stereo Vision 21

2.1.6 Laser Scanner 23

2.2 Point Cloud Registration 25

2.2.1 Iterative Closest Point 25
2.2.2 Random Sample Consensus 26
2.3 Force Control 27
2.4 Summary 29

3 Robotic Manufacturing Background 30
3.1 CAD/CAM Software 30
3.1.1 Local Path Planning 31
3.1.2 Localize Part 33
3.1.3 Tool Path Correction 35
3.2 Computer Vision Path Planning 36
3.2.1 Image-Space Path Planning 37
3.2.2 Transfer the Path into Workspace 41
3.3 Summary 44

4 RoboDeb Platform 45
4.1 Robot Manipulator 45
4.2 Host PC 47
4.3 Workspace 49
4.4 Computer Vision 51
4.4.1 Laser Scanner 51
4.4.2 Stereo Camera 51
4.5 Summary 54

5 Planar Computer Automated Trajectory Planner 55
5.1 PCAT Algorithm Description 56
5.1.1 Read Entities 56
5.1.2 Create Segments 57
5.1.3 Offset Segments 59

5.1.4 Transform Segments 61
5.1.5 Define Workspace Waypoints 62
5.2 RoboDeb Implementation 67
5.3 Summary 70

6 Planer Image Space Trajectory Planner 71
6.1 PIST Algorithm Description 72
6.1.1 Perspective Transformation 73
6.1.2 Image-Space Path Planning 73
6.1.3 Define Trajectory 77
6.1.4 Define Image-Space Waypoints 79
6.1.5 Define Workspace Waypoints 80
6.2 Three-Dimensional PIST Algorithm 82
6.2.1 Orthorectification 83
6.3 PIST Algorithm Implementation 84
6.4 The 3D PIST Algorithm Implementation 87
6.4.1 Orthophoto 87
6.4.2 Image-space Path Planning 88
6.5 Summary 91

7 Deburring Experimentation 92
7.1 Planar Experimentation 92
7.1.1 Results Summary 95
7.1.2 Detected Path Position Error 97
7.1.3 Applied Force Error 98
7.1.4 Process Time 99
7.1.5 Part Finish 99
7.2 Three Dimensional Experimentation 100

7.2.1 Procedure 100
7.2.2 Results Summary 101
7.2.3 Detected Path Position Error 102
7.2.4 Applied Force Error 102
7.2.5 Machining Result 102
7.3 Summary 105

8 Conclusion **106**
8.1 PCAT Algorithm 106
8.2 PIST Algorithm 107
8.3 Deburring Experimentation 107
8.4 Future Work 108

List of Figures

1.1 RoboDeb deburring platform 3

2.1 The perspective projection 10
2.2 Planar homography 14
2.3 Camera distortion 16
2.4 Camera calibration procedure 18
2.5 Two dimensional orthorectification procedure 20
2.6 Three dimensional orthorectification procedure 20
2.7 Three dimensional orthorectification results 20
2.8 Stereo camera configuration 21
2.9 Micro Epsilon laser scanner system 24
2.10 Laser scanner diagram 24
2.11 Correspondence between point clouds 26
2.12 Impedance and admittance control diagrams 28

3.1 Contour following machining path 31
3.2 Image-space path planning example 38
3.3 Projection of pixels onto planar surfaces 42
3.4 Projection of pixels onto 3D surface 43

4.1 RoboDeb platform diagram 46
4.2 RoboDeb deburring platform. 47

4.3 RoboDeb end effector package. 48
4.4 Vacuum clamp design. 50
4.5 Laser scanner point cloud 52
4.6 Stereo image pair 53
4.7 Point cloud generation using stereo matching 53
4.8 Point cloud generation using a monocular camera 53

5.1 PCAT algorithm workflow. 56
5.2 PCAT algorithm entities 58
5.3 Offset procedure workflow. 59
5.4 Offset corner conditions 60
5.5 Interpolated waypoints 63
5.6 Tool pose selection 64
5.7 Tool pose results 65
5.8 RoboDeb RANSAC and ICP Registration 69
5.9 Output waypoints of the PCAT algorithm 69

6.1 Workspace of the PIST algorithm. 72
6.2 Process of the PIST algorithm. 73
6.3 Orthorectification using planar homography 74
6.4 Image-space path planning process for the PIST algorithm 75
6.5 Tool feedrate selection 78
6.6 Workspace of the 3D PIST algorithm 82
6.7 Orthorectification results of 3D PIST 84
6.8 PIST algorithm applied to example image 85
6.9 Image-space path planning applied to example image 86
6.10 Output waypoints of the PIST algorithm 86
6.11 Geometry of the 3D test part for validation of the 3D PIST algorithm 89

6.12 Orthorectification on an example image of a 3D part 89
6.13 Output waypoints of the 3D PIST algorithm 90
6.14 Image-space path planning applied to an example image of a 3D part 90

7.1 Sheet metal test part geometry and deburring path 93
7.2 Surface finish for sheet metal deburring 95
7.3 Sheet metal deburring results 96
7.4 Three dimensional test part geometry and deburring path 102
7.5 Three dimensional contour following results 103
7.6 Surface finish for three dimensional contour following 104

8.1 Estimated normal direction of point cloud 110

List of Tables

7.1 Positional Error results summary 96

Nomenclature

Variables

$\mathbf{I}_{\mathrm{b}}$	Image with description b
$\mathbf{X}$	Matrix
$\mathbf{x}$	Vector
$\hat{\mathbf{x}}$	Unit vector
ω	Frequency
θ	Angle
$\tilde{x}$	Estimatd value
${}^{\mathrm{a}}\mathbf{b}_{\mathrm{b}}$	2D Point in space a with description b and coordinates x and y
${}^{\mathrm{a}}\mathbf{p}_{\mathrm{b}}$	3D Point in space a with description b and coordinates x, y and z
${}^{\mathrm{a}}\mathbf{q}_{\mathrm{b}}$	Pixel in image a with description b and coordinates u and v
f	Force
v	Feedrate
w	Waypoint

Mappings

${}^{a}\mathbf{H}_{b}$	Homography Matrix, from space a to b
${}^{a}\mathbf{K}_{b}$	Camera Intrinsic Matrix, from space a to b
${}^{a}\mathbf{M}_{b}$	Perspective projection Matrix, from space a to b
${}^{a}\mathbf{R}_{b}$	Rotation Matrix, from a space to b
${}^{a}\mathbf{S}_{b}$	Similarity transformation, from space a to b with scaling s.
${}^{a}\mathbf{T}_{b}$	Transformation Matrix, from space a to b.
${}^{a}\mathbf{t}_{b}$	Translation Matrix, from space a to b

Abbreviations

AOA	Angle of Attack
CAD	Computer Aided Design
CAM	Computer Aided Manufacturing
DOF	Degree Of Freedom
ICP	Iterative Closest Point
NSERC	Natural Sciences and Engineering Research Council of Canada
PCAT	Planar Computer Aided Trajectory Planner
PIST	Planar Image-space Trajectory Planner
RANSAC	Random Sample Consensus
RoboDeb	Robotic Deburring
STD	Standard Deviation

Superscripts

$\mathbf{X}^\mathsf{T}$	The transpose of the the matrix or vector $\mathbf{X}$
$x^{(i)}$	References the i^{th} item in a series of variations of the variable x.

Subscripts

$(\square)_{\Pi}$	Planar space
$(\square)_{\text{CAM}}$	CAD model space
$(\square)_{\text{cb}}$	Clamp base space
$(\square)_{\text{c}}$	Camera space
$(\square)_{\text{I}}$	Image space
$(\square)_{\text{o}}$	Orthophoto space
$(\square)_{\text{w}}$	World space

Referencing

$\mathbf{X}[i, j]$	References the entry in the i^{th} row and j^{th} column of the matrix $\mathbf{X}$
$\mathbf{x}[i]$	References the i^{th} entry of the vector $\mathbf{x}$
$I[\mathbf{q}]$	References the pixel intensity of image $\mathbf{I}$ at pixel $\mathbf{q}$
$X(y)$	X is a function of y

Chapter 1

Introduction

1.1 Motivation

Burrs are rough imperfections present on metal and composite parts after manufacturing, which can compromise a parts functionality, onset fatigue, and present a safety risk when handling and in service. It is standard in aerospace manufacturing for a skilled worker to remove burrs after machining, which is a laborious and time-consuming process. Manual deburring introduces inconsistencies and possible damage at a critical phase of production when the product is nearing completion and has a considerable sunken cost. For workers, manual deburring is monotonous and hazardous due to exposure to harmful dust, prolonged vibrations, and ergonomic hazards. As a result, deburring is an expensive process, and estimated to account for between 9 - 35% [1, 2] of manufacturing costs.

Deburring is an excellent candidate for robotic automation, due to its financial incentives and repetitive nature. Serial manipulator robots are fundamental in the automation of modern manufacturing as they provide a large, reconfigurable workspace

at 20 - 30% the cost of a comparable gantry [3]. Currently, serial manipulator robots in manufacturing are primarily used in material handling or non-contact processes, such as welding or painting. Despite ongoing research in the past 50 years, merely 1 - 3 % of serial manipulator robots are intended for machining operations such as cutting, grinding, deburring, milling and polishing [4]. Two major limitations inhibiting robotic machining with serial manipulator robots are the insufficient positional accuracy, despite high repeatability, and the large setup time associated with path planning [1].

The work presented in this thesis is part of the Robotic Deburring (RoboDeb) project. The objective of the RoboDeb project is to examine the application of a serial manipulator robot to deburring sheet metal parts and related applications. The leads of the RoboDeb team are Mojtaba Ahmadi and Rishad Irani. The RoboDeb team members are Steffan Lloyd [5,6], Grael Miller [7], Reza Dehghanitafti, Ryan McKenzie [8], and Devon Wang, who designed and built much of the RoboDeb platform prior to the author's arrival. Figure 1.1 displays the specialized RoboDeb platform which uses a 6 Degree Of Freedom (DOF) Denso serial manipulator robot. The robot is enclosed in a work cell with a robust safety system designed and built by Grael Miller and Devon Wang [7]. The robot is actuated using an open architecture control box, allowing for the use of custom robot control logic. Equipped to the robot is an end effector package, including a spindle, force-torque sensor and a laser scanner.

The two major limitations inhibiting robotic machining are the insufficient positional accuracy and the large setup time which are addressed on the RoboDeb platform. The accuracy limitation is addressed through a precise calibration procedure designed by Reza and Ryan and presented in Dehghanitafti et al. [8]. The robot was locally calibrated and achieved positional accuracy of $\pm$ .15 mm. Experimentation proved that even with great repeatability and precise local calibrations, the accuracy

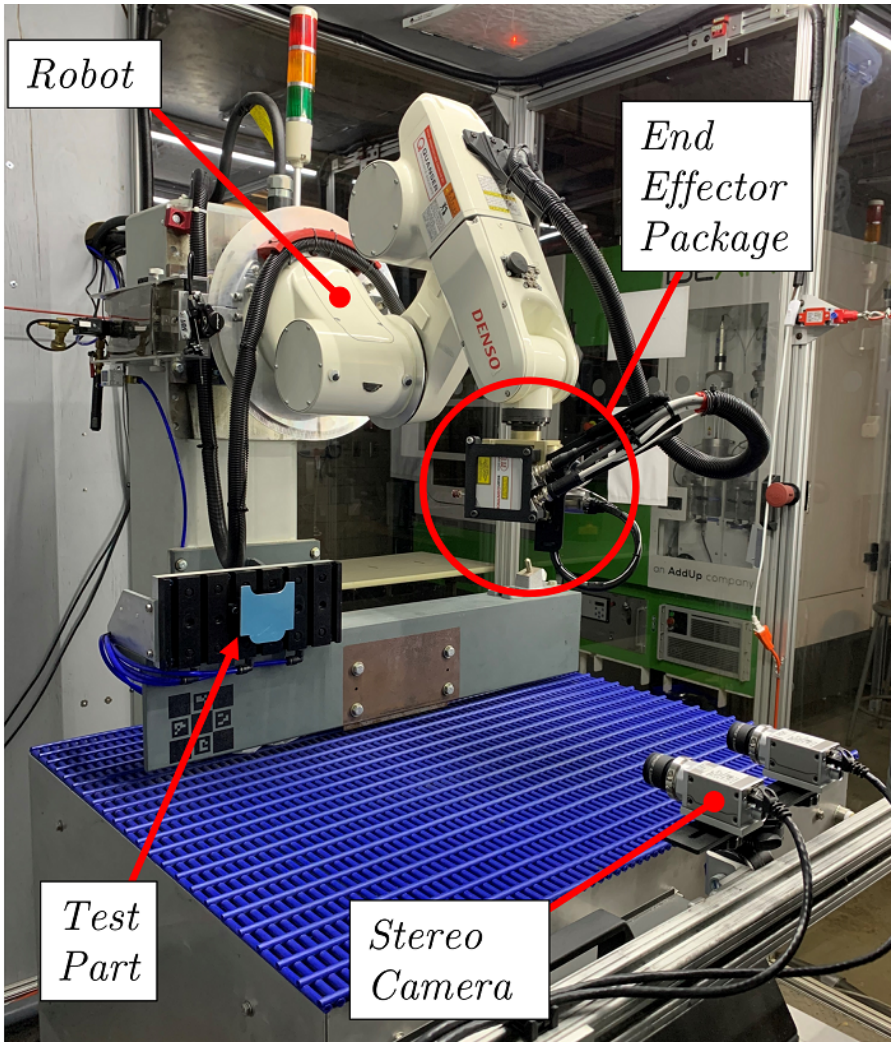


Figure 1.1: RoboDeb deburring platform

of the industrial robot is not sufficient for machining with position control alone [8]. Therefore, Steffan has designed an admittance controller using online feedback from the force sensor. The admittance controller improves the edge tracking accuracy of the robot by admitting a positional error in the pre-defined trajectory to maintain a set contact force. The setup time limitation is addressed using custom trajectory planning algorithms created by the author. **The contribution of this thesis to the RoboDeb project is the proposed methods, investigations, developments and implementations of automated trajectory planning algorithms for robotic deburring.**

1.2 Background of Thesis

Path planning is the process of defining an ordered geometric path for a robot to follow. Trajectory planning is an extension of path planning, through adding time parameterization to the planned path. Currently, it is standard practice to program industrial robots through teach and playback methods where the robot is manually taught a finite series of waypoints which are replayed in operation. A skilled operator performs the teaching process manually, which requires a significant amount of setup time, sometimes taking weeks [9]. For each unique contour, teaching must be repeated, making teach and playback impractical for high-mix, low-volume applications.

A ubiquitous task for an industrial robot in manufacturing is tracking the periphery of a planar contour, present in robotic deburring. There is a considerable amount of research on developing automated path and trajectory planners for programming industrial robots. The most prevalent offline planning method uses Computer-Aided Manufacturing (CAM) software to design a path with reference to a CAD model of the workpiece [1]. CAD/CAM software is extensively used in CNC machining, therefore, there are a variety of powerful off-the-shelf solutions available. With CAD/-CAM software, path created around the CAD model is transformed into the robot's workspace based on the workpiece's location, determined through a localization procedure. A skilled operator can perform localization on a fixed workplace manually through placement or measurement relative to a datum, or localization is automated using the robot equipped with a touch-off probe [10–12] or a vision sensor [13–15]. The benefit of localization using computer vision is that it is non-contact and can easily be automated. Off-the-shelf CAD/CAM software is typically designed for CNC milling, which often does not include the applied force direction required for machining with force control. In this thesis, the Planar Computer Automated Trajectory

Planning (PCAT) algorithm is developed to design trajectories with a CAD/CAM software approach for deburring planar sheet metal parts on the RoboDeb platform with an admittance controller. Localization is performed using both the laser scanner and stereo camera systems on the RoboDeb platform for comparison. The PCAT algorithm establishes a benchmark for future trajectory planners created. **Thus, the first objective of this thesis is to develop the PCAT algorithm for designing planar contour following trajectories using CAD data.**

A requirement for CAD/CAM path planning is an accurate CAD model of each workpiece, which may be impractical with low-volume work, repair applications or on parts with manufacturing imperfections. The localization step required by CAD/-CAM software increases the total process time, complexity, and cost. Computer vision alone can provide all the necessary spatial information for path planning, removing the need for CAD data. Computer vision path planning is typically performed using a camera system to capture an image of a fixed workpiece. The workpiece and desired path is detected in the image, and the image-space path is projected into the workspace for the robot to playback. Computer vision path planning is quick, completely automated, and does not require a good initial guess of the workpiece location. The absence of a model of the workpiece geometry restricts computer vision path planning to machining applications with minor defects, like welding or deburring.

The field of robotic welding has adopted computer vision path planning. In Micallef et al. [16] and Shah et al. [17], computer vision path planning was performed using a monocular camera placed perpendicular to a planar workpiece. The welding path was designed in image-space along the detected weld seam and projected onto the planar surface in the workspace. A perpendicular perspective is desirable, as there is uniform scaling between the image and the planar surface, useful for designing paths relative to detected features. Micallef et al. and Shah et al. assumed

the perpendicular perspective of the camera to perform the projection of the image-space path into the workspace. However, the position of a planar surface relative to a monocular camera can be extracted using planar homography. Chang et al. [18] used planar homography to detect a plane's location and project the path into the workspace. Planar homography also can perform perspective corrections, useful for obtaining a perpendicular perspective of a planar surface.

Robotic deburring requires high accuracy to track the edges of a workpiece with a consistent surface finish, which is impractical on a serial manipulator robot without a force control scheme [8]. Jinno et al. [19] used computer vision path planning to design deburring and chamfering paths around planar workpieces using a monocular camera. The selected path was offset to the detected workpiece to account for the radius of the deburring tool with a known normal direction to apply force using a force controller. The selection of the offset path and normal direction was not addressed in their study [19]. Section 3.2 of this thesis presents a literature review on computer vision path planning, but there was no solution found for selecting an offset path with force control in image-space. In this thesis, the Planar Image-Space Trajectory (PIST) planning algorithm is introduced and developed using computer vision for deburring sheet metal parts on the RoboDeb platform with an admittance controller. The PIST algorithm uses a fixed monocular camera to provide spatial information of a planar sheet metal part. Path planning is performed entirely in a geometrically corrected image-space, perpendicular to the part obtained using planar homography. The part is detected in the corrected image and an offset path with an applied force direction is designed. The perpendicular perspective is required for designing the image-space path relative to the detected part. **The second objective is to develop the PIST algorithm for designing contour following trajectories using computer vision.**

Computer vision path planning is predominantly used for, but not restricted to, planar applications. Path planning along a 3D surface requires a model of the surface for path design. Solvang et al. [20, 21] performed path planning in image-space and projected the path onto a localized CAD model of the workpiece. Solvang's approach requires the re-introduction of CAD data and localization, avoided with computer vision path planning. Chang et al. [18] and Pagano et al. [22] performed 3D path planning using a Kinect RGB-D camera to obtain a textured point cloud model of the workpiece. A proof of concept 3D PIST algorithm is created using orthorectification to obtain a perpendicular perspective of the a 3D workpiece in a 2D image-space. Orthorectification is the process of removing the effects of perspective and surface elevation in an image, to create a to-scale representation of the surface. With the perpendicular perspective, an offset path with an applied force direction is again obtained in image-space, then projected into the robot's workspace.

The following thesis will describe the design of two trajectory planning algorithms; the PCAT algorithm using a traditional CAD/CAM software approach, and the novel PIST algorithm using computer vision. Both trajectory planners perform planar contour following using force control and are applied to deburring sheet metal parts on the RoboDeb platform. The PCAT algorithm establishes a benchmark for deburring performance, while the PIST algorithm is a novel computer vision approach to trajectory planning to design paths without the requirement for CAD data.

1.3 Contributions

The contributions made over the course of this thesis are:

- The PIST algorithm employed planar homography and orthorectification to correct for the perspective of the camera and obtain a to-scale representation of 2D and 3D surfaces.

- Established a novel image processing procedure to design machining paths in image-space. The procedure uses the Euclidean distance transform and its gradient to extract an offset path with a known normal direction for force control used in the PIST algorithm.
- Developed and implemented the PCAT and PIST trajectory planners in a robotic deburring application and evaluated their performance based on positional accuracy, applied force error, processing time and part finish.
- Compared the performance of the laser scanner and camera system for workpiece localization and evaluated their performance based on positional accuracy and processing time.
- Contributed to the development of the RoboDeb platform through retrofitting a brake to Joint 1 of the Robot and designing a custom vacuum clamping solution as an essential part of the process to hold parts while deburring.

1.4 Organization

This thesis is organized as follows. The introduction chapter is followed by Chapter 2 summarizing mathematical preliminaries. Chapter 3 contains a background and literature review on contour following path planning in robotic machining. Chapter 4 details the RoboDeb platform designed for robotic deburring experimentation. Chapter 5 and Chapter 6 describe the design of the PCAT and PIST algorithms respectively and their implementation on the Robodeb Platform. Chapter 7 describes the results of each trajectory planning algorithm when applied to robotic deburring. Finally, Chapter 8 describes the conclusions of the thesis.

Chapter 2

Mathematical Preliminaries

The following chapter will present mathematical preliminaries required to introduce trajectory planning in this thesis. The chapter begins with Section 2.1 which gives a background on computer vision using various camera systems. Section 2.2 describes point cloud registration using the Iterative Closest Point (ICP) algorithm and Random Sample Consensus (RANSAC) algorithm. Section 2.3 presents online force control schemes used in contour tracking applications.

2.1 Computer Vision

Computer vision is an interdisciplinary field that deals with acquiring, processing, and analyzing digital images. The following section will provide the geometrical foundations of computer vision and applications for extracting spatial information. Section 2.1.1 will begin with the perspective projection, which will model the projection of point features in the world onto an image from a monocular camera. Next, in Section 2.1.2 the perspective projection is simplified to the projection of planar surfaces onto an image to introduce the concept of planar homography. Section 2.1.3

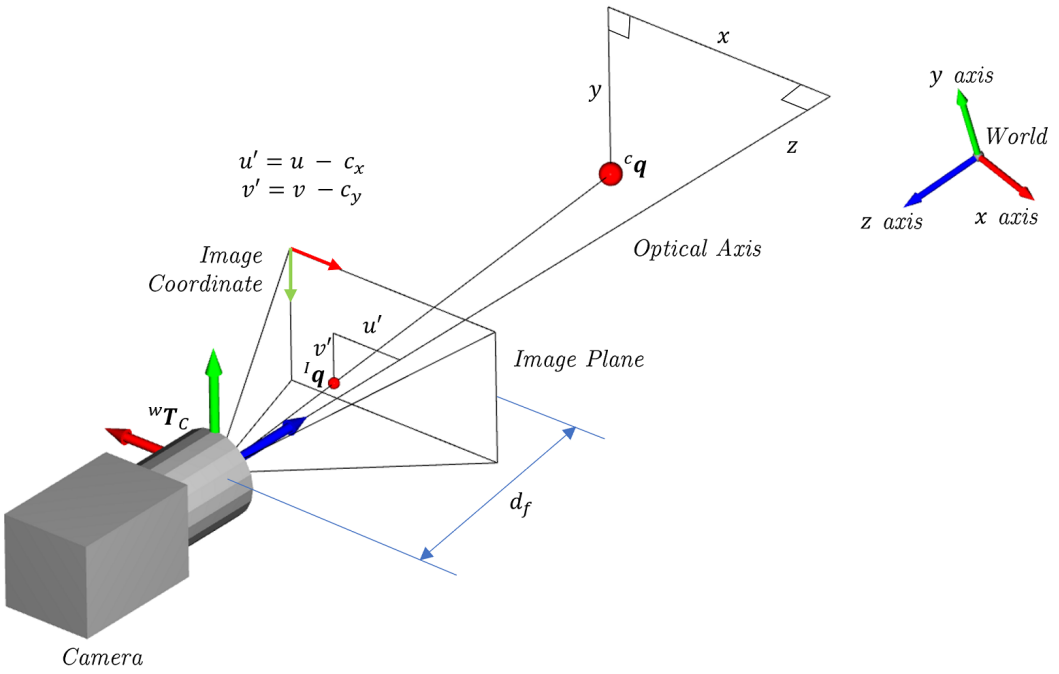


Figure 2.1: The linear projection of a point feature **p** onto a pixel feature **q** in an image using the pinhole camera model.

presents the calibration of a real monocular camera. Section 2.1.4 presents a perspective transformation onto a normal view of a surface using orthorectification. Finally, Section 2.1.5 and Section 2.1.6 present the extraction of 3D information using a stereo camera and laser scanner, respectively.

2.1.1 Perspective Projection

The projection of 3D features onto a 2D image provided by a camera is modelled with the pinhole camera model. The pinhole camera model characterizes the projection of a point feature as a single ray of light, projected through a pinhole aperture and onto the sensor of the camera. Figure 2.1 displays the pinhole camera model where a point feature ${}^{c}\mathbf{p} = \begin{bmatrix} x & y & z \end{bmatrix}^T \in \mathcal{R}^3$ is projected onto a pixel on the image ${}^{I}\mathbf{q} = \begin{bmatrix} u & v \end{bmatrix}^T \in \mathcal{R}^2$. Mathematically, the linear perspective projection is modeled

using a similar triangles relationship as,

$$u' = d_f \frac{x}{z}, \quad v' = d_f \frac{y}{z}, \tag{2.1}$$

where, $u' = u - c_x$ and $v' = v - c_y$ are used to translate the reference coordinate frame from the optical centre of the image (c_x, c_y) into the image coordinate frame. Image data is stored as a matrix of pixel intensity values. The image coordinate frame is placed at the upper left corner of the image so that $\mathbf{I}[0, 0]$ references the pixel intensity at the first column and row of the image matrix. d_f is the focal length in pixels. The similar triangles relationship is put into matrix form,

$$\begin{bmatrix} {}^{\mathrm{I}}\mathbf{q} \\ 1 \end{bmatrix} = \lambda \, {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \, {}^{\mathrm{c}}\mathbf{p}, \tag{2.2}$$

where λ is a non-zero scaling factor to transfer the pixel $\mathbf{q}$ to homogeneous coordinates denoted as $\begin{bmatrix} \mathbf{q} & 1 \end{bmatrix}^T$. The camera intrinsic matrix ${}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}}$ transforms from the camera's coordinate frame c to image-space I and has the form,

$${}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} = \begin{bmatrix} d_{fx} & 0 & c_x \\ 0 & d_{fy} & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{2.3}$$

where the focal lengths d_{fx} and d_{fy} are split into the x and y component to account for non-square imager elements.

Features ${}^{\mathrm{c}}\mathbf{p}$ in equation (2.2) are in reference to the camera coordinate frame, which is often not aligned with the world coordinate frame. To use equation (2.2) referencing features the world coordinate frame ${}^{\mathrm{w}}\mathbf{p}$, the points are first transformed into the camera coordinate frame with,

$$\begin{bmatrix} {}^{\mathrm{I}}\mathbf{q} \\ 1 \end{bmatrix} = \lambda \, {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \begin{bmatrix} {}^{\mathrm{c}}\mathbf{R}_{\mathrm{w}} & {}^{\mathrm{c}}\mathbf{t}_{\mathrm{w}} \end{bmatrix} \begin{bmatrix} {}^{\mathrm{w}}\mathbf{p} \\ 1 \end{bmatrix}, \tag{2.4}$$

where the camera extrinsic matrix ${}^{\mathrm{c}}\mathbf{T}_{\mathrm{w}} \in SE(3)$ is the camera location with respect to the world, composed as,

$$ {}^{\mathrm{c}}\mathbf{T}_{\mathrm{w}} = \begin{bmatrix} {}^{\mathrm{c}}\mathbf{R}_{\mathrm{w}} & {}^{\mathrm{c}}\mathbf{t}_{\mathrm{w}} \\ 0 & 1 \end{bmatrix}, \tag{2.5} $$

using the rotation ${}^{\mathrm{c}}\mathbf{R}_{\mathrm{w}} \in SO(3)$ and translation ${}^{\mathrm{c}}\mathbf{t}_{\mathrm{w}} \in \mathcal{R}^3$ between the camera and world coordinate frames. The perspective projection matrix ${}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}$ can be defined as,

$$ {}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}} = {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \begin{bmatrix} {}^{\mathrm{c}}\mathbf{R}_{\mathrm{w}} & {}^{\mathrm{c}}\mathbf{t}_{\mathrm{w}} \end{bmatrix}, \tag{2.6} $$

such that the projection of 3D point features onto an image is now performed with,

$$ \begin{bmatrix} {}^{\mathrm{I}}\mathbf{q} \\ 1 \end{bmatrix} = \lambda \, {}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}} \begin{bmatrix} {}^{\mathrm{w}}\mathbf{p} \\ 1 \end{bmatrix}. \tag{2.7} $$

The inverse relation of equation (2.7) projects pixel features ${}^{\mathrm{I}}\mathbf{q}$ into lines in $\mathcal{R}^3$ which contains the corresponding point features $\mathbf{p}$. A monocular camera alone cannot directly locate pixel features in 3D space, as the scaling factor λ is unknown. By introducing a planar constraint on features of interest in the scene, planar homography can map between pixels in an image and features on the plane.

2.1.2 Planar Homography

The projection of a planar surface onto an image results in an elegant simplification of equation (2.4). A rotation matrix is composed of three orthonormal unit vectors as,

$$ {}^{\mathrm{a}}\mathbf{R}_{\mathrm{b}} = \begin{bmatrix} {}^{\mathrm{a}}\hat{\mathbf{n}}_{\mathrm{b}} & {}^{\mathrm{a}}\hat{\mathbf{s}}_{\mathrm{b}} & {}^{\mathrm{a}}\hat{\mathbf{a}}_{\mathrm{b}} \end{bmatrix}, \tag{2.8} $$

where ${}^{\mathrm{a}}\hat{\mathbf{n}}_{\mathrm{b}}$, ${}^{\mathrm{a}}\hat{\mathbf{s}}_{\mathrm{b}}$, ${}^{\mathrm{a}}\hat{\mathbf{a}}_{\mathrm{b}} \in \mathcal{R}^3$ and are column vectors corresponding to unit vectors along the x, y and z axis of the coordinate system with rotation ${}^{\mathrm{a}}\mathbf{R}_{\mathrm{b}}$. By assigning a coordinate frame to the planar surface, restricting the projection to points on a plane

${}^{\Pi}\mathbf{b} \in \mathcal{R}^2$ and expanding the rotation matrix, equation (2.4) reduces to,

$$\begin{aligned}
\begin{bmatrix} {}^{\mathrm{I}}\mathbf{q} \\ 1 \end{bmatrix} &= \lambda\ {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \begin{bmatrix} \begin{bmatrix} {}^{\mathrm{c}}\hat{\mathbf{n}}_{\Pi} & {}^{\mathrm{c}}\hat{\mathbf{s}}_{\Pi} & {}^{\mathrm{c}}\hat{\mathbf{a}}_{\Pi} \end{bmatrix} & {}^{\mathrm{c}}\mathbf{t}_{\Pi} \end{bmatrix} \begin{bmatrix} \begin{bmatrix} {}^{\Pi}x \\ {}^{\Pi}y \\ 0 \end{bmatrix} \\ 1 \end{bmatrix} \\
&= \lambda\ {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \begin{bmatrix} {}^{\mathrm{c}}\hat{\mathbf{n}}_{\Pi} & {}^{\mathrm{c}}\hat{\mathbf{s}}_{\Pi} & {}^{\mathrm{c}}\mathbf{t}_{\Pi} \end{bmatrix} \begin{bmatrix} {}^{\Pi}x \\ {}^{\Pi}y \\ 1 \end{bmatrix} \\
&= {}^{\mathrm{I}}\mathbf{H}_{\Pi} \begin{bmatrix} {}^{\Pi}\mathbf{b} \\ 1 \end{bmatrix}.
\end{aligned} \tag{2.9}$$

The projection from planar features onto the image is now performed with a 3×3 homography matrix ${}^{\mathrm{I}}\mathbf{H}_{\Pi}$ defined as,

$${}^{\mathrm{I}}\mathbf{H}_{\Pi} = \lambda\ {}^{\mathrm{I}}\mathbf{K}_{\mathrm{c}} \begin{bmatrix} {}^{\mathrm{c}}\hat{\mathbf{n}}_{\Pi} & {}^{\mathrm{c}}\hat{\mathbf{s}}_{\Pi} & {}^{\mathrm{c}}\mathbf{t}_{\Pi} \end{bmatrix}. \tag{2.10}$$

Matrix ${}^{\mathrm{I}}\mathbf{H}_{\Pi}$ provides a non unique mapping between pixel features on an image onto a planar surface. Figure 2.2 describes the mapping from an image taken from camera 1 onto a plane, where ${}^{\mathrm{I1}}\mathbf{H}_{\Pi}$ can be computed with equation (2.10). Planar homography can be expanded to mappings between two planes or two images [23]. An image-to-image mapping is referred to as a perspective transformation, which is defined as,

$${}^{\mathrm{I1}}\mathbf{H}_{\mathrm{I2}} = \lambda\ {}^{\mathrm{I1}}\mathbf{K}_{\mathrm{c1}} \left({}^{\mathrm{c1}}\mathbf{T}_{\mathrm{c2}} + \frac{{}^{\mathrm{c1}}\mathbf{T}_{\mathrm{c2}}}{d_1} \overrightarrow{\mathbf{n}}^T \right)\ {}^{\mathrm{I1}}\mathbf{K}_{\mathrm{c1}}^{-1}, \tag{2.11}$$

Where $\overrightarrow{\mathbf{n}}$ is the normal vector of the plane and d_1 is the distance between the camera and the plane [24]. Note that a perspective transformation is performed relative to the plane and pixels associated with out-of-plane features will be distorted. Planar homography has the common structure of,

$$\begin{bmatrix} {}^{2}\mathbf{b} \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} {}^{1}\mathbf{b} \\ 1 \end{bmatrix}, \tag{2.12}$$

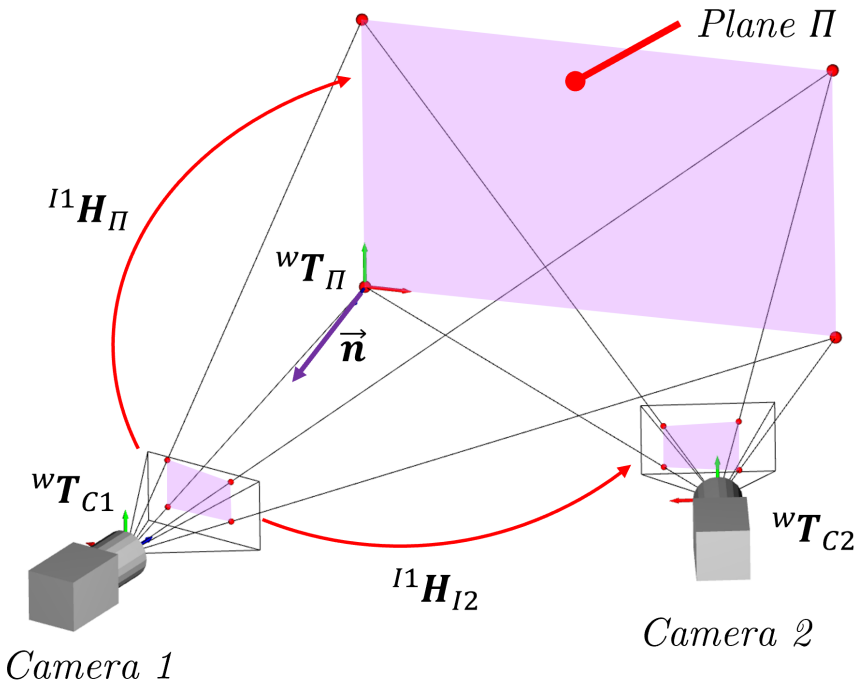


Figure 2.2: Relation of planar surfaces through planar homography.

where the 3×3 homography matrix $\mathbf{H}$ can be identified using equations (2.10), (2.11) or using the Direct Linear Transformation (DLT) algorithm [23]. The DLT algorithm uses a minimum of four known correspondences $i = \{1 \dots n\}$ between points ${}^{1}\mathbf{b}$ and ${}^{2}\mathbf{b}$ to compute $\mathbf{H}$. The formulation of the DLT algorithm begins with noting that the homogeneous vectors on the right and left hand side of equation (2.12) share the same direction, thus,

$$\begin{bmatrix} {}^{2}\mathbf{b}^{(i)} \\ 1 \end{bmatrix} \times \mathbf{H} \begin{bmatrix} {}^{1}\mathbf{b}^{(i)} \\ 1 \end{bmatrix} = 0, \tag{2.13}$$

since the cross produce between two parallel vectors is zero. With n correspondences between points ${}^{1}\mathbf{b}$ and ${}^{2}\mathbf{b}$, there is $2n$ constraints from n numbers of equation (2.13). Exploiting the skew symmetric property of the cross product and denoting the j^{th} row of $\mathbf{H}$ as $\mathbf{h}^{j}$ we obtain,

$$\begin{bmatrix} 0^{T} & -1\ {}^{1}\mathbf{b}^{(i)T} & {}^{2}y^{(i)}\ {}^{1}\mathbf{b}^{(i)T} \\ 1\ {}^{1}\mathbf{b}^{(i)T} & 0^{T} & -\ {}^{2}x^{(i)}\ {}^{1}\mathbf{b}^{(i)T} \\ -{}^{2}y^{(i)}\ {}^{1}\mathbf{b}^{(i)T} & {}^{2}x^{(i)}\ {}^{1}\mathbf{b}^{(i)T} & 0^{T} \end{bmatrix} \begin{bmatrix} \mathbf{h}^{1T} \\ \mathbf{h}^{2T} \\ \mathbf{h}^{3T} \end{bmatrix} = 0, \tag{2.14}$$

where ${}^2\mathbf{b}^{(i)} = [{}^2x^{(i)}, {}^2y^{(i)}]^{\mathrm{T}}$. The 3×3 skew-symmetric matrix only contains two linearly independent rows and is reduced to,

$$\begin{bmatrix} 0^T & -1\ {}^1\mathbf{b}^{(i)T} & {}^2y^{(i)}\ {}^1\mathbf{b}^{(i)T} \\ 1\ {}^1\mathbf{b}^{(i)T} & 0^T & -\ {}^2x^{(i)}\ {}^1\mathbf{b}^{(i)T} \end{bmatrix} \begin{bmatrix} \mathbf{h}^{1T} \\ \mathbf{h}^{2T} \\ \mathbf{h}^{3T} \end{bmatrix} = 0. \tag{2.15}$$

Equation (2.15) is now in the form of $\mathbf{A}^{(i)}\mathbf{h} = 0$ where $\mathbf{A}^{(i)}$ is a 2×9 matrix. Given n correspondences, a $2n \times 9$ matrix $\mathbf{A}$ is computed by vertically concatenating each $\mathbf{A}^{(i)}$ term. Singular value decomposition (SVD) is then used to compute $\mathrm{SVD}(\mathbf{A}) = \mathbf{UDV}^{\mathrm{T}}$, where the columns of $\mathbf{U}$ are left singular vectors, $\mathbf{D}$ is a diagonal matrix of singular values and $\mathbf{V}^T$ has rows of right singular vectors. The entries for $\mathbf{H}$ are extracted from the last row of $\mathbf{V}^T$ [23]. The DLT algorithm allows for the computation of a homography matrix without any camera intrinsic or extrinsic properties. Reflecting on equation (2.12), the homography matrix is composed of the camera calibration parameters, therefore the DLT algorithm will play a crucial role in camera calibration.

2.1.3 Camera Calibration

So far, perspective projection has been described using the pinhole camera model, which does not account for distortion and therefore does not accurately represent many cameras. The following will address a practical calibration procedure for characterizing the camera's distortion and computing the intrinsic and extrinsic matrices.

Distortion

The nonlinear effects of distortion are characterized and accounted for to implement the pinhole camera model. There are two main types of distortion present in cameras: radial distortion and tangential distortion. Radial distortion arises from the imperfect shape of the lens of the camera which causes the pixels to be translated

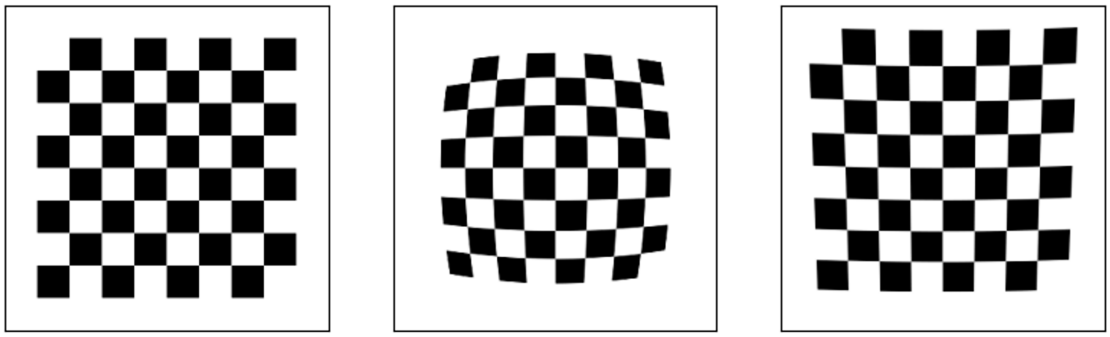


Figure 2.3: A) Checkerboard image, B) Radial distortion on checkerboard image, and C) Tangential distortion on checkerboard image.

radially from the center of the image. Figure 2.3 image B displays the effects of radial distortion. The effect of radial distortion is most pronounced at the outermost edges of the image. Radial distortion can be modelled as a Taylor series expansion of the translation of pixels in the radial direction as,

$$\begin{bmatrix} u'' \\ v'' \end{bmatrix}_{\text{radial}} = \begin{bmatrix} 1 + k_{\text{r1}}\ r^2 + k_{\text{r2}}\ r^4 + k_{\text{r3}}\ r^6 + \dots \\ 1 + k_{\text{r1}}\ r^2 + k_{\text{r2}}\ r^4 + k_{\text{r3}}\ r^6 + \dots \end{bmatrix}, \tag{2.16}$$

where $r = u'^2 + v'^2$ is the radial distance in pixels from the optical center, and the k_{r} terms are the Taylor series coefficients, which is typically truncated at the third term.

Tangential distortion arises from manufacturing imperfections in the assembly of the camera imager. If the imager is not assembled perfectly, there will be some cant in the perspective of the camera. Figure 2.3 image C displays the effect of tangential distortion. The checkerboard appears canted relative to the original image and the width of the upper edges is larger than the lower. Tangential distortion is characterized as,

$$\begin{bmatrix} u'' \\ v'' \end{bmatrix}_{\text{tangential}} = \begin{bmatrix} 2k_{\text{t1}}\ u'\ v' + k_{\text{t2}}(r^2 + 2u'^2) \\ k_{\text{t1}}(r^2 + 2v'^2) + 2k_{\text{t2}}\ u'\ v' \end{bmatrix}, \tag{2.17}$$

which is typically modelled using two additional terms, k_{t1} and k_{t2}. The radial and tangential distortion are therefore characterized by the five k terms. The total lens distortion is the summation of both distortion models and corrected by calibrating the camera and remapping the pixels accordingly.

Monocular Camera Calibration

The goal of monocular camera calibration is to obtain the five distortion terms (k_{r1}, k_{r2}, k_{r3}, k_{t1}, k_{t2}) and four intrinsic camera parameters (f_x, f_y, c_x, c_y). The camera extrinsic matrix is excluded from the calibration by referencing points relative to the camera coordinate frame. The camera calibration procedures presented in Zhang [25] and Tsai et al. [26] are well established, which use multiple photos of a planar calibration artifact to solve for the camera parameters. A checkerboard array is a common calibration pattern as it is easily detected with grid detection algorithms in Wang et al. [27] or He et al. [28] but any artifact with known detectible points $\mathbf{p}_a \in \mathcal{R}^2$ can be used.

Figure 2.4 displays the calibration procedure according to Zhang [25], where multiple images are first taken of the calibration artifact from different perspectives. Initially, it is assumed that there is no distortion present. For each view, pixel features $\mathbf{q}_a^{(i)}$ corresponding to key points ${}^a\mathbf{b}$ on the calibration artifact are detected in the image. Fig. 2.4 displays an example of key point detection on a checker board artifact. The known correspondence between the pixel features $\mathbf{q}_a^{(i)}$ and artifact points ${}^a\mathbf{b}$ are used to compute a homography matrix ${}^I\mathbf{H}_a^{(i)}$ using the DLT method. A series of homography matrices are computed from each view and decomposed to obtain a best fit camera intrinsic matrix ${}^I\tilde{\mathbf{K}}_c$ and a series of artifact locations ${}^c\tilde{\mathbf{T}}_a$. Initially distortion is neglected and the location of the pixel features on each image $\mathbf{q}_a^{(i)}$ do not match their re-projected location using equation (2.4) with the new intrinsic matrix ${}^I\tilde{\mathbf{K}}_c$ and the respective artifact location ${}^c\tilde{\mathbf{T}}_a^{(i)}$. The re-projection error is calculated by

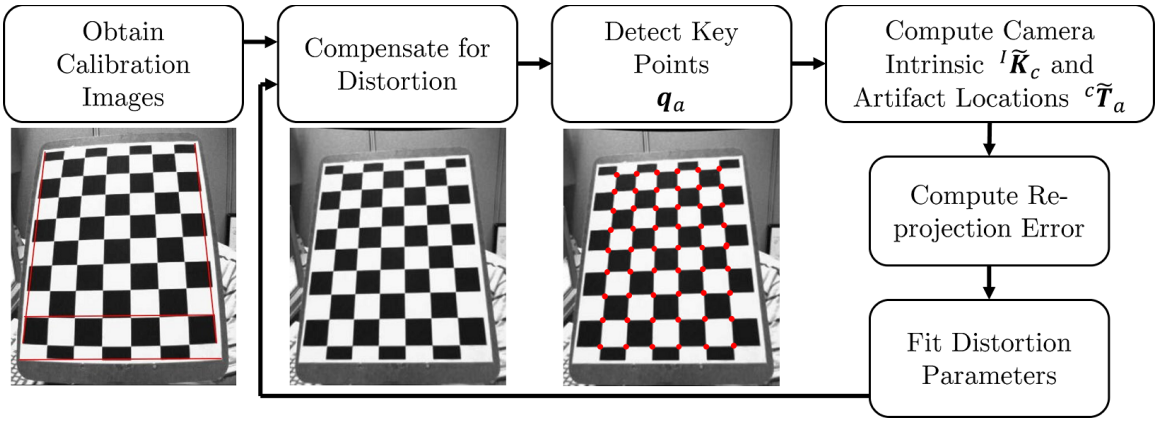


Figure 2.4: Camera calibration procedure described in Zhang [25].

measuring the distance between the re-projected points and their detected location. The distortion parameters are fit to minimize the re-projection error. The original images are remapped according to the new distortion parameters, and the procedure is iterated until an optimal solution is obtained. With the camera model obtained, one can model the perspective projection onto the camera and perform perspective transformations.

2.1.4 Orthorectification

Orthorectification is the process of removing the effects of perspective and surface elevation in an image, to create a to-scale representation of the surface, called an orthophoto. Orthorectification can be performed on 2D and 3D surfaces.

Planar Surfaces

Planar orthorectification will create a 2D image-space representation of a planar surface using planar homography. 2D Orthorectification is performed with the following procedure, reflected in Fig. 2.5.

1. Define the orthophoto plane and discretize it into an equally spaced grid of points ${}^{o}\mathbf{p}_{g}$ corresponding to pixel coordinates ${}^{o}\mathbf{q}_{g}$ in the orthophoto $\mathbf{I}_{o}$.

2. Detect the pixel location ${}^{\mathrm{I}}\mathbf{q}_{\mathrm{corner}}$ of the four external corners of the plane.
3. Define the homography matrix ${}^{\mathrm{I}}\mathbf{H}_{\mathrm{o}}$ between perspective of the camera, onto the orthophoto using the DLT method. The correspondences are from the four points on the plane in the first image ${}^{\mathrm{I}}\mathbf{q}_{\mathrm{corner}}$, onto the outer corners of the orthophoto image ${}^{\mathrm{o}}\mathbf{q}_{\mathrm{g}}$.

Figure 2.5 Image B displays the results of the orthorectification procedure. Orthorectification is expanded into 3D space using a 3D model of the observed surface.

Three Dimensional Surfaces

Three dimensional orthorectification is commonly performed in aerial photography to create maps relative to a true ground plane at a constant scale throughout the image. Orthorectification is performed with the following procedure reflected in Fig. 2.6.

1. Define the orthophoto plane and discretize it into equally spaced grid of points ${}^{\mathrm{o}}\mathbf{p}_{\mathrm{g}}$ corresponding to pixel coordinates ${}^{\mathrm{o}}\mathbf{q}_{\mathrm{g}}$ in the orthophoto $\mathbf{I}_{\mathrm{o}}$.
2. Project orthophoto grid points ${}^{\mathrm{o}}\mathbf{p}_{\mathrm{g}}$ in the normal direction to the plane, onto the 3D surface. The surface points are denoted as ${}^{\mathrm{o}}\mathbf{p}_{\mathrm{s}}$. The projected distance between the orthophoto plane and 3D surface are saved as a depth image $\mathbf{I}_{\mathrm{d}}$.
3. Project surface points ${}^{\mathrm{o}}\mathbf{p}_{\mathrm{s}}$ onto pixel features the image ${}^{\mathrm{I}}\mathbf{q}_{\mathrm{s}}$ using the perspective projection in equation (2.7).
4. Match the pixel intensity between the pixel features ${}^{\mathrm{I}}\mathbf{q}_{\mathrm{s}}$ and the orthophoto such that $\mathbf{I}[\ {}^{\mathrm{I}}\mathbf{q}_{\mathrm{s}}\ ] = \mathbf{I}_{\mathrm{o}}[\ {}^{\mathrm{o}}\mathbf{q}_{g}\ ]$.

Figure 2.7 displays the results of the orthorectification procedure. A requirement for 3D orthophotography is a 3D model of the surface relative to the camera, which must be captured using a 3D vision system.

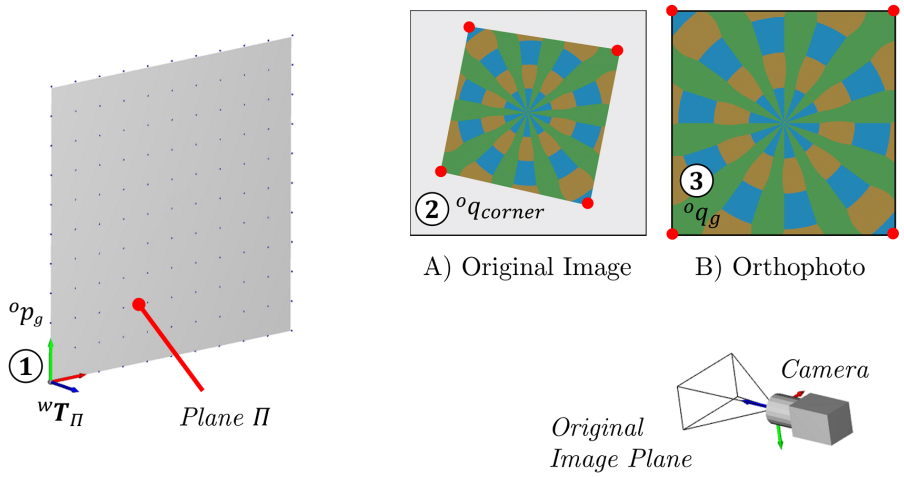


Figure 2.5: Orthorectification procedure using a monocular camera and a 2D surface.

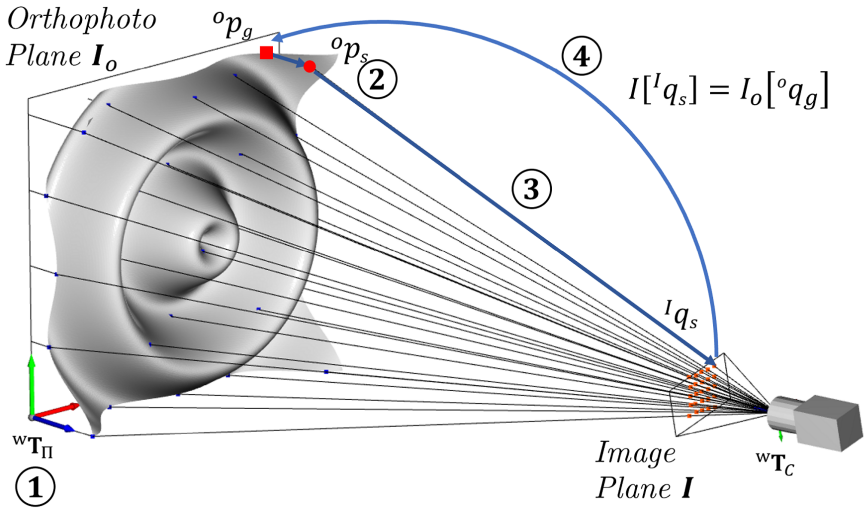


Figure 2.6: Orthorectification procedure using a monocular camera and model of the 3D surface.

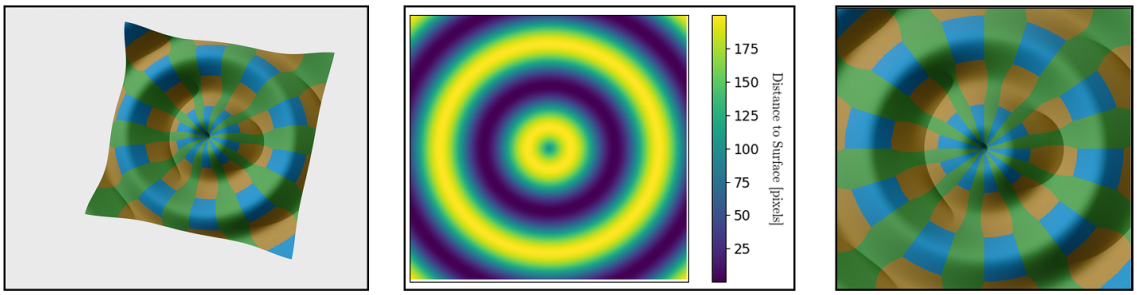


Figure 2.7: A) Orhorectification input image, B) calculated depth image, and C) orthophoto using the scene displayed in Fig. 2.6.

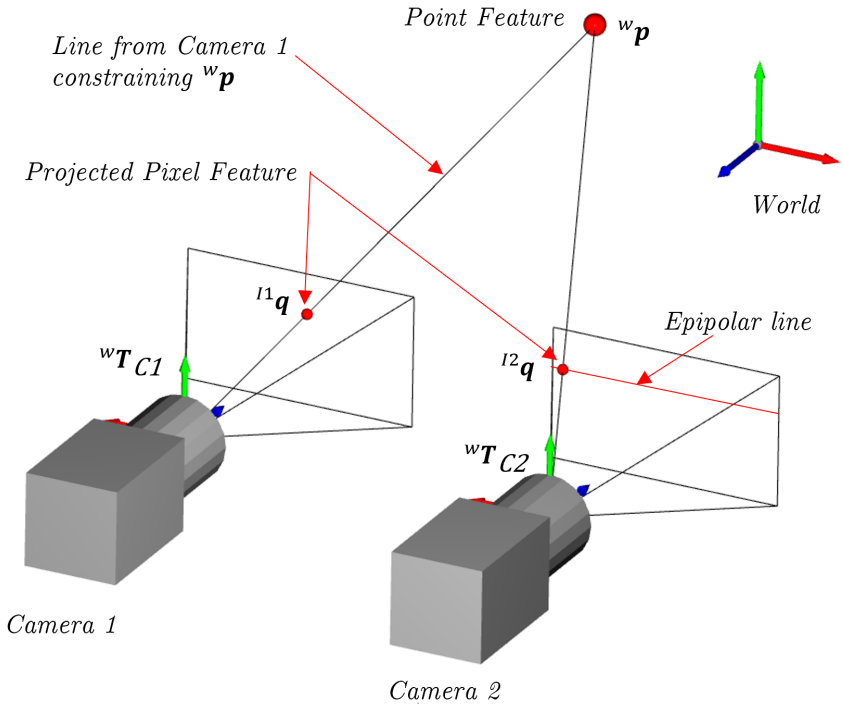


Figure 2.8: Stereo camera system composed of two parallel monocular cameras.

2.1.5 Stereo Vision

With a calibrated monocular camera, the projection of point features onto an image can be modelled, yet the inverse relation is ill defined. Point features can be located in 3D space by introducing a second monocular camera in a stereo camera configuration to perform triangulation. Figure 2.8 displays a stereo camera system triangulating a point feature $^{\mathrm{w}}\mathbf{p}$. The pixel feature $^{\mathrm{I1}}\mathbf{q}$ identified in camera 1 constrains the location of the corresponding point feature $^{\mathrm{w}}\mathbf{p}$ to a 3D line using the pinhole camera model. The point feature is triangulated by identifying its corresponding pixel feature $^{\mathrm{I2}}\mathbf{q}$ in camera 2, defining two lines in which the point feature is located. The feature's 3D location is identified as the intersection, or minimum distance, between the two lines. Identifying the location of a point feature using a stereo camera is referred to as triangulation, which is performed analogous to the DLT algorithm [23]. First note that the homogeneous vectors on the right and left-hand side of equation

(2.7) share the same direction thus,

$$\begin{bmatrix} {}^{\mathrm{I}}\mathbf{q} \\ 1 \end{bmatrix} \times \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right) = 0, \tag{2.18}$$

the cross product is expanded into the x, y and z components,

$$0 = {}^{\mathrm{I}}u \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{3T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right) - \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{1T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right), \tag{2.19}$$

$$0 = {}^{\mathrm{I}}v \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{3T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right) - \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{2T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right), \tag{2.20}$$

$$0 = {}^{\mathrm{I}}u \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{2T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right) - {}^{\mathrm{I}}v \left({}^{\mathrm{I}}\mathbf{M}_{\mathrm{w}}^{1T} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix} \right), \tag{2.21}$$

respectivly. out of equations (2.19), (2.20), (2.21), there are only two linearly dependant equations, therefore equation (2.21) is unused. An equation in the form of $\mathbf{Ax} = 0$ is composed using (2.19) and (2.20) with both cameras as,

$$0 = \begin{bmatrix} {}^{\mathrm{I1}}u\ {}^{\mathrm{I1}}\mathbf{M}_{\mathrm{w}}^{3T} - {}^{\mathrm{I1}}\mathbf{M}_{\mathrm{w}}^{1T} \\ {}^{\mathrm{I1}}v\ {}^{\mathrm{I1}}\mathbf{M}_{\mathrm{w}}^{3T} - {}^{\mathrm{I1}}\mathbf{M}_{\mathrm{w}}^{2T} \\ {}^{\mathrm{I2}}u\ {}^{\mathrm{I2}}\mathbf{M}_{\mathrm{w}}^{3T} - {}^{\mathrm{I2}}\mathbf{M}_{\mathrm{w}}^{1T} \\ {}^{\mathrm{I2}}v\ {}^{\mathrm{I2}}\mathbf{M}_{\mathrm{w}}^{3T} - {}^{\mathrm{I2}}\mathbf{M}_{\mathrm{w}}^{2T} \end{bmatrix} \begin{bmatrix} {}^{w}\mathbf{p} \\ 1 \end{bmatrix}, \tag{2.22}$$

where the solution to ${}^{w}\mathbf{p}$ is the singular vector corresponding to the smallest singular value of $\mathbf{A}$ [23].

Triangulation requires matching features between two images, which is referred to as stereo matching. Figure 2.8 displays an identified pixel feature ${}^{\mathrm{I1}}\mathbf{q}$ in camera 1, which constrains the location of the 3D feature ${}^{\mathrm{w}}\mathbf{p}$ onto a 3D line in space. The 3D line provided by the first pixel feature is projected onto a line the second image, called an epipolar line. The second pixel feature ${}^{\mathrm{I1}}\mathbf{q}$ is constrained along the epipolar line, reducing stereo matching to a one-dimensional search. Matching is performed by characterizing the pixel feature based on the local intensity in a template window

and selecting the closest matching window along the corresponding epipolar line [29]. Since the matching is simply based on the local pixel intensity, there will be poor results with low texture or repetitive patterns [30]. Stereo matching can be avoided by replacing the second camera with a structured projector, utilized in laser scanner systems.

2.1.6 Laser Scanner

Figure 2.9 displays a diagram of the off-the-shelf laser scanner system used on the Robodeb platform. The laser scanner performs triangulation using a monocular camera and a laser projector at a fixed location relative to each other. Figure 2.10 displays an in depth diagram of the triangulation of 3D features ${}^{\mathrm{w}}\mathbf{p}$ using a laser scanner. The laser projector can project one or many beams of light onto a scene in a known direction. Similar to the pinhole camera model, the laser beams are modelled as light rays travelling linearly in space. The laser illuminates points ${}^{\mathrm{w}}\mathbf{p}$ at the intersection between the beams and the surface. The monocular camera is used to detect the illuminated point features ${}^{\mathrm{I}}\mathbf{q}$ in an image. A pixel feature detected with a monocular camera constrains the point feature into a line in 3D space. Illuminated points are triangulated by finding the intersection, or minimum distance, between the detected line and the original laser beam, as analogues to the stereo camera. On the Robodeb platform, part localization can be performed using the laser scanner to obtain an accurate point cloud model of the part in the workspace. Using a registration procedure, a model of the part can be matched to the captured point cloud.

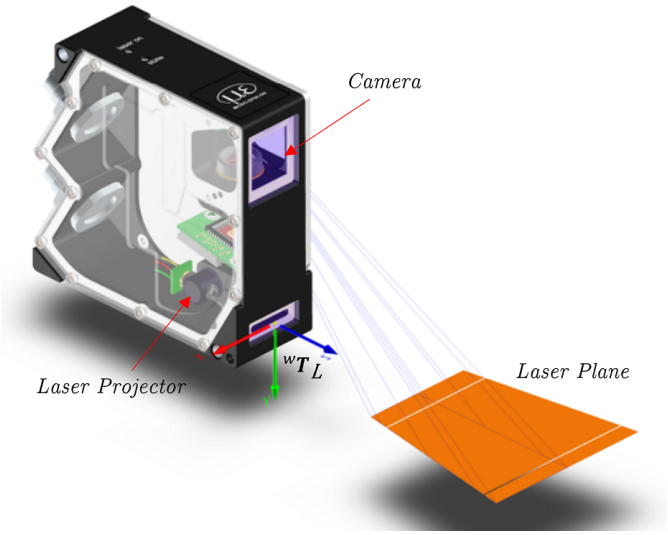


Figure 2.9: Micro Epsilon scanControl 29000 series laser scanner system. Adopted from [31].

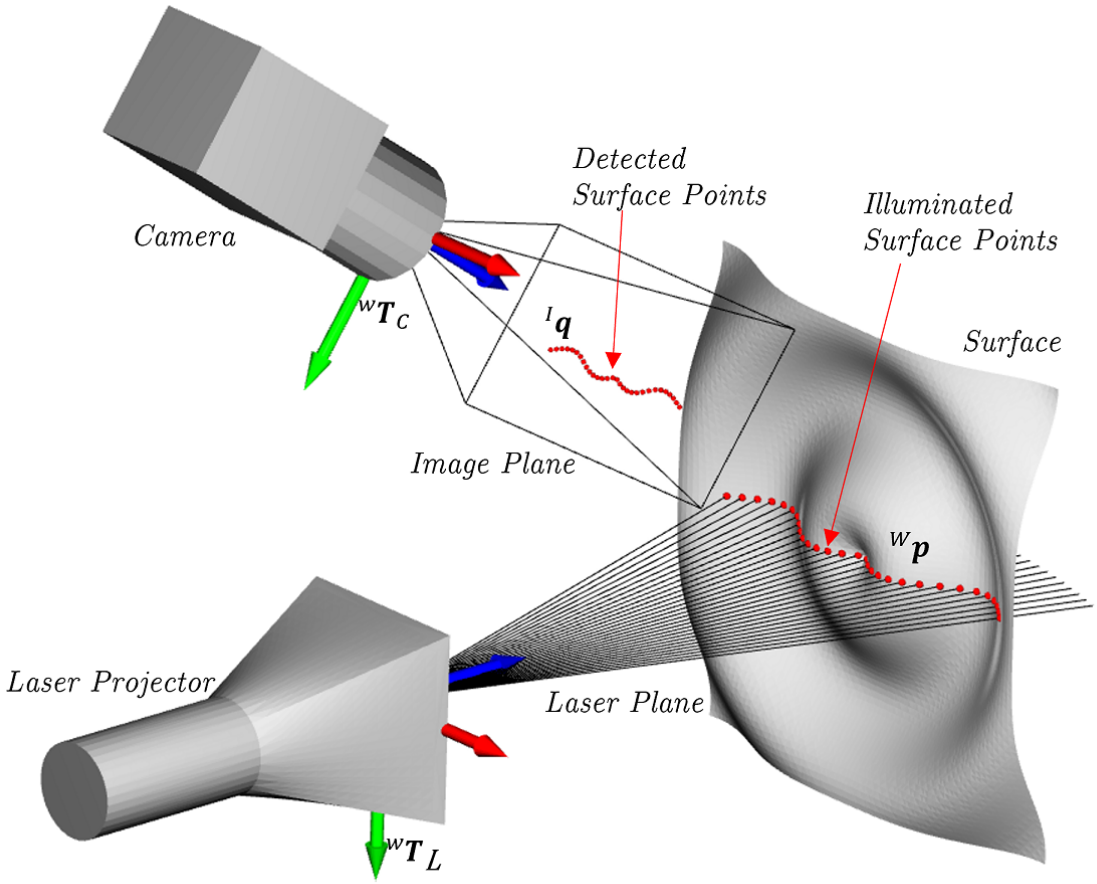


Figure 2.10: Laser scanner system composed of a laser projector and a monocular camera.

2.2 Point Cloud Registration

Point cloud registration is commonly used in part localization to match a reference point cloud, to a point cloud observed from the robot. Point cloud registration is commonly performed with an ICP algorithm. ICP algorithms require an initial guess for the alignment, which can be estimated using an RANSAC algorithm.

2.2.1 Iterative Closest Point

The ICP algorithm aligns two similar point clouds through calculating a transformation $\mathbf{T}$ that the best aligns the two point clouds. An ICP algorithm is an example of an optimization problem, where the error function minimized captures the overall distance between the two point clouds. The most basic error function used is the point-to-point error defined as,

$$E(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^{n} w^{(i)} \|\mathbf{h}^{(i)}(\mathbf{R}, \mathbf{t}) - \mathbf{p}_{\mathrm{t}}^{(i)}\|^2, \tag{2.23}$$

where $\mathbf{p}_{\mathrm{t}}$ is the target point cloud and w is a weighting term. The transformed source point cloud $\mathbf{h}^{(i)}(\mathbf{R}, \mathbf{t})$ is composed as,

$$\mathbf{h}^{(i)}(\mathbf{R}, \mathbf{t}) = \mathbf{R}\mathbf{p}_{\mathrm{s}}^{(i)} + \mathbf{t}, \tag{2.24}$$

where the rotation $\mathbf{R}$ and translation $\mathbf{t}$ are applied to the source point cloud $\mathbf{p}_{\mathrm{s}}$ [32,33]. The point-to-point error function $E(\mathbf{R}, \mathbf{t})$ is the sum of the squared Euclidean distance between the point clouds [32, 33]. The weighting term w may be set to unity for simple cases, but faster convergence may be achieved by weighting the correspondences $w^{(i)}$ based on local curvature [34]. The error function requires an estimation of the correspondences i of points between the point clouds. Figure 2.11 displays correspondences estimated based on proximity, which is adequate for small

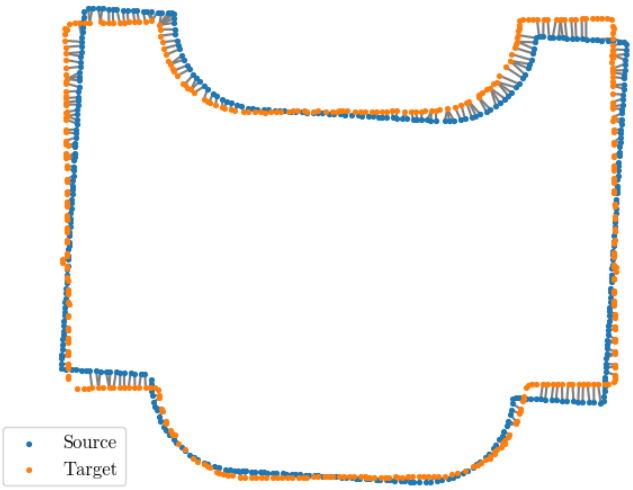


Figure 2.11: Correspondence drawn between a source $\mathbf{p}_\mathrm{s}$ and target $\mathbf{p}_\mathrm{t}$ point clouds.

perturbances, but may be improved by creating and referencing descriptions of points [35]. An improvement to the point-to-point error function is the point-to-plane error function defined as,

$$E(\mathbf{R},\mathbf{t}) = \sum_{i=1}^{n} w^{(i)} \| \overrightarrow{\mathbf{n}}^{(i)} \cdot (h^{(i)}(\mathbf{R},\mathbf{t}) - \mathbf{p}_\mathrm{t}^{(i)}) \|^2, \tag{2.25}$$

where $\overrightarrow{\mathbf{n}}^{(i)}$ is the normal direction to the point cloud. In the point-to-plane error function, the normal distance between the point clouds is minimized, which has been shown to achieve faster convergence [36, 37]. ICP algorithms require a good initial estimation of the alignment for convergence, which can be estimated with a RANSAC algorithm.

2.2.2 Random Sample Consensus

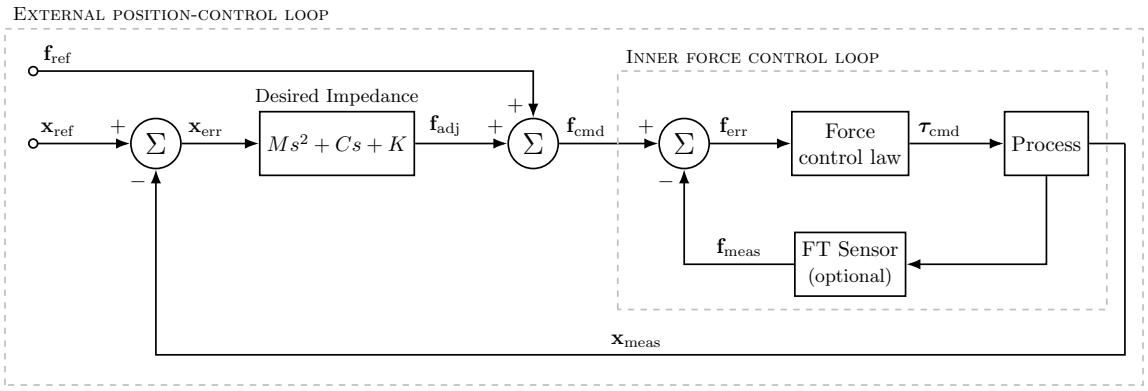
A Random Sample Consensus (RANSAC) algorithm estimates an approximate alignment between point clouds. RANSAC is useful for obtaining an initial guess for an ICP algorithm. RANSAC is an iterative method to estimate the transformation ${}^{\mathrm{s}}\tilde{\mathbf{T}}_t$ from the observed point cloud data using the following procedure [38]:

1. Randomly select a sample set of three points from the source point cloud.
2. Compute the transformation ${}^{\mathrm{t}}\tilde{\mathbf{T}}_{\mathrm{s}}^{(i)}$ between the sample set and the target point cloud using an estimated closet distance correspondence.
3. Score the alignment using ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{s}}^{(i)}$ with the point-to-point (2.23) error metric.
4. Repeat steps 1-3 until it is improbable of fining a better alignment.

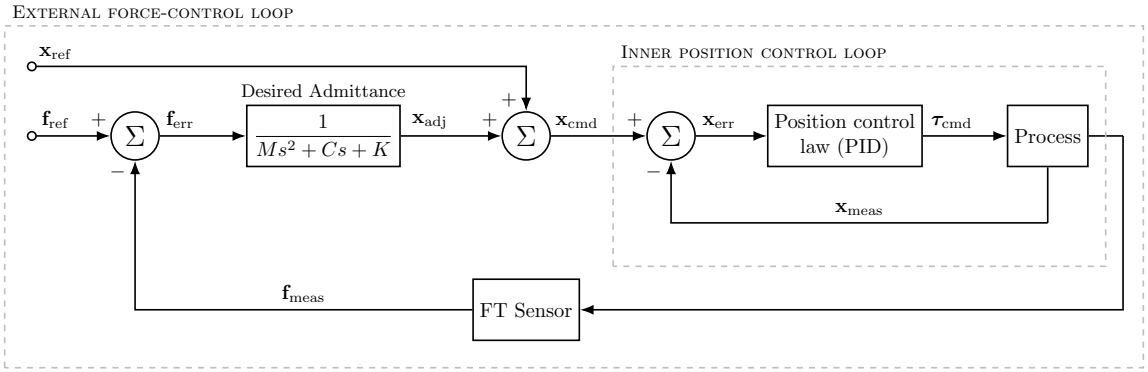
The transformation ${}^{\mathrm{t}}\tilde{\mathbf{T}}_{\mathrm{s}}^{(i)}$ which obtained the with the lowest error score is chosen as the best estimate of the alignment. Imperfect alignment after point cloud registration will cause errors in the perceved object location in the robot's workspace. The robot can track objects with imperfect alignments using a force controller to provide feedback on the contact between the robot and the part.

2.3 Force Control

In Dehghanitafti et al. [8] it was shown that an industrial robot could not archive sufficient accuracy to contact the edge of a part while machining with pure position control. The edge tracking accuracy of the robot can be improved using a force control scheme. Pure force control has stability issues and performs poorly with fluctuating contact forces, often displayed when machining and deburring. An improvement to pure force control is impedance or admittance control, which controls the relationship between the applied force and displacement from the desired path. Figure 2.12 displays the two force control schemes. The force-displacement relation is modelled as a second order mass M spring K and damper B system. During the interaction between the robot and the environment, one must impede or push, while the other admits or deflects. Impedance and admittance control are reciprocals of each other and seek to either control the position or the applied force. Impedance control uses an external position loop to convert path deviations to a desired force for a given impedance, while an internal loop controls the applied force. Admittance control uses an external



(a) An impedance controller.



(b) An admittance controller.

Figure 2.12: Representative block diagrams of a) impedance and b) admittance control architectures. Adapted from [39].

force control loop to convert force deviations into a deviation in the desired path for a given impedance, while the inner loop performs position control. Impedance control has the capability to exhibit any compliance required within the capabilities of the system. To implement impedance control properly, the non-linearities in the dynamic model of the robot must be accurately compensated for. In practice, no system is perfectly modelled, and disturbances will result in poor path tracking. Admittance control displays the inverse performance as it can track trajectories well, but encounters issues with interacting with high compliance environments, which causes limit cycle oscillations [39].

2.4 Summary

This chapter provided a background on the mathematical preliminaries required to introduce trajectory planning concepts in this thesis. The chapter begun with a background on computer vision, beginning with the perspective projection of a monocular camera using the pinhole camera model. Camera calibration using Zhang's method was presented to obtain the intrinsic parameters of the pinhole camera model. Orthorecrification was presented to obtain a perpendicular, to-scale, perspective of a 3D surface. The monocular camera concepts were extended into 3D vision systems, such as stereo cameras and laser scanners. Point cloud registration was introduced to align point clouds using an ICP algorithm to minimize a cost function relating to the distance between the point clouds. ICP registration requires an initial guess of the alignment provided by a RANSAC algorithm. Lastly, online force control schemes were described, useful for improving the edge tracking accuracy of the robot. Impedance and admittance control schemes were introduced which are reciprocal force control schemes that control the relationship between the applied force and displacement from the desired path. The preliminaries presented are next applied to investigate trajectory planning for robots in manufacturing.

Chapter 3

Robotic Manufacturing Background

The task of path planning is universal to all robotic applications, but the study in this chapter is concentrated on contour tracking in robotic machining. In robotic machining, contour tracking is present in processes such as deburring, chamfering, polishing etc. where the desired path is around the periphery of a workpiece with sustained contact. Path planning can be performed manually through teach and playback techniques or automated using an algorithm to design the path. The following chapter provides a concise review of literature studied on path planning for an industrial robot. In Section 3.1 automated path planning using Computer Aided Design (CAD) and Computer Aided Manufacturing (CAM) software is presented. In Section 3.2 computer vision path planning is presented, which uses a vision system to perceive the desired path.

3.1 CAD/CAM Software

The most common form of automated path planning uses CAD/CAM software to design a path in reference to a CAD model of the workpiece. The following section

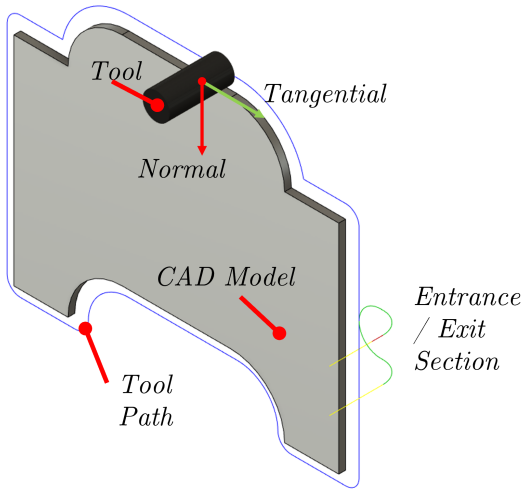


Figure 3.1: Contour following machining path generated using Fusion 360 [42] CAD/CAM software.

presents path planning using CAD/CAM software. In Section 3.1.1, path design using CAD/CAM software is explored. In Section 3.1.2 approaches for localizing the designed path in the workspace are presented. Section 3.1.3 explores modifications to CAD/CAM software to adjust to imperfections perceived on the workpiece.

3.1.1 Local Path Planning

CAD/CAM software is widespread in CNC machining, therefore there is an abundant of powerful, off-the-shelf solutions available. Leading CAD/CAM software includes Mastercam [40], NX CAM [41] and Autodesk Fusion 360 [42]. CAD/CAM software can design 2D or 3D milling, contour following and surfacing paths to suit a variety of applications. The literature reviewed by the author revealed that off-the-shelf or custom CAD/CAM software is the most popular approach for contour following path planning for robotic machining [1]. Figure 3.1 displays a contour following path designed around a test part created with Autodesk Fusion360. The tool path is offset to account for the radius of the cylindrical tool used.

Off-the-shelf CAD/CAM software designed for CNC machining may not address

chamfering or force control implementations possible with machining on an industrial robot. Therefore, prior research has developed custom CAD/CAM software or add-ons to address shortcomings. Asakawa et al. [43] designed custom CAD/CAM software for machining and chamfering the complex case of a hole in a free-curve surface with an industrial robot. Asakawa designed machining paths by identifying the desired edges in the CAD model and discretizing the path into a series of waypoints. The waypoints were selected with a position and tool pose which were interpolated between to define a trajectory.

Robotic machining requires sustained contact between the robot and the workpiece. An industrial robot with position control alone is not sufficient for consistent contact with the workpiece in a contour following application, therefore online force controllers have been introduced to improve the accuracy [8]. A force controller requires the designed path to contain a direction to apply force onto a workpiece, which is not available for most off-the-shelf CAD/CAM software. Jeon et al. [44] implemented a hybrid force and position controller in a robotic deburring application. Pure force control has stability issues and performs poorly with fluctuating contact forces, often displayed when machining and deburring. An improvement to pure force control is impedance or admittance control, which controls the relationship between the applied force and displacement from the desired path. In Kazerooni [45] and Song et al. [46] an impedance control scheme was implemented in the normal direction towards the workpiece and tangential direction along the tool path. The normal force controller maintained the tool's contact with the workpiece. The tangential force controller was used to slow down the tool on large burrs. Kazerooni and Song chose impedance control to maintain adequate performance when interacting in a ridged environment.

CAD/CAM software is currently the gold standard for automated path planning.

In this thesis, the PCAT algorithm is developed using a CAD/CAM software approach for path planning to establish a baseline performance. The PCAT algorithm is developed to design trajectories with an admittance controller. Admittance control is used on the RoboDeb platform as it provides high accuracy trajectory tracking, while deburring with a low compliance flap wheel tool. The path design for the PCAT algorithm is performed similar to Asakawa et al. by dividing a CAD model into waypoints and allows for the selection of deburring or chamfering paths.

3.1.2 Localize Part

CAD/CAM software designs a local path relative a CAD model, which must be transformed into the robot's workspace for playback. Localization can be performed manually by a skilled operator, or automated using the robot to sense the workpiece. The robot performs automated localization by feeling the workpiece with a touch-off procedure or seeing its location using a computer vision system.

Touch-off localization

Touch-off localization detects the location of the workpiece through physical contact recorded with a displacement sensor [47–49] or a force sensor [10, 11, 46]. In Murphy et al. [10], robotic deburring was performed in two passes. In the first path, the robot tracked the periphery of the workpiece using force control, deviating from the defined path as necessary. On the second pass, the robot followed the corrected path to deburr the workpiece. Murphy's approach requires a good initial estimate of the part location to the workpiece with a force control.

The initial estimate can be removed by directly teaching the robot touch-off points using a human operator [11, 46–48]. The robot's joint angles are recorded at each touch-off point, and the forward kinematics are used to calculate a point cloud from

the touched off positions. A model of the workpiece is fit to the point cloud obtained. In Song et al. [11], simple circle and square workpieces were localized using three to four touch-off points. The small sample of points sufficiently defined the simple geometries for localization. Small sample sets are prone to inaccuracies, therefore the same authors expanded their touch-off localization to sampling many points on more complex workpieces in Song et al. [46]. An ICP algorithm using the point-to-point metric was then used to fit a reference point cloud created from the CAD model to the observed touch-off data. The touch-off procedure was automated in Huang et al. [49] and Dehghani et al. [8], but required a good initial estimate of the part location, and specific path planning for the touch-off procedure on each part. Automated touch-off localization is infeasible in high-mix low-volume work, which is why 3D vision systems are often employed to perceive the workpiece.

3D Vision Localization

A computer vision system can quickly obtain a densely sampled and accurate point cloud model of the workpiece. Localization is again performed using an ICP algorithm to match between the observed point cloud and a reference point cloud created from a CAD model. In Kabir et al. [50], an Ensenso N10 stereo camera system was used to acquire a point cloud model of a workpiece for finishing operations. Stereo cameras must correctly match features between cameras to obtain an accurate point cloud, which is unreliable in low textured environments, common to robotic machining. The Ensenso N10 camera projected a high contrast texture onto the scene, to aide in stereo matching, and achieved a depth accuracy of $1-2$ mm. A stereo camera can obtain a depth image nearly instantaneously from a single perspective of the scene, allowing the camera to be either fixed in an eye-to-hand configuration with the camera external to the robot or located on the robot's end effector in an eye-in-hand configuration.

Laser scanners were utilized in several robotic machining papers reviewed [2,13,14,

51, 52]. Laser scanner systems provide an order of magnitude higher depth accuracy $.1 - .2$ mm and are more robust to outliers then a stereo camera [2]. A line or point laser scanner is usually restricted to eye-in-hand configurations, so the robot can manipulate the scanner and capture multiple perspectives of the workpiece.

The PCAT algorithm developed in this thesis requires a localization procedure, to transform the local path developed into the robot's workspace. The RoboDeb platform is equipped with both a laser scanner, stereo camera, and force sensor, therefore all localization approaches mentioned are possible. Touch-off localization is discarded as it requires a significantly longer time to collect a dense point cloud and complex path planning. 3D vision localization can be performed with the eye-in-hand laser scanner or the eye-to-hand stereo camera system. The laser scanner presents higher accuracy, while the camera system does not require a scanning pass. Both 3D vision systems will be explored for localization to determine the preferred solution. The point cloud obtained with a vision system captures manufacturing imperfections on the part, which can be corrected for, before performing the path.

3.1.3 Tool Path Correction

In CAD/CAM software, the path is designed relative to an ideal CAD model of the workpiece. In finishing, deburring, and rework applications, imperfections on the workpiece may significantly alter the desired machining path. The following papers investigate adapting CAD/CAM software to imperfections observed on the part. In Bearee et al. [13] a variation of the ICP algorithm was proposed to localize warped aircraft structures by subdividing the point cloud into different areas, based on the local curvature. ICP registration is performed on every sub region to better approximate the warped structure. After the registration, the overlapping sub-regions are smoothly connected, and the path is designed. In Kuss et al. [14], a series of ICP

algorithms were performed between the observed point cloud and several CAD models with varying dimensions within the part tolerances. The best match CAD model was determined based on the lowest resulting point-to-point error function (2.23) and the matching CAD model was used to design the path. Kuss's solution does not scale well into complex parts, as more tolerances mean exponentially more CAD models to check. In Villagrossi et al. [15] deburring was performed on cast parts with predictable burr locations, that often suffered from thermal warpage. The warpage was estimated by touching off test points in an area where there was expected to be no burrs, then compensated for by adjusting the path.

The tool path correction papers highlight a key drawback of CAD/CAM software; the path is only as accurate as the CAD model used. Tool path correction approaches to CAD/CAM software add to the complexity and process time of the system and are not explored in this thesis. Instead, the PIST algorithm will be created using computer vision itself to design trajectories.

3.2 Computer Vision Path Planning

Computer vision path planning uses a computer vision system to provide the necessary spatial information for path design. The path is designed relative to the detected model of the workpiece, directly adapting for different geometries observed. The common structure of computer vision path planners consists of capturing an image of the workspace, planning the path in the captured image-space, and transferring the path into the workspace. Section 3.2.1 describes approaches taken for image-space path planning and Section 3.2.2 describes the projection of the image-space path into the robot's workspace. This section will reference the field of robotic machining as well as robotic , which has been quicker to adopt computer vision path planning.

3.2.1 Image-Space Path Planning

The image-space path planning process is a multi-step procedure. Figure 3.2 displays the common structure of the image-space path planning process, using an example image provided from the RoboDeb platform. Image-space path planning begins with A) an image of the original workpiece then, B) preprocessing to remove noise, C) region of interest detection to remove unused background pixels, D) edge detection to identify the contour, and E) parameterization to record the path as a series of waypoints.

Pre-processing

A drawback of image-space path planning is that there is no reference model to compare the detected part against. Additionaly, image-space path planning approaches are often prone to noise and perform poorly in low contrast lighting conditions. Pre-processing involves filtering and equalizing the pixel intensity data before detection to remove noise and imperfections. A median filter or image smoothing with a Gaussian blur adequately removes imperfections perceived along the part's edge or salt and pepper noise. Poor and changing lighting may cause low and inconsistent contrast, which may be compensated for using an adaptive histogram equalization technique such as CLAHE [24]. Figure 3.2 B displays an example pre-processing step using a median filter. The PIST algorithm will employ median filtering to remove unwanted noise, but CLAHE equalization is not required as the RoboDeb platform has controlled lighting in the enclosure.

Region of Interest

An image provides an abundant amount of information, much of which is irrelevant to the task of identifying a path. A region of interest may be identified in an image

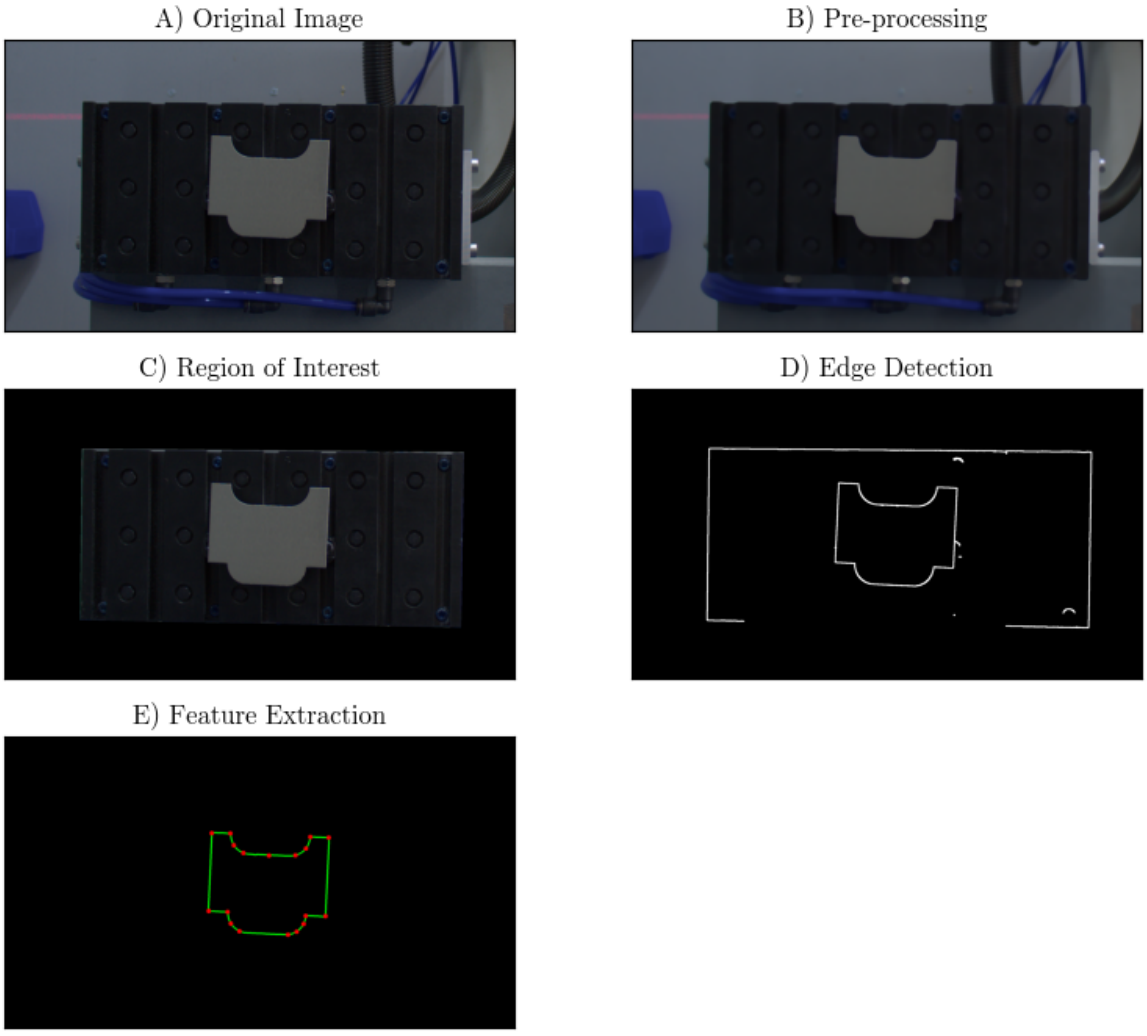


Figure 3.2: The common procedure of image-space path planning using an example image taken from the RoboDeb platform for illustration. A) the original image, B) preprocessing by applying a median filter to remove noise, C) region of interest detection to remove unwanted information, D) Contour detection with the Canny edge detector, and E) parameterization as a series of red waypoints.

using pre-emptive knowledge of the workpiece location, allowing the user to discard irrelevant data. In Dinham et al. [53] a region of interest was identified for robotic welding by detecting the corners of the rectangular workpiece in the image, and removing pixels external to the workpiece. Figure 3.2 C displays an example of the approach used in Dinham et al., where corners of the vacuum clamp were used to isolate the region of interest, and external pixels are masked off. The actual image size in Dinham et al.is not reduced, using an unnecessary amount of memory and time to

process the images. The PCAT algorithm uses a perspective transformation to isolate the region of interest. Chapter 2 Fig. 2.5 displays a perspective transformation for orthorectification, where the region of interest is the surface, which is transformed to occupy the entire image.

Contour Detection

Contour detection is concerned with detecting the desired path for tracking in an image. In the case of robotic welding, the contour is a seam to weld. In robotic welding, the seam can be identified using an edge detection algorithm. Edges in an image are designated as areas with high rates of change in pixel intensity. Edge detection algorithms use a discrete differentiation operator to highlight edges and threshold the highlighted image for detection. The Canny edge de is a popular edge detection algorithm and has been employed in various papers [18, 20, 21]. However, the focus of some robotic welding papers such as Kong et al. [54] Shah et al. [17] and Dinham et al. [55] was to create custom edge detection techniques specific for seam detection. In robotic machining the contour is around the periphery of the part. In Lai et al. [56] Canny edge detection is also used to detect the periphery of the workpiece. Figure 3.2 D displays edge detection on the RoboDeb platform. Canny edge detection selects all prominent edges in an image, making it unsuitable for high texture environments. Even in the RoboDeb environment, there is too much texture to use the canny edge detector alone. A variety of other approaches can be use to isolate the contour such as blob detection [57], or background subtraction [58]. Contour detection on the PIST algorithm will employ a background subtraction procedure, as it achieved the most consistent detection results.

Parameterize The Path

The last phase of image-space path planning is to parameterize the image-space path along the detected contour. Parameterization is performed by defining a series of ordered waypoints along the edge which are interpolated between to obtain a trajectory. In Lai et al. [56], detected edges were segmented into linear sections using the Hough transform. Figure 3.2 E displays the results of path parameterized as a series of linear points, where pixel waypoints are drawn in red and the interpolated path is in green. Linear interpolation introduces discontinuities in the path at waypoints, where the direction changes abruptly. Higher order approximations of the path may be employed to remove sharp transitions in the trajectory, and achieve higher positional accuracy with fewer waypoints. Solvang et al. [20] and Chang et al. [18] used Bezier curves and splines respectively, to describe the path. The PIST algorithm will linearly interpolate between points, as it is the simplest approach.

Computer vision path planning so far has been the most popular in robotic welding, in which the welding path is simply the position along a detected edge. Jinno et al. [19] used image-space path planning to perform robotic deburring using a force control scheme. Force control requires a direction to apply force onto the workpiece, but the selection of the direction was not discussed in detail. The author of this thesis did not find any papers discussing the selection of the applied force direction along an image-space path. Another difficulty in robotic deburring is that the desired tool path is offset outward from the detected contour to account for the radius of the deburring tool. In Jinno et al. [19], the offset was described but not addressed. In Lai et al. [56], the offset was performed manually using CAD/CAM software. The PCAT algorithm will perform image-space path planning with a comprehensive solution for designing an offset path with an applied force direction.

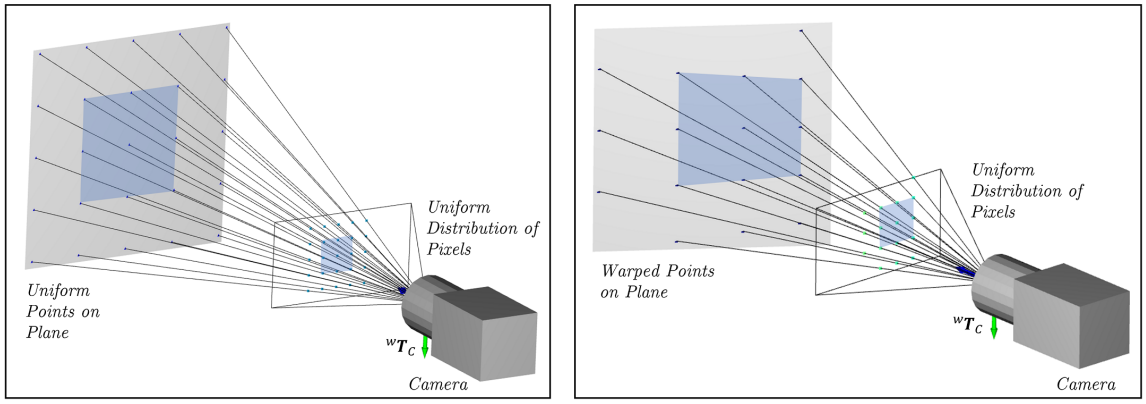
3.2.2 Transfer the Path into Workspace

The image-space waypoints created must be transformed into the robot's workspace for playback. Depending on the information available and restrictions imposed, the computer vision path planners perform the image to workspace mapping differently. The following section begins with a review of approaches taken to transform image-space trajectories onto planar surfaces and then is generalized to 3D surfaces.

Planar Surfaces

Section 2.1.1 describes the projection of a pixel or image-space waypoint into a 3D line in the workspace, using the pinhole camera model. Figure 3.3 displays the linear projection of pixel coordinates onto a plane in the workspace. The intersection between the line and the plane can be computed to locate the waypoint in 3D space. To map the image-space waypoints into the workspace, the location of the plane relative to the camera must be known or identified. In Jinno et al. [19], Micallef et al. [16] and Shah et al. [17] the plane was assumed to be perfectly perpendicular to the monocular camera used, at a known distance away. The assumption requires the camera imager to be placed perfectly perpendicular to the plane, which is not feasible. Assuming the camera pose is unnecessary, and avoided in Lai et al. [56] and Chang et al. [18] who identified the planar surface relative to the monocular camera using planar homography.

The perpendicular perspective is preferred for path planning, as there is uniform scaling between the image and the planar surface. Figure 3.3 A and B display the projection of pixels in a perpendicular and angled perspective of a planar surface, respectively. With an angled perspective, the projected pixels are warped on the surface, while with a perpendicular perspective, the uniform distribution is maintained. The uniform scaling allows for paths to be designed relative to the detected contour



A) Perpendicular Perspective of Plane

B) Angled Perspective of Plane

Figure 3.3: Projection of uniformly distributed pixels onto planar surfaces at A) a perpendicular perspective and B) an angled perspective.

in image-space and projected on to the planar surface. The PCAT algorithm uses a monocular camera to design offset paths in a perpendicular image-space. With an angled perspective, the offset would be warped when projected onto the surface. Section 2.1.2 describes orthorectification using planar homography, used to obtain a perpendicular to-scale perspective of the surface. Using the known scaling factor, pixel waypoints are simply scaled from the image onto the surface in the workspace. The orthorectification procedure introduced is also expanded into 3D surfaces.

3D Surfaces

Computer vision path planning is not restricted to planar surfaces. Unlike the planar case, a monocular camera alone cannot map an image-space path onto a 3D surface. The mapping must now be performed using a 3D model of the surface in the form of a CAD model or a point cloud. Figure 3.4 describes the projection of pixel points onto a 3D surface at a known location. In Solvang et al. [20, 21] 3D path planning was proposed for deburring using a monocular computer vision system. Deburring paths were drawn onto a 3D workpiece with a marker. The marked path was the desired path for the robot to follow. The marked path was detected in an image, then projected onto a CAD model of the workpiece localized relative to the

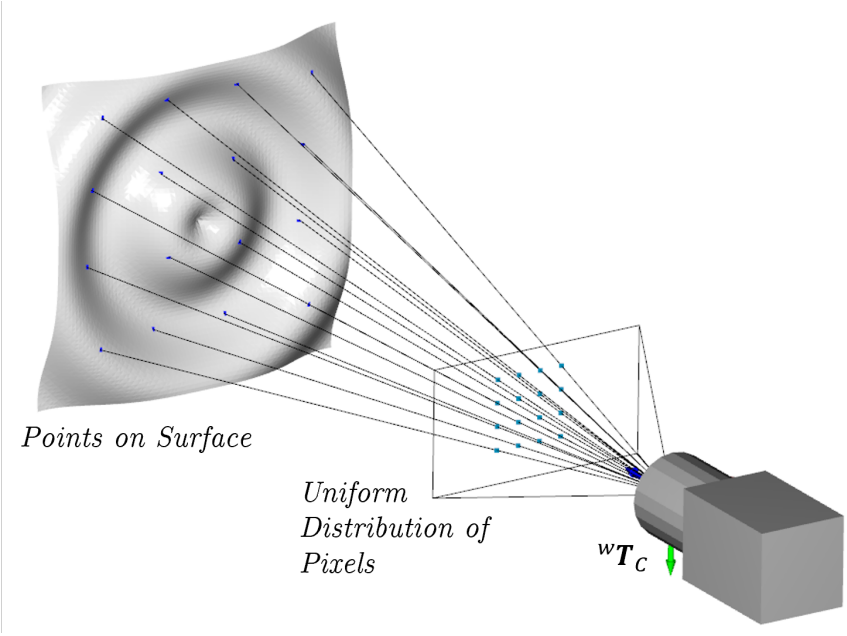


Figure 3.4: Projection of uniformly distributed pixels onto a 3D surface.

camera, similar to Fig. 3.4. The drawback of Solvang's approach is the re-introduction of a CAD model and localization procedure. A 3D vision system can capture a point cloud model of a workpiece. Tsai et al. [26] performed path planning for welding a golf club by scanning the club with a laser scanner and identified the location of the desired weld seam. In Pagano and Nakhaeinia [22, 59] contour tracking was performed with a Kinect RGB-D camera to generate both 2D and 3D paths. The contour was detected in the depth image by masking pixels with a depth value greater than the expected distance to the workpiece. Edge detection was used on the 2D mask to identify pixels on the external contour. The image-space path is projected back into the workspace by referencing the depth image. The PCAT algorithm will be expanded into 3D using a 3D vision system to capture a point cloud model of the part. Instead of performing the path planning on a point cloud or depth image, the 3D orthorectification procedure presented in Section 2.1.2 is used to create a to-scale image-space representation of the 3D surface for image-space path planning.

3.3 Summary

This chapter reviewed machining path planning using CAD/CAM software and computer vision. CAD/CAM software designs paths relative to a CAD model of the workpiece. CAD/CAM software is widespread in industry, with powerful off-the-shelf solutions available. In this thesis, the PCAT algorithm is developed, using a CAD/CAM software approach to provide a benchmark for future trajectory planners developed. Localization is performed using both the laser scanner and camera system to evaluate each vision system's performance. A drawback of CAD/CAM software is that the designed path does not account for deviations from the CAD model. The CAD model is avoided in computer vision path planning, which uses a computer vision system to provide the spatial information for path design. In this thesis, the PIST algorithm is designed using computer vision to create contour following trajectories similar to the PCAT algorithm. The PIST algorithm performs 2D and 3D path planning in a geometrically corrected image-space obtained using orthorectification.

Chapter 4

RoboDeb Platform

The RoboDeb test platform is designed for robotic deburring experimentation. The following chapter will discuss the infrastructure of the RoboDeb platform illustrated in Fig. 4.1. Section 4.1 describes the industrial robot central to the Robodeb platform and its custom end effector package. Section 4.2 details the software framework developed on the host PC for planning and control of the robot. Section 4.3 specifies equipment located in the robot's workspace. Section 4.4 describes the computer vision systems used for perception.

4.1 Robot Manipulator

The Robodeb platform is displayed in displayed in Fig. 4.2, built around a 6 DOF Denso VS-6556w serial manipulator robot. The VS-6556w robot has a length of 0.565 m to provide a large workspace with a repeatability of ± 0.02 mm for precise machining. The payload of the robot is 7 kg which is sufficient to manipulate the end effector and withstand machining loads. The robot is controlled by an open architecture control box from Quanser. Equipped to the robot is a specialized end

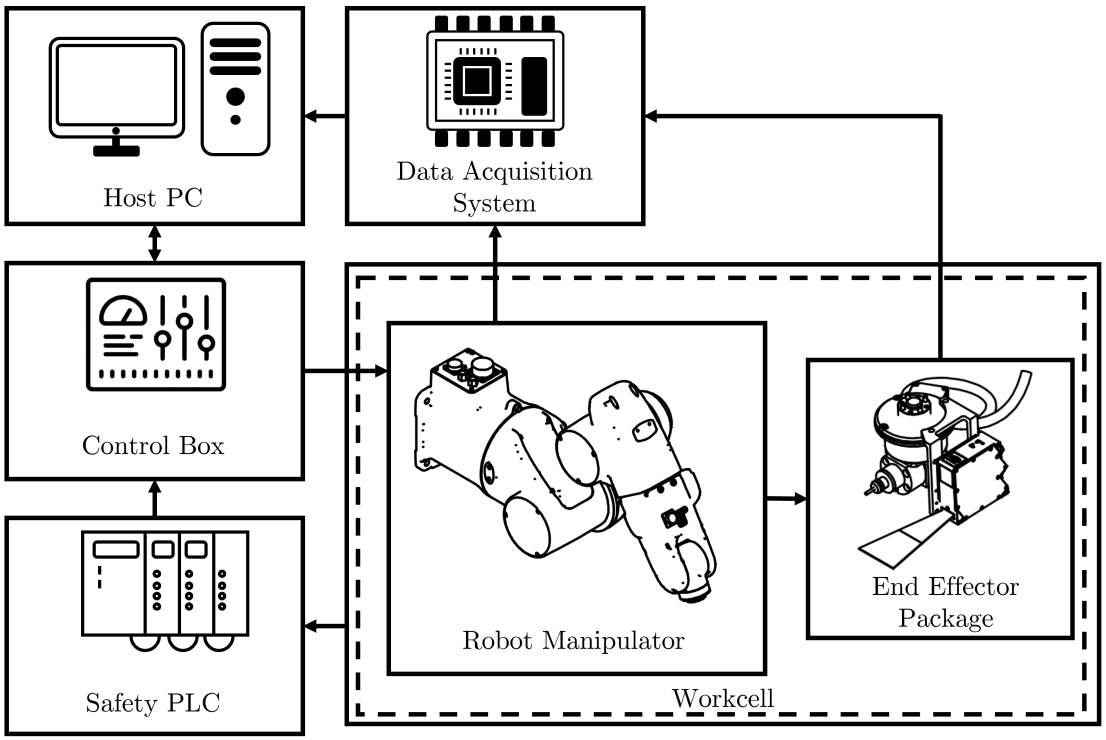


Figure 4.1: RoboDeb platform diagram adopted from [7].

effector package displayed in Fig. 4.3, composed of a Nakanishi NR-3060S spindle, ATI Gamma force-torque sensor and Micro-Epsilon scanCONTROL laser scanner. Deburring is performed with an abrasive tool mounted on the spindle. The preferred tool for deburring is an 3/8 inch diameter abrasive 120 Grit flap wheel tool. The flap wheel tool has a fine abrasive that can adequately remove burrs and imperfections while achieving a smooth surface finish. The abrasive flaps on the flap wheel tool erode as it is used, exposing fresh abrasive sections of the tool. Figure 4.1 displays communication between components on the RoboDeb platform. the Sensory feedback from the robot and end effector is collected by a data acquisition system and streamed to the host PC running the robot control logic.

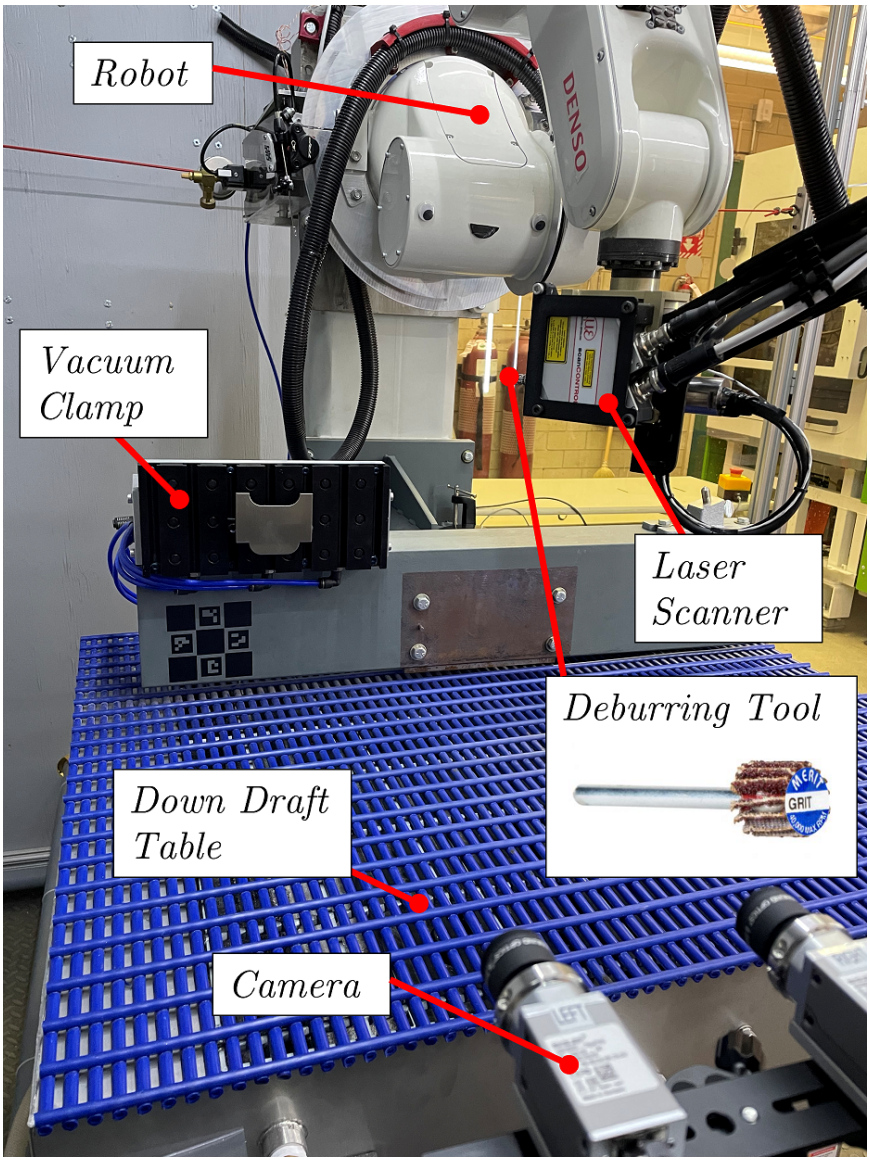


Figure 4.2: RoboDeb deburring platform.

4.2 Host PC

The RoboDeb host PC performs online control logic and offline trajectory planning. The high-level trajectory planners investigated in this thesis are run on the host PC to generate a trajectory, parametrized by a series of waypoints in the robot's workspace. The series of waypoints are interpolated for playback using the low-level trajectory planner using quintic spline interpolation to generate the complete

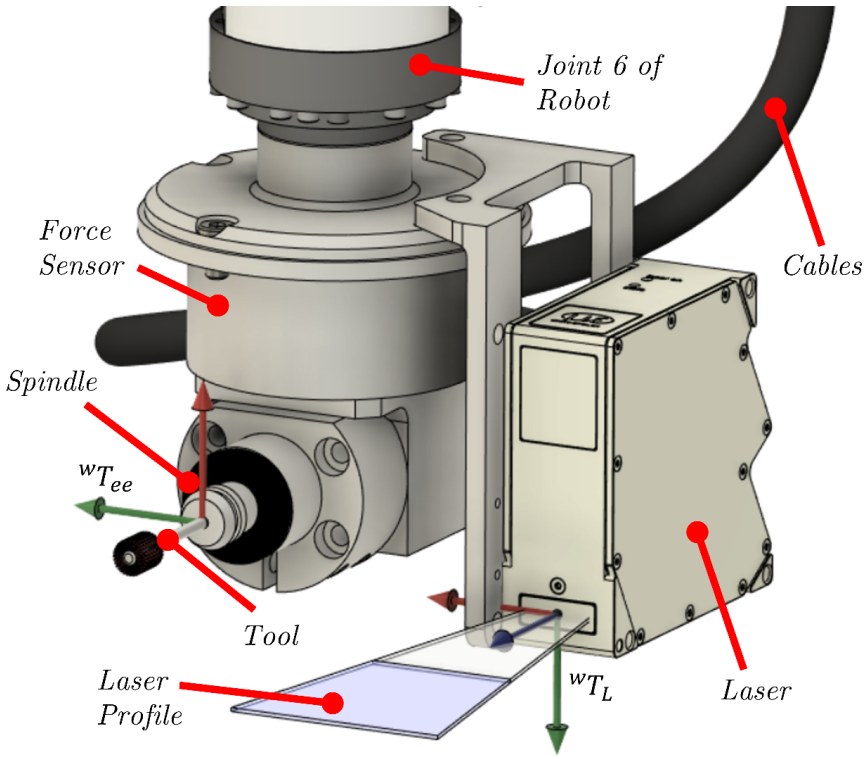


Figure 4.3: RoboDeb end effector package.

Cartesian trajectory with smooth accelerations. The low-level trajectory planner was developed by Steffan Lloyd. Once the path has been generated in Cartesian space, it is converted into a series of joint angles using the inverse kinematics procedure presented in Lloyd et al. [6]. Robot kinematics require a precise calibration of the robot geometric parameters for accurate operation. The geometric parameters have been locally calibrated to a positional accuracy of $\pm.15$ mm by as presented in Dehghanitafti et al. [8]. The local calibration is performed in the workspace around the vacuum clamp for the utmost accuracy. The generated joint-space trajectory is executed in the robot control logic contained in a Simulink model. The Simulink model implements the admittance controller, using force feedback provided by the force sensor. The admittance controller requires an estimation of the robot's dynamic parameters, which are modelled according to Lloyd et al. [5]. Admittance control performs poorly in high compliance environments, driving the selection of a compliant flap wheel tool.

The control logic outputs a series of joint positions in real time to the Quanser control box which actuates the robot's motion through the workspace.

4.3 Workspace

The robot's workspace is enclosed by a cell, which forms a physical barrier between the robot and its operators. The cell is equipped with a safety system controlled by a Reer M1 safety PLC and includes various emergency stops and sensors to turn off the robot in unsafe operation [7]. Within the cell is a Diversitech Monsoon wet downdraft table to remove dust and debris from machining. Figure 4.2 displays the downdraft table positioned below the vacuum clamp designed by the author of this thesis. The custom vacuum clamp is shown in Fig. 4.4, which is a hybrid fixturing solution that can hold parts through suction (method 1) or mechanical fastening (method 2). Fixturing the part through suction is ideal for robotic deburring as it allows access to all edges on the part. The vacuum is supplied via an external pump and transferred through internal cut outs in the clamp and sealed to the bottom of the part with a vacuum pod. Parts can be fastened to the base of the clamp, which serves as a versatile T-slot fixturing plate. The vacuum clamp is localized relative to the robot with a transformation ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{cb}}$ attached to the bottom left corner of the clamp base. Parts are placed on the vacuum clamp and therefore constrained to the clamping plane Π_{cp} with coordinate frame ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{cp}}$, which is normal to, and offset in the clamp base z axis. Figure Fig. 4.5 displays a point cloud of a test part on the vacuum clamp with the clamp base and clamping plane coordinate frames drawn in for reference. A stereo camera system is placed behind the downdraft table to provide a perspective of the vacuum clamp.

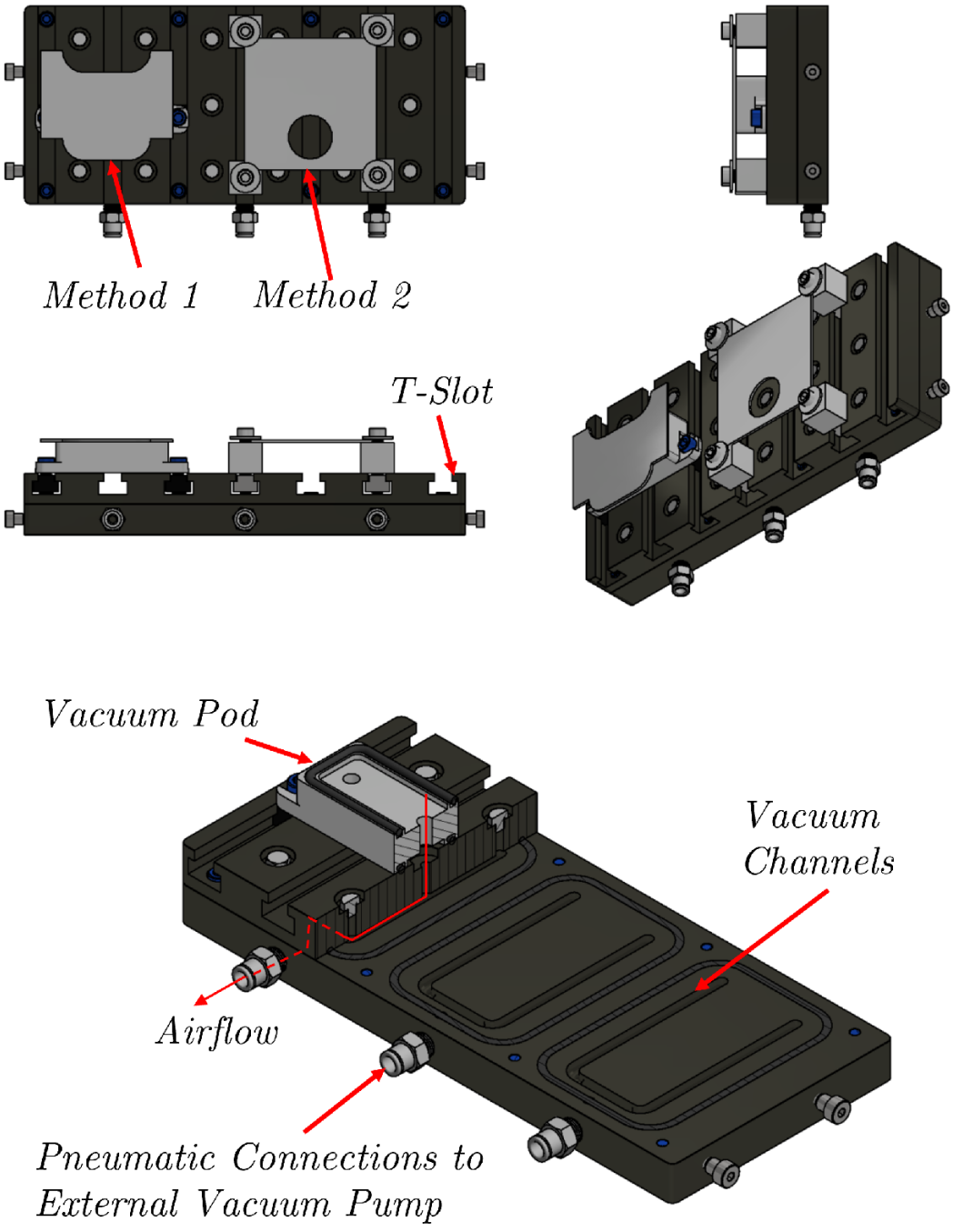


Figure 4.4: Vacuum clamp design.

4.4 Computer Vision

The RoboDeb platform uses two computer vision systems for perception. Equipped to the end effector of the robot is a laser scanner in an eye-in-hand configuration and a stereo camera in an eye-to-hand configuration.

4.4.1 Laser Scanner

The laser profile scanner is displayed in Fig. 4.3 which provides measurements along a 2D laser profile with a resolution of 0.04 μm. When triggered, the laser scanner collects points along a single laser profile consisting of 1280 points ${}^{\text{l}}\mathbf{p}$ in the laser coordinate frame. The points are transformed into the world coordinate frame using the transformation ${}^{\text{w}}\mathbf{T}_{\text{L}}(\Theta)$ computed from the forward kinematics using the calibrated robot parameters and current joint angles Θ. A large 3D point cloud is obtained by collecting many laser profiles while moving the laser scanner around an object. A sample point cloud of a test part is displayed in Fig. 4.5 with the vacuum clamp drawn in for reference.

4.4.2 Stereo Camera

External to the robot's workspace is a fixed stereo vision system comprising two Basler a2A3840-13gcPRO cameras in a parallel configuration. The cameras are calibrated to correct for lens distortion and calculate the cameras' intrinsic ${}^{\text{I}}\mathbf{K}_{\text{c}}$ and extrinsic ${}^{\text{c}}\mathbf{T}_{\text{w}}$ parameters. The RMS re-projection error after the camera calibration is 0.15 pixels. The stereo camera system can quickly obtain a point cloud by capturing images and triangulating features through stereo matching. A pair of stereo images of a test part is presented in Fig. 4.6 with the corresponding triangulated point cloud displayed in Fig. 4.7. The surface of the part has low texture, therefore stereo matching is only successful along the edges. The stereo matching results are

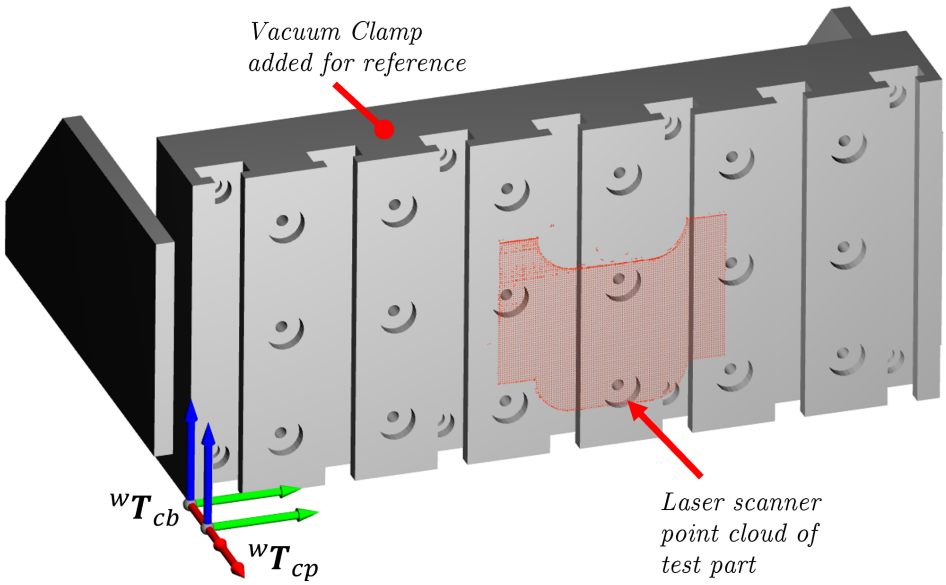


Figure 4.5: Laser scanner point cloud of a test part.

poor along horizontal edges, as the epipolar lines are horizontal in the parallel camera configuration, providing low texture on parallel edges.

An application of the Robodeb platform is to deburr planar sheet metal parts. Here, a monocular camera alone is sufficient for localizing pixels on a known plane. The monocular camera setup is displayed in Fig. 4.8. To map between the detected part in the image and the clamping plane, a homography matrix ${}^{\text{cp}}\mathbf{H}_{\text{I}}$ is defined using the DLT method. The correspondences are between the four corners of the clamping plane ${}^{\text{cp}}\mathbf{p}_{\text{corner}}$ and their projection onto the image ${}^{\text{I}}\mathbf{q}_{\text{corner}}$. Pixels on the detected part are projected onto the clamping plane using equation (2.12) with the defined homography matrix ${}^{\text{cp}}\mathbf{H}_{\text{I}}$.

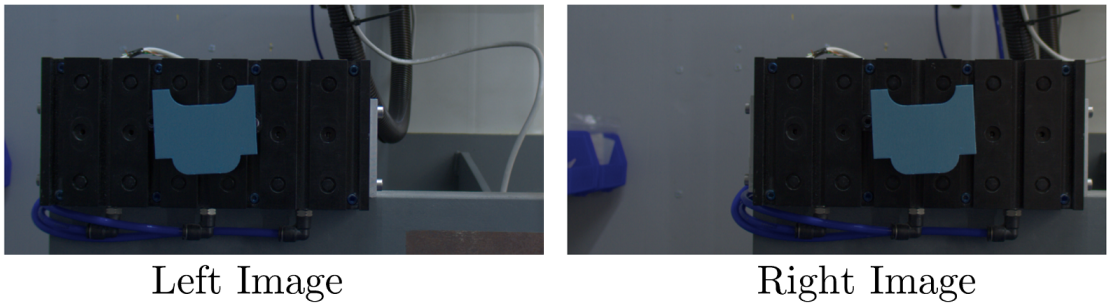


Figure 4.6: Stereo image pair of a test part.

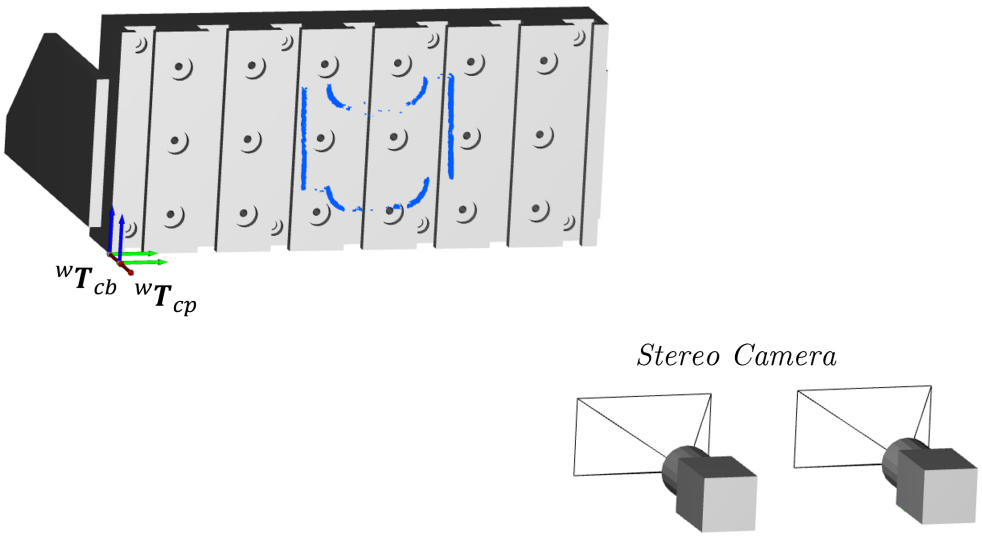


Figure 4.7: Point cloud generation using stereo matching on the images from Fig. 4.6.

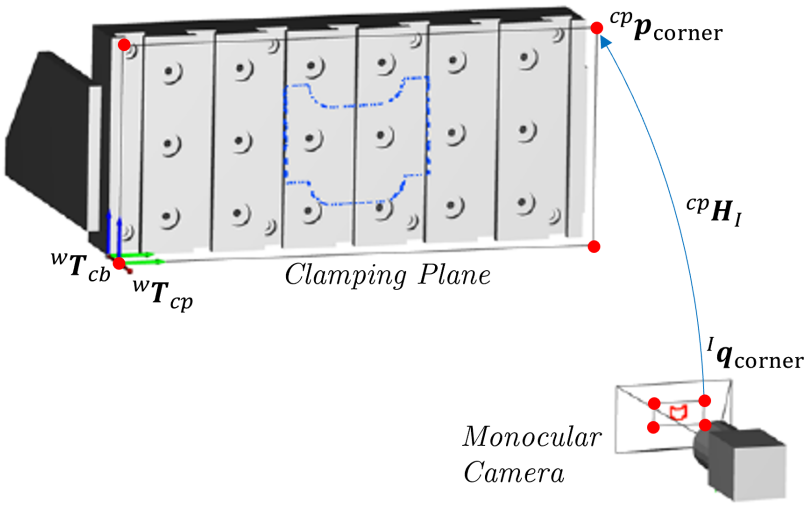


Figure 4.8: Point cloud generation using a monocular camera and a known plane.

4.5 Summary

This chapter described the Robodeb platform designed for robotic deburring experimentation. The Robodeb platform uses a 6-DOF Denso robot with an open architecture control box. Equipped to the end effector of the robot is a spindle, force sensor and laser scanner. The robot can perceive objects in the workspace using either the laser scanner or a stereo camera system. Trajectory planners investigated in this thesis outputs a series of waypoints, which are interpolated to obtain a Cartesian trajectory. The Cartesian trajectory is then mapped into joint space using the inverse kinematics and executed in the robot control logic, including an admittance controller. The robot's workspace is enclosed by a cell with a safety system. In the workspace, there is a custom vacuum clamp used to hold parts for deburring and a downdraft table below to remove metal dust and debris. The trajectory planning and controls of the robot are performed on a host PC. This setup and equipment allow for the development and testing of new trajectory planners for robotic deburring.

Chapter 5

Planar Computer Automated Trajectory Planner

The current benchmark for trajectory planning is CAD/CAM software, which designs a trajectory with reference to a CAD model of the workpiece. The following chapter will develop the *Planar Computer Automated Trajectory* (PCAT) planning algorithm using a CAD/CAM software approach. The PCAT algorithm is used in this thesis to establish a benchmark for trajectory design for robotic machining. The PCAT algorithm requires the workpiece to be localized in the workspace, performed using a computer vision system for ease of automation. The PCAT algorithm is first described in Section 5.1 which creates contour following trajectories for robotic machining and outputs a series of waypoints for the robot to playback. Section 5.2 details the PCAT implementation on the RoboDeb platform.

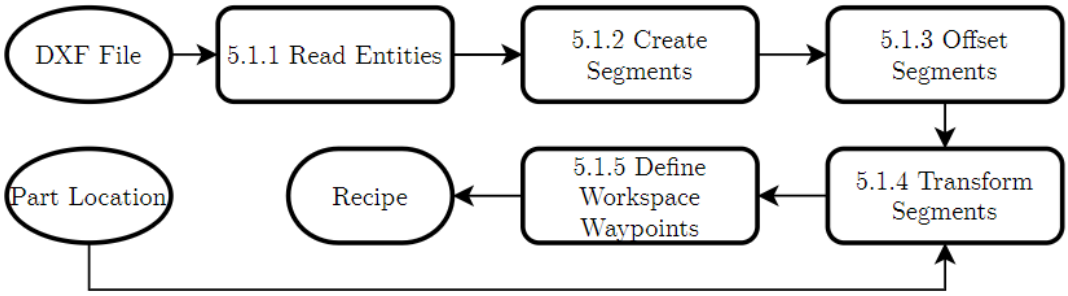


Figure 5.1: PCAT algorithm workflow.

5.1 PCAT Algorithm Description

The PCAT algorithm designs contour following trajectories for robotic machining using a CAD model for path design and computer vision for localization. Figure 5.1 displays the workflow of the PCAT algorithm. The trajectory planner begins with a DXF file of a planar workpiece. In Section 5.1.1, the file is read as a series of geometric primitives detailing the model, which are converted into entity objects. Section 5.1.2 orders the entities, and categorizes them into closed contours saved as segment objects. In Section 5.1.3, the tool path is created by offsetting the segments. Section 5.1.4 transforms the tool path into the robot's workspace based on the workpiece location. Finally, in Section 5.1.5, waypoints are generated from the ordered entities to describe the end effector trajectory.

5.1.1 Read Entities

The PCAT algorithm first reads the DXF file using the DXFTool function from [60]. The output of the DXFTool function is an unordered list of 2D geometric primitives, which describe the geometry of the workpiece. Typical geometric primitives are: lines, arcs, circles, polygons, splines, etc., all of which are converted into one or many line or arc entities. Figure 5.2 A, displays the unordered entities composing the

original DXF file. Line entities are described by a start and end point. Arc entities are described by a center point, radius and start and end angle. Entity objects are described in 3D space by assuming a zero z dimension, and a normal vector along the positive z axis, perpendicular to the xy plane of the 2D DXF file. Each entity is assigned a transformation at its start ${}^{\text{CAD}}\mathbf{T}_{\text{E1}}$ and end ${}^{\text{CAD}}\mathbf{T}_{\text{E2}}$ point, where the x axis is along the direction of travel and a z axis along the normal vector. The end point transformation is displayed on each entity in Fig. 5.2 A as an axis at the end point. The entities initially define the workpiece geometry, but will also be used to represent the tool path designed.

5.1.2 Create Segments

A segment object is composed of an ordered list of entities forming a close loop that can be machined without the tool leaving the surface of the workpiece. Figure 5.2 B displays the test part with the entities arranged into one ordered segment. The entities are organized by matching the entities' start and end points and sequentially ordering them tip to tail. A closed loop is obtained when the end of a matched entity, matches the beginning of the first entity in the segment. Once a closed segment is made, it is saved and the ordering process continues until all entities are accounted for. Once the segments are formed, the start point is defined as the midpoint of the largest entity. To smoothly transfer onto and off the workpiece, an entrance and exit arc objects are added.

A part with interior cut-outs will have multiple segments. Since each DXF file is assumed to represent one part, the segment with the largest enclosed area is denoted as the exterior, while the interior segments are cut-outs. The area of each segment is estimated using the shoelace algorithm [61] which computes the signed area of a polynomial from an ordered list of vertices, where a positive sign corresponds to a

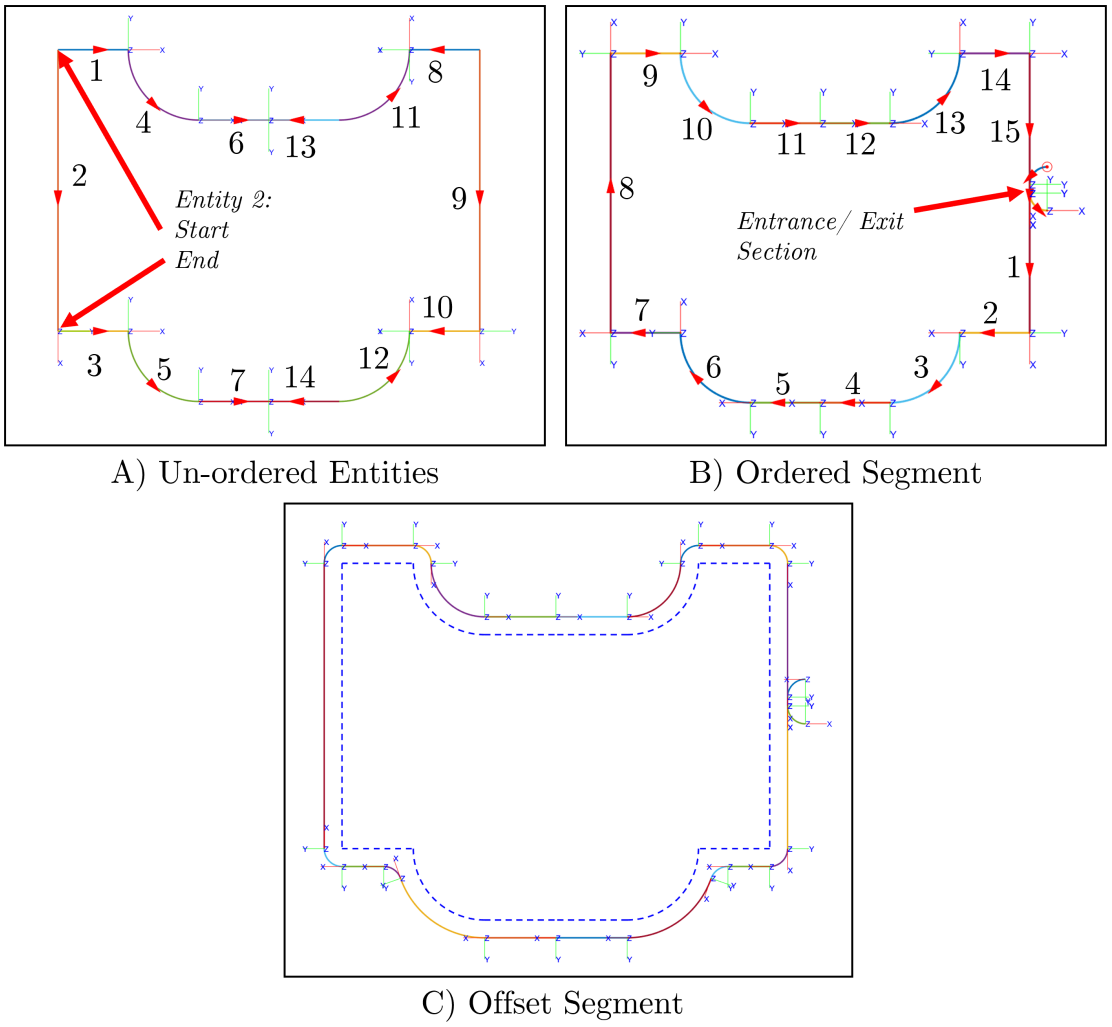


Figure 5.2: A) Input DXF entities describing the workpiece in a random order, B) Ordered entities within one segment, and C) the offset segments.

counter-clockwise direction. The endpoints of each entity are used as vertices for the shoelace algorithm. The segments are re-directed, such that the exterior segment is clockwise, and the interior segments are counter-clockwise. The specified directions point the y axes of the end point transformations ${}^{\text{CAD}}\mathbf{T}_{\text{E1}}$ and ${}^{\text{CAD}}\mathbf{T}_{\text{E2}}$ away from the workpiece. Figure 5.2 B also displays the end point transformation as an axis. The designation of the y axis gives the direction to offset the segment.

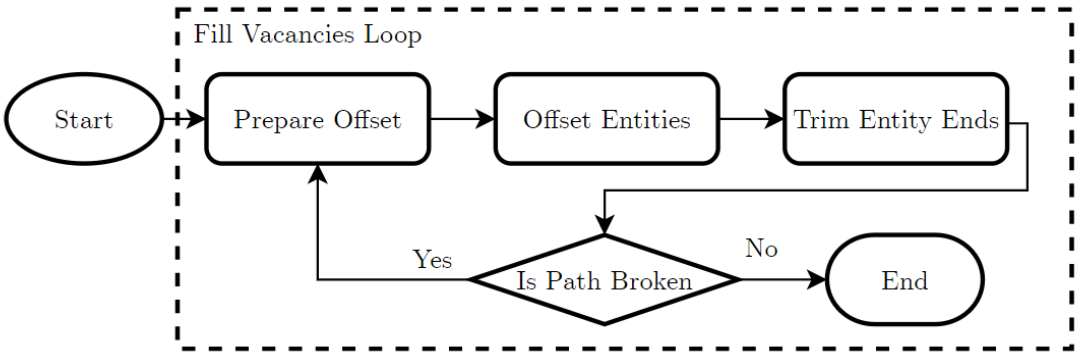


Figure 5.3: Offset procedure workflow.

5.1.3 Offset Segments

Figure 5.2 C displays the desired tool path offset from the workpiece to account for the radius of the machining tool. Figure 5.3 presents the offset procedure used to define the tool path with segments offset from the DXF model. Offsetting may cause entities in the segments to be deleted; therefore, the loop process in Fig. 5.3 is used to fill vacant locations and maintain a continuous path. The following section is used to describe each step of Fig. 5.3.

Prepare Offset

The 'Prepare Offset' procedure adds corner arc entities at sharp corners and vacant locations in the segment, which causes discontinuities in the offset path. The corner arc entity definition is identical to the arc entity, but denotes a discontinuity in the path. A vacancy is identified if the endpoints between adjacent entities do not match. A sharp corner is identified using the difference in the endpoint pose between adjacent entities. In Fig. 5.4, the three different intersections between entities are displayed, and corner arcs are placed at sharp internal and external intersections. Figure 5.4 A displays an external corner, where the original entity is shown in blue and the offset

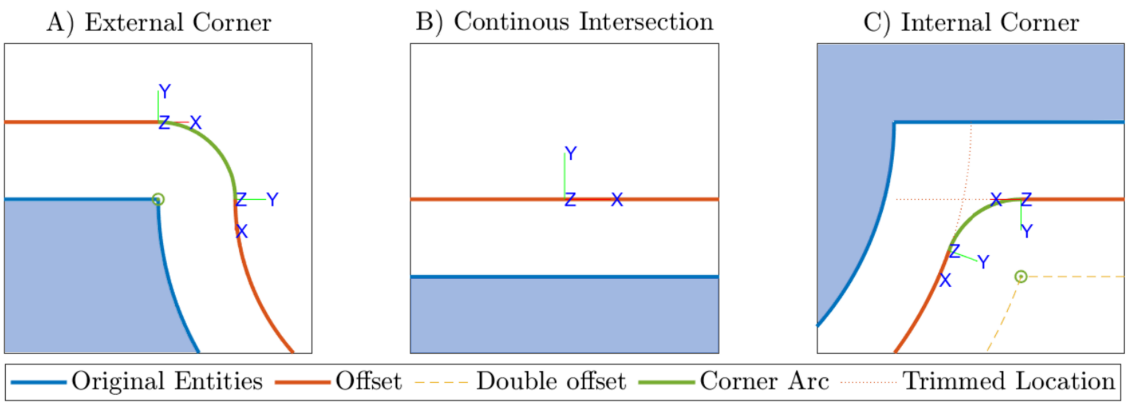


Figure 5.4: Offsetting A) external, B) continuous, and C) internal intersections between entities.

entities are shown in red. The corner arc is drawn in green, the center point of the corner arc is placed at the intersection of the neighbouring entities. Figure 5.4 B displays a continuous intersection between two line entities. Figure 5.4 C displays an internal corner, the center point of the corner arc is placed at the intersection of the 2× offset neighbouring entities. In 5.4, the double offset entities are drawn as yellow dashed lines. All corner arcs are placed at discontinuities with zero radius, which is increased when the entities are offset.

Offset Entities

The segments are offset by moving the entities away from the original DXF model. Linear entities are offset by translating the end points along the y axis of the endpoint pose. Arc entities are offset by increasing or decreasing the radius in the direction of the y axis of the endpoint pose. The offset entities are flagged so that each entity can only be offset once in the fill vacancies loop. After the offset, internal corner locations will have overlapping entities which must be trimmed back.

Trim Entity Ends

The offset procedure will cause entities at internal corners to overlap. The entities' endpoints are trimmed backed to remove the overlap and maintain a continuous path. Figure 5.4 C displays an internal corner, where the offset entity and trimmed region is shown as a red dotted line. The trim back location is calculated as the intersection between the neighbouring entities and the overlapping corner arc. With the entities properly trimmed, the path should be continuous.

Is Path Broken

A check is performed to see if the endpoints of each entity match their corresponding neighbour in the segment. The path will be broken if the offset procedure caused an entity to be deleted. Deletion may occur by reducing an arc's radius to zero or completely removing an entity while trimming. If an entity is deleted, the fill vacancy loop will iterate to add new corner arcs at vacant locations to reconstruct a close segment. Figure 5.2 C displays the results of the offset procedure, which is a closed contour following path in the DXF files coordinate frame, which is transformed into the robot's workspace for playback.

5.1.4 Transform Segments

The segments are transformed into the robots coordinate frame based on the transformation ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{p}}$ defined by the workpiece's location in the workspace. The workpiece's location is determined using a localization procedure, described in Section 5.2 for the RoboDeb platform. Line and arc entities are transformed with ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{p}}$ by transforming their respective definition points and vectors. The transformation changes the entities' start and endpoint transformations into the world coordinate frame ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{E1}}$ and ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{E2}}$. The transformed segments described the tool path around the workpiece,

which are exported into a set of waypoints for playback.

5.1.5 Define Workspace Waypoints

The output of the PCAT algorithm is a series of waypoints to describe the end effector trajectory. The series of waypoints are derived from the ordered entities in the offset segments. Line entities are exported into line waypoints which are linearly interpolated between. A line waypoint $\mathbf{w}_{\mathrm{L}}$ is composed as,

$$\mathbf{w}_{\mathrm{L}}^{(i)} = \{{}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}}^{(i)},\ {}^{\mathrm{w}}\mathbf{R}_{\mathrm{e}}^{(i)},\ {}^{\mathrm{w}}\mathbf{R}_{\mathrm{f}}^{(i)},\ f^{(i)},\ v^{(i)}\}, \tag{5.1}$$

where $\mathbf{x}^{(i)}$ References the i^{th} item in a series of variations of the variable $\mathbf{x}$. Arc and corner arc entities are exported into arc waypoints, which defines a circular trajectory. An arc waypoint $\mathbf{w}_{\mathrm{A}}$ is composed as,

$$\mathbf{w}_{\mathrm{A}}^{(i)} = \{{}^{\mathrm{w}}\mathbf{p}_{\mathrm{c}}^{(i)},{}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}}^{(i)},\ {}^{\mathrm{w}}\mathbf{R}_{\mathrm{e}}^{(i)},\ {}^{\mathrm{w}}\mathbf{R}_{\mathrm{f}}^{(i)},\ f^{(i)},\ v^{(i)}\}. \tag{5.2}$$

Each waypoint includes the end effector position ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}} \in \mathcal{R}^3$, applied force $f \in \mathcal{R}^1$, force pose ${}^{\mathrm{w}}\mathbf{R}_{\mathrm{f}} \in SO(3)$, end effector pose ${}^{\mathrm{w}}\mathbf{R}_{\mathrm{e}} \in SO(3)$ and end effector feedrate $v \in \mathcal{R}^1$. Figure 5.5 A and B, displays linear and circular interpolation between waypoints $\mathbf{w}^{(1)}$ and $\mathbf{w}^{(2)}$, where the waypoints are displayed as axes at the end effector position with the force pose. The following provide the selection of the waypoint composition.

End Effector Position ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}}$

The end effector position ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}}$ is taken as the endpoint of the line or arc entity defined in the robot's workspace. The arc waypoint center point ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{c}} \in \mathcal{R}^3$ is the arc entity center point.

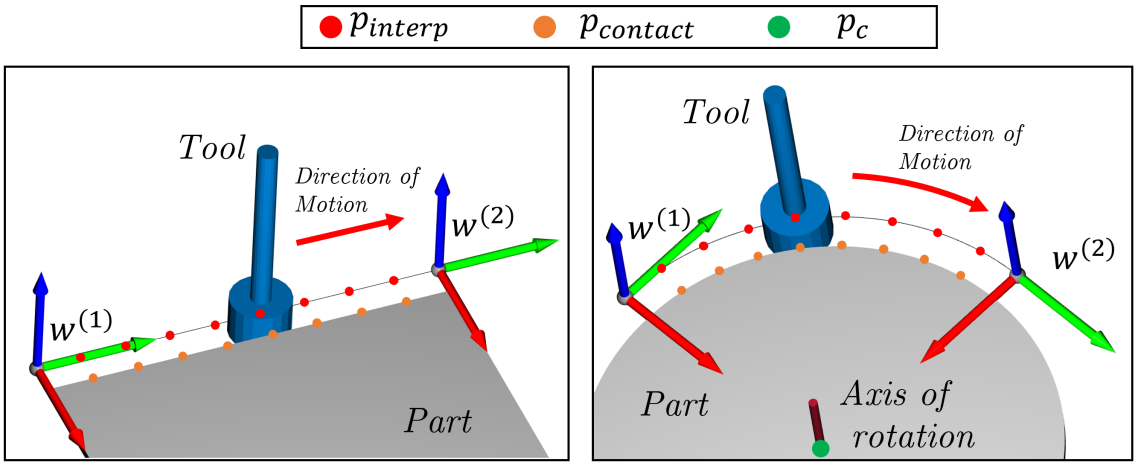


Figure 5.5: Interpolated A) line and B) arc waypoints.

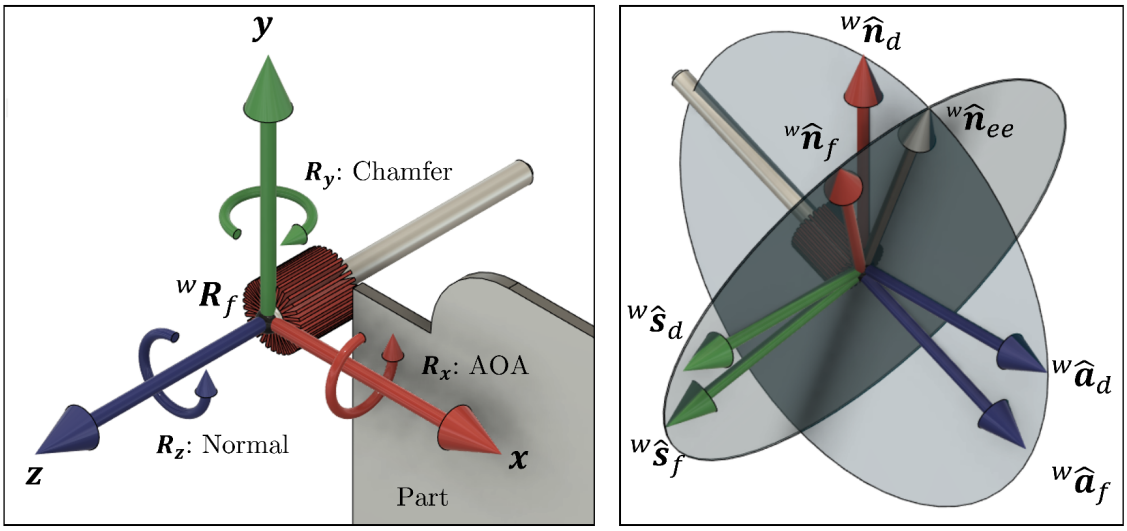
Applied Force f

The applied force f is a constant nominal value f_{nom} for line and arc waypoints, as contact is expected with the workpiece. Corner arc entities were placed at discontinuities in the path, therefore the applied force is removed.

Force Pose ${}^{\text{w}}\mathbf{R}_{\text{f}}$

Figure 5.6 A displays the force pose and interaction between the tool and part for deburring with a cylindrical tool. The force pose x axis is the direction to apply force onto the part, the y axis is the direction of motion and the z axis is colinear to the tool axis of rotation. Figure 5.2 C displays the offset segment with the end point poses ${}^{\text{w}}\mathbf{R}_{E2}$. The end point pose y axis is the normal direction away from the part. The force pose ${}^{\text{w}}\mathbf{R}_{\text{f}}$ is selected referencing the entity endpoint pose ${}^{\text{w}}\mathbf{R}_{E2}$, including a chamfer angle θ_{chfr} or Angle of Attack (AOA) θ_{AOA} as,

$$ {}^{\text{w}}\mathbf{R}_{\text{f}} = {}^{\text{w}}\mathbf{R}_{E2}\ \mathbf{R}_y(\theta_{\text{chfr}} + \pi)\ \mathbf{R}_x(\theta_{\text{AOA}}), \tag{5.3} $$



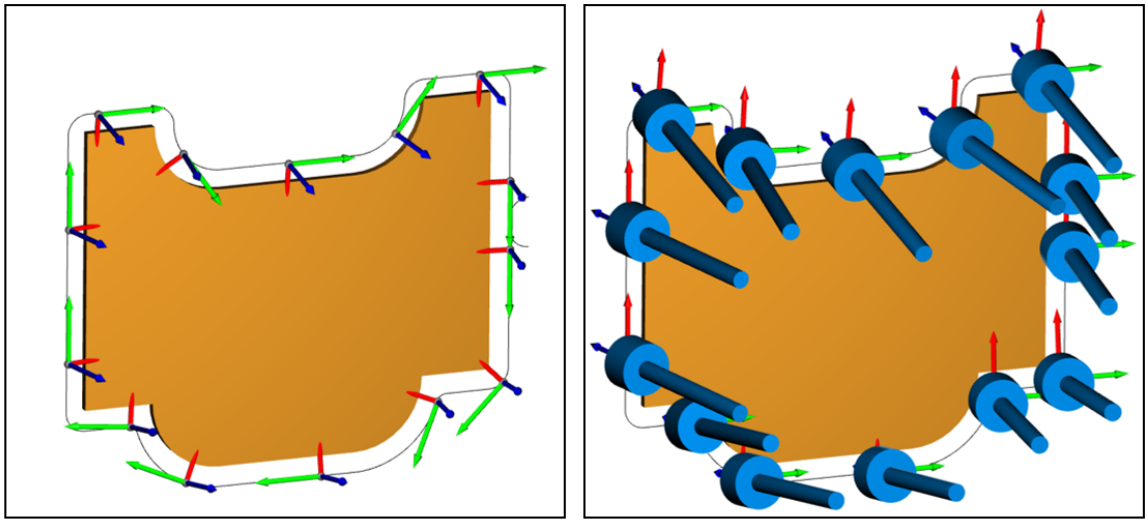
A) Tool Part Interaction B) Tool Pose Selection

Figure 5.6: A) Applied force direction and interaction between tool and the part B) Calculation of the end effector x axis from default pose and force pose. Note that ${}^{a}\hat{\mathbf{n}}_{b}$, ${}^{a}\hat{\mathbf{s}}_{b}$, ${}^{a}\hat{\mathbf{a}}_{b}$ are the x, y, and z components of the axis/pose represented as rotation ${}^{a}\mathbf{R}_{b}$.

where the rotation matrices $\mathbf{R_x}(\theta)$ and $\mathbf{R_y}(\theta) \in SO(3)$ are rotations around the x and y axis, respectively, by θ. Figure 5.7 A displays the interpolated force pose along a trajectory with a 10 deg chamfer angle applied around a test part.

End Effector Pose ${}^{w}\mathbf{R}_{e}$

When deburring with a cylindrical tool, material removal may occur anywhere around the circumference of the tool, permitting movement in 1 DOF of the end effector pose along the axis of rotation of the tool. In the simple case of edge deburring shown in Fig. 5.6 A with no chamfer or Angle of Attack (AOA), the end effector is held perpendicular to the planar part. The entire part periphery is deburred at a constant pose, as the end effector rotation about the z axis is null. In Chapter 4, Fig. 4.3 illustrates end effector of the RoboDeb platform, a suitable default end effector pose ${}^{w}\mathbf{R}_{d}$ for edge deburring has the z axis perpendicular to the planar part and the x axis is vertical as it is convenient for the robot.



A) Force Pose B) End Effector Pose

Figure 5.7: The selection of the A) force pose and the B) end effector pose around the part.

The PCAT algorithm can modify the end effector pose along the trajectory to perform chamfering paths or change the tool AOA relative to the part. Figure 5.7 B, describes the end effector pose and tool drawn in for a 10 degree chamfering trajectory, with minimal changes to the pose around the part. In the following procedure, the end effector pose is defined for complex tool orientations with minimal deviations from the default deburring pose, using a similar approach to Asakawa et al. [43]. The procedure begins by referencing the force pose to select the end effector z axis. Next, the end effector pose x axis is selected for minimal adjustments from the default pose. The selection of the two z and x orthonormal components fully constrains the end effector pose.

Figure 5.7 A displays the force pose containing an chamfer angle. The tool axis of the end effector is selected to be colinear to the force pose z axis, along the direction of the default pose ${}^{\mathrm{w}}\mathbf{R}_{\mathrm{d}}$ z axis.

If the end effector pose was simply assigned as the force pose, the robot would rotate the entire end effector around the part as the x axis is constantly pointing

towards the part. The reduced DOF is exploited to select a convenient end effector pose. Figure 5.6 B depicts the selection of the end effector x axis, calculated as the intersection between the xz plane of the default pose and the xy plane of the modified force pose with,

$$|{}^{\mathrm{w}}\hat{\mathbf{n}}_{\mathrm{ee}}^{(i)}| \;=\; {}^{\mathrm{w}}\hat{\mathbf{s}}_{\mathrm{d}} \times {}^{\mathrm{w}}\hat{\mathbf{a}}_{\mathrm{ee}}^{(i)}, \tag{5.4}$$

which yields two solutions in equal and opposite directions. The end effector x axis direction is selected along the default x axis. With the z and x axis selected, the end effector pose is fully constrained with,

$${}^{\mathrm{w}}\hat{\mathbf{s}}_{\mathrm{ee}}^{(i)} \;=\; {}^{\mathrm{w}}\hat{\mathbf{a}}_{ee}^{(i)} \times {}^{\mathrm{w}}\hat{\mathbf{n}}_{ee}^{(i)}. \tag{5.5}$$

Figure 5.7 B displays the results of the procedure where the tool and the end effector pose are displayed along the trajectory. Note that the x axis of the end effector pose is always pointing upwards, with minor adjustments to account for the chamfer angle of the tool.

End Effector Feedrate

Interpolation obtains the end effector position, end effector pose and force pose at each time step along the robot's trajectory. The robot uses a fixed time step, therefore, the velocity is proportional to the distance between the interpolated positions at each time step. Figure 5.5 A and B, displays linear and circular interpolation between waypoints at a constant velocity with the tool shown to represent the end effector. In machining applications, the resulting finish depends on the interaction between the tool and the workpiece, while the end effector trajectory is defined relative to the tool center. Figure 5.5 also displays the expected contact point, estimated by projecting the end effector position ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{e}}$ along the x axis of the force pose. For line waypoints, there is no change in the force pose, therefore the spacing of the contact points, and

consequently the observed velocity at the workpiece, is constant. Line waypoints have a constant nominal feedrate v_{nom} to be held until the next waypoint is reached. In Fig. 5.5 B for arc waypoints, the end effector position and poses are rotated around the force pose z axis at the center point, causing a difference in the spacing of the end effector position and the expected contact point. The different spacing indicates a variation between the end effector velocity and the observed velocity on the workpiece. The feedrate along an arc waypoint is constant, and calculated to achieve an observed feedrate equal to the nominal feedrate, calculated as,

$$v^{(i)} = v_{\text{nom}} \frac{d_{\text{radius}}}{(d_{\text{radius}} + d_{\text{off}})^a} \quad a = \begin{cases} 1 & \text{if external} \\ -1 & \text{if internal} \end{cases}, \tag{5.6}$$

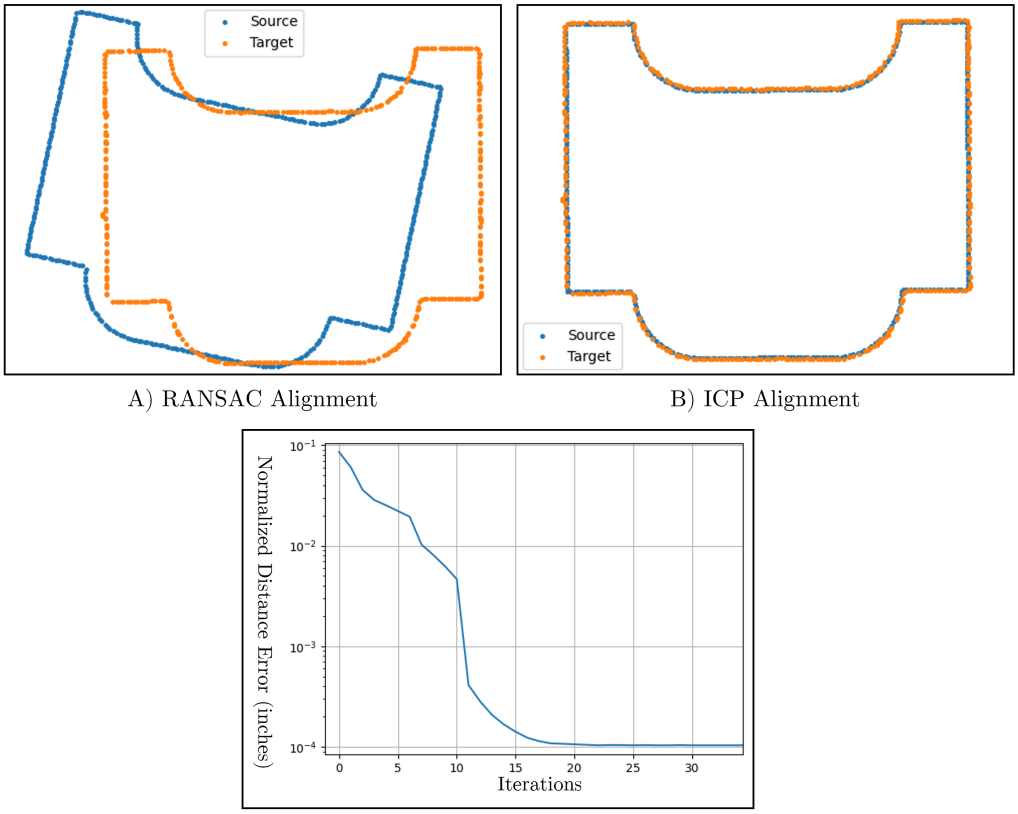
where d_{radius} is the arc radius, d_{off} is the offset distance, and a is a sign which is $+1$ for internal corners and -1 for external corners. Small external arcs will cause large velocities therefore v is saturated to double v_{nom}. An exception is for arc waypoints created from corner arc entities, which have a feedrate of twice the nominal velocity, as there is no applied force.

5.2 RoboDeb Implementation

The PCAT algorithm is implemented on the RoboDeb platform for robotic deburring experimentation, where the results are presented in Chapter 5 of this thesis. To implement the PCAT algorithm on the RoboDeb platform, a localization procedure is designed. Localization is performed using the Robodeb's laser scanner and camera system to perceive the part location. Chapter 4, Section 4.4 presented the generation of point clouds using the Robodeb's computer vision systems and example point clouds in Fig. 4.5 and Fig. 4.7. In the deburring experimentation, both the laser scanner and monocular camera systems are used and compared, based on accuracy

and process time.

A registration procedure will seek to fit a CAD model of the part to the observed point cloud model obtained. First, the CAD model of the part is discretized into a source point cloud ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{s}}$ which is to be aligned with the target point cloud ${}^{\mathrm{w}}\mathbf{p}_{\mathrm{t}}$ collected from a vision system. A two-step registration procedure is used to obtain the rigid body transformation ${}^{\mathrm{s}}\mathbf{T}_{t}$ to match the point clouds. First, global registration is performed with a RANSAC algorithm (Chapter 2, Section 2.2.2) to provide an initial estimate of the alignment; the implementation used was included in the Open3D library [62]. Second, a ICP custom algorithm written by the author is used to refine the alignment. Figure 5.8 A displays the alignment results of the RANSAC algorithm and B displays the refinement of the ICP algorithm. The SciPy minimize function [63] was used to minimize the point-to-plane metric in equation (2.25) with the Newton-conjugate gradient procedure. Figure 5.8 C displays the point-to-plane error minimization as the ICP algorithm iterates. The correspondences between point clouds were estimated using the closest point metric. After the local refinement, the transformation ${}^{\mathrm{t}}\mathbf{T}_{\mathrm{s}}$ obtained, which is the same transformation ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{p}}$ for the parts location in the workspace to design the path with the PCAT algorithm. Figure 5.9 displays the generated workspace waypoints around a localized test part placed on the vacuum clamp. Waypoints are depicted as axes at the end effector position with the force pose.



A) RANSAC Alignment

B) ICP Alignment

C) ICP Error Minimization

Figure 5.8: RoboDeb registration procedure using A) RANSAC as an initial guess and B) an ICP algorithm which C) literately minimizes the the point-to-plane metric.

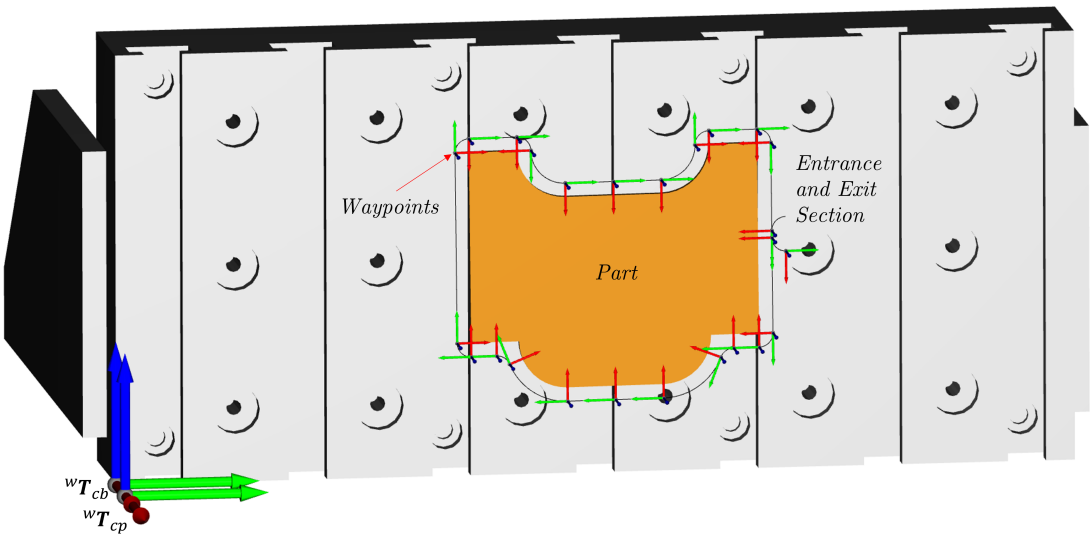


Figure 5.9: Output waypoints of the PCAT algorithm displayed as axes at the waypoint end transformations.

5.3 Summary

In this chapter, a CAD/CAM trajectory planner was created for planar contour following for robotic machining. The PCAT algorithm developed, designed deburring and chamfering trajectories using a 2D DXF model of a planar part. The output of the PCAT algorithm was a series of line and arc waypoints describing an offset trajectory with a known direction normal to the part to apply force. The PCAT algorithm required the location of the part in the workspace, found through a localization procedure. The PCAT algorithm was applied to the RoboDeb Platform by designing a localization procedure using the computer vision systems equipped to capture a point cloud of the part and a registration procedure for locating the part. The registration procedure employed a RANSAC algorithm for an initial guess and an ICP algorithm for refinement. The results of the PCAT algorithm for robotic deburring experimentation are presented in Chapter 5 of this thesis. The PCAT algorithm was developed as a benchmark for new PIST algorithm.

Chapter 6

Planer Image Space Trajectory Planner

The following section will develop the novel *Planer Image Space Trajectory* (PIST) algorithm for contour following in robotic machining. The PIST algorithm uses a computer vision approach to design a trajectory based on the perceived workpiece geometry. The benefit of computer vision trajectory planning is that it doesn't require any CAD data, adapts to manufacturing imperfections, and quickly defines a trajectory, allowing it to be automated in high mix low volume applications. The novelties of the PIST algorithm is that it uses a perspective correction to ensure distortion free path planning and uses a novel image-space path planning procedure to design an offset path with a known normal direction. Section 6.1 presents the PIST algorithm which uses a monocular camera to design planar contour following trajectories. Section 6.2 expands the PIST algorithm into 3D trajectory planning. Section 6.3 and Section 6.4 describe the implementation of the 2D and 3D PIST algorithm on the RoboDeb platform.

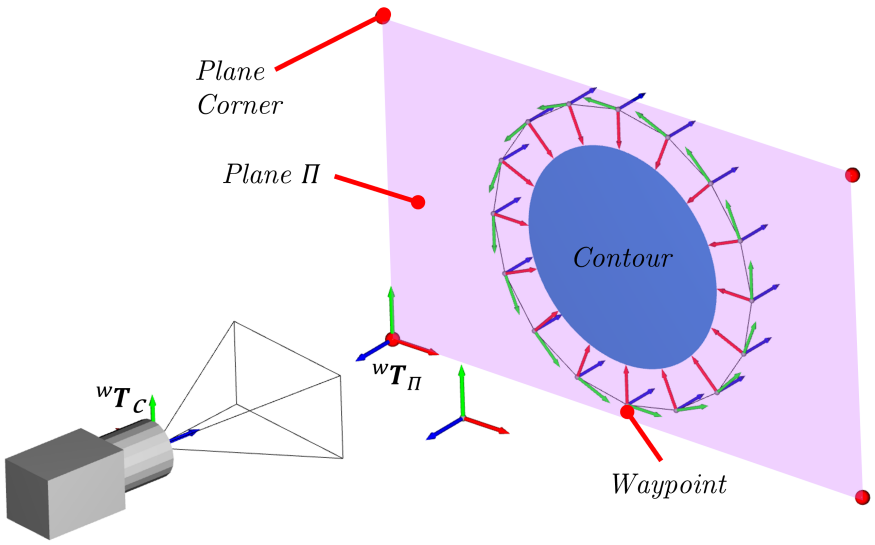


Figure 6.1: Workspace of the PIST algorithm.

6.1 PIST Algorithm Description

The PIST algorithm creates contour following trajectories for robotic machining that is offset to the workpiece with a known normal direction to apply force using a force control scheme. Figure 6.1 displays the workspace of the PIST algorithm which uses a monocular camera to detect a planar contour on a workpiece. The output of the PIST algorithm is a series of waypoints which are interpolated to generate a trajectory. Figure 6.2 describes the workflow of the PIST algorithm; in Section 6.1.1, planar orthorectification is used to transform the perspective of the image into a perpendicular, to-scale, view of the planar contour, Section 6.1.2 performs path planning in the to scale image by detecting the contour of interest and plans an offset path with a normal direction, Section 6.1.3 designs the feedrate along the planned path, Section 6.1.4 the trajectory is described in image-space with a minimal set of waypoints, which is transformed into the robot's workspace in Section 6.1.5.

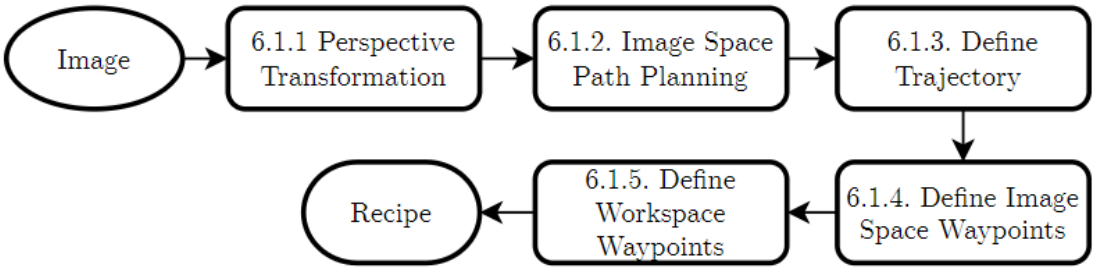


Figure 6.2: Process of the PIST algorithm.

6.1.1 Perspective Transformation

The first step of the PIST algorithm is to perform orthorectification on the original image, to obtain a to-scale orthophoto of the plane. Figure 6.3 displays the orthorectification of Fig. 6.1 according to Section 2.1.4. The homography matrix ${}^{\text{I}}\mathbf{H}_{\text{o}}$ is obtained using the correspondence between the projection of four rectangular points on the plane onto image one, and the four corners of the orthophoto. In the orthophoto, an offset path can be planned relative to the detected workpiece.

6.1.2 Image-Space Path Planning

The orthophoto provides a to-scale representation of the planar surface in which path planning is performed. The task of image-space path planning is to detect the contour of the workpiece, and define an ordered list of pixels ${}^{\Pi}\mathbf{q}_{\text{ee}}$ offset to the contour and an angle θ_f normal to the contour. The pixel path and angle provide spatial and directional information for the robot end effector to traverse the planar contour. Algorithm 1 and Fig. 6.4 A to D describes the four-step process for identifying a contour following path using image processing, by identifying a mask image, performing the distance transform, taking the gradient of the distance transform and selecting the offset path.

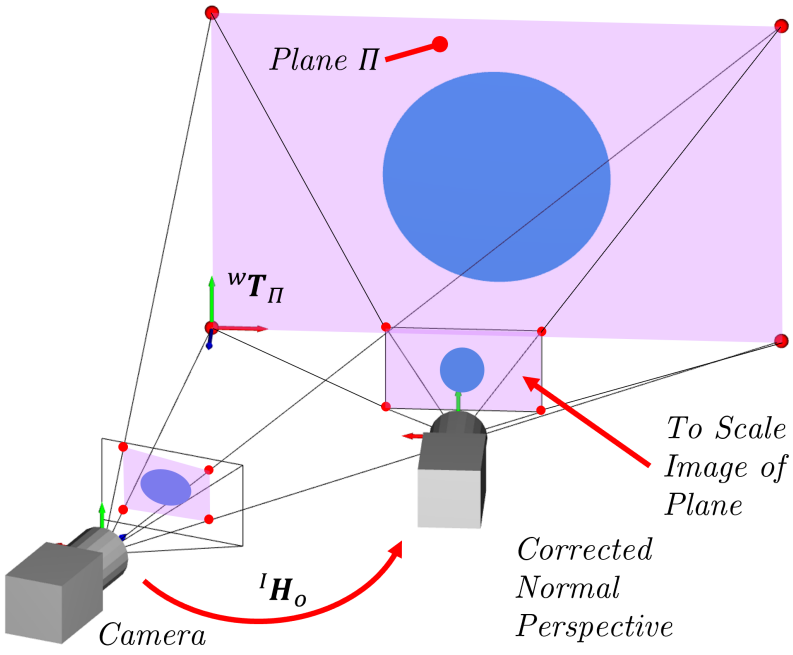


Figure 6.3: Orthorectification of Fig. 6.1 using planar homography. The corrected perspective on the right is perpendicular to the plane and provides a to-scale representation of the surface.

Identify the Mask Image

There is an assortment of established techniques for detecting an object in an image, such as edge detection using in Lai et al. [56], blob detection used in Princely et al. [57], and others. Detection results will vary based on the specific application, but for the RoboDeb Platform, background subtraction using the subtractor from Zivkovic et al. [58] achieved the most consistent detection results. The workpiece contour is detected in the image, and a binary mask image is produced which identifies the contour. Image A in Fig. 6.4 displays a sample binary mask of an identified circle. Burrs or inconsistencies along the detected part may be removed using a median filter.

Distance Transform

The distance transform is taken on a binary image, which scales the intensity of each high pixel based on the distance to the nearest low pixel, indicating a position

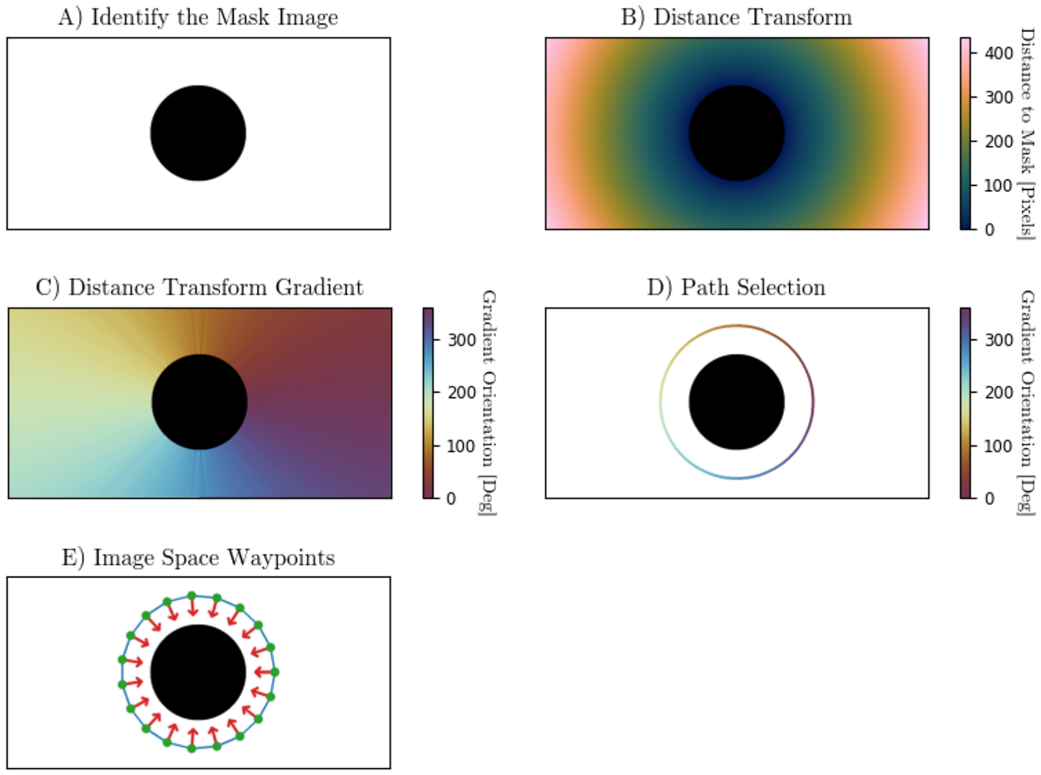


Figure 6.4: Identifying an offset path in image-space using the distance transform and gradient. A) a binary image $\mathbf{I}_{\mathrm{M}}$ of a circle as an example of a detected object, B) the distance transform image $\mathbf{I}_{\mathrm{DT}}$ of the binary image, C) the gradient orientation image $\mathbf{I}_{\theta}$ of the distance transform and D) the selected path, offset to the mask with the gradient orientation, E) the selected image-space waypoints with position (green) and pose (red arrow) information, which are interpolated (blue) to define the path. The gradient orientation is an angle, in degrees from the x axis of the image.

on the part. Using the distance transform, Image B in Fig. 6.4 is obtained in which the pixel intensity describes the distance to the identified circle. Notice that the pixel intensity increases uniformly in the radial direction away from the center of the circle, while it is constant at a fixed radius away from the center.

Distance Transform Gradient

The spatial gradient of an image yields the directional rate of change of pixel intensity, described as a 2D vector at each pixel location. The gradient is taken of the distance transform image to define the direction away from the contour, for

Algorithm 1 Image-space Path Planning

Identify the Mask Image

$\mathbf{I}_{\mathrm{M}} = \mathrm{BackgroundSubtraction}(\mathbf{I})$

Distance Transform

$\mathbf{I}_{\mathrm{DT}} = \mathrm{DistanceTransform}(\mathbf{I}_{\mathrm{M}})$

Distance Transform Gradient

$\mathbf{I}_{\mathrm{dx}} = \frac{\partial}{\partial x}\mathbf{I}_{\mathrm{DT}}$

$\mathbf{I}_{\mathrm{dy}} = \frac{\partial}{\partial y}\mathbf{I}_{\mathrm{DT}}$

$\mathbf{I}_{\theta} = \tan^{-1}(\frac{\mathbf{I}_{\mathrm{dy}}}{\mathbf{I}_{\mathrm{dx}}})$

Path Selection

$\mathbf{I}_{\mathrm{offset1}} = \mathbf{I}_{\mathrm{DT}} > d_{\mathrm{off}} + d_{\mathrm{min}}$

$\mathbf{I}_{\mathrm{offset2}} = \mathrm{DistanceTransform}(\mathrm{not}(\mathbf{I}_{\mathrm{offset1}})) > \mathrm{d}_{\mathrm{min}}$

${}^{\Pi}\mathbf{q}_{\mathrm{ee}} = \mathrm{EdgeDetection}(\mathbf{I}_{\mathrm{offset2}})$

$\theta_{\mathrm{f}} = \mathbf{I}_{\theta}[{}^{\Pi}\mathbf{q}_{\mathrm{ee}}] - \pi$

designing the applied force direction. The spatial gradient is calculated in the x and y directions, through convolution with a discrete differentiation operator, such as the Sobel operator, implemented in Opencv [64]. Derivatives in the x and y direction are converted into polar form to obtain the gradient orientation image $\mathbf{I}_{\theta}$. Image C in Fig. 6.4 displays the gradient orientation image $\mathbf{I}_{\theta}$ of the distance transform, where the pixel intensity describes the angle θ_{g} pointing normal to and away from the circle. Notice that the pixel intensity is constant in the radial direction from the center of the circle, while it increases in the counter-clockwise direction at a fixed radius away from the center.

Path Selection

The PIST algorithm uses the distance transform and its gradient to select an offset path around the detected planar contour with a known normal direction. An offset mask is obtained by thresholding the distance transform at a specified offset distance

d_{off} in pixel units. The border of the thresholded image an offset path in image-space.

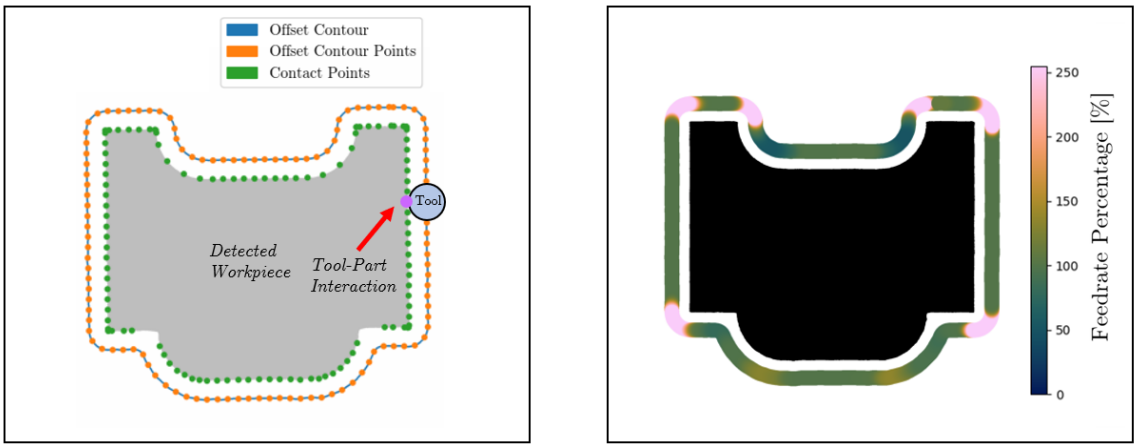
When offsetting at a constant distance d_{off} away from a mask, sharp external corners are projected into arcs, which maintain a constant distance away from the corner, while sharp internal corners are maintained. Sharp corners in the path should be avoided; to enforce a minimum corner radius d_{min} at all corners, offset outwards from the workpiece by $d_{\text{off}} + d_{\text{min}}$, then offset inwards by d_{min}. The offset procedure is similar to the process presented in Xu et al. [65], excluding the minimum corner radius.

Pixels along the border of the offset mask describe the image-space path for the end effector ${}^{\Pi}\mathbf{q}_{\text{ee}}$ and can be extracted with the border following algorithm presented in Suzuki et al. [66]. The applied force orientation θ_{f} normal to the workpiece is retrieved at each pixel in the path by referencing the gradient orientation. Results of the procedure are shown in image D of Fig. 6.4, where the offset path is drawn next to the mask image and the pixel intensity describes the angle normal to the detected contour. The defined path is augmented with timing information to define a trajectory.

6.1.3 Define Trajectory

Similar to Section 5.1.5, the end effector feedrate is modulated to ensure the observed feedrate, and machining results are consistent. The expected contact point between the tool and the workpiece ${}^{\Pi}\mathbf{q}_{\text{contact}}$ is calculated by projecting each pixel ${}^{\Pi}\mathbf{q}_{\text{ee}}$ in the force orientation θ_{f} direction by the offset distance d_{off} in pixel units with,

$$\begin{bmatrix} {}^{\Pi}\mathbf{q}_{\text{contact}} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{2\times 2}(\theta_f) & \begin{bmatrix} d_{\text{off}} \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^{\Pi}\mathbf{q}_{\text{ee}} \\ 1 \end{bmatrix}, \tag{6.1}$$



A) Constant Feedrate Visualization

B) Feedrate Modulation

Figure 6.5: A) Projected tool contact points at a constant feedrate. B) Feedrate modulation along the selected contour. The feedrate is shown as a percentage of the nominal feedrate.

where $\mathbf{R}_{2\times2}(\theta_f) \in SO(2)$ is a rotation matrix about θ_f. Figure 6.5 A displays the image-space path and contact points for a complex part. Pixel positions on the offset path are initially uniformly distributed and projected in the force orientation direction. The curvature in the contour, and consequently the force orientation, cause a non-uniform spacing of the contact points, revealing the expected change in the observed feedrate.

For compensation, the end effector feedrate v is derived from the nominal feedrate v_{nom} scaled by the inverse of the expected change in the observed feedrate as,

$$v^{(i)} = \frac{v_{\text{nom}}}{|| \ {}^{\Pi}\mathbf{q}^{(i)}_{contact} - \ {}^{\Pi}\mathbf{q}^{(i-1)}_{contact} ||}. \tag{6.2}$$

Figure 6.5 B displays the results of the feedrate modulation. The feedrate is reduced on internal corners and increased on external corners. Sharp external corners create large feedrates, therefore the resulting end effector feedrate is saturated at double the nominal feedrate to prevent excessive velocities. The image-space trajectory data is now completely described by the position, orientation and feedrate at a series of pixel

locations. The following section will export the trajectory data into a minimal set of waypoints.

6.1.4 Define Image-Space Waypoints

The pixel path is now converted into a reduced set of image-space waypoints, defined as,

$$^{\Pi}\mathbf{w}_I^{(i)} = \{^{\Pi}\mathbf{q}_{\mathrm{e}}^{(i)},\ \theta_f^{(i)},\ f^{(i)},\ v^{(i)}\}, \tag{6.3}$$

The position $^{\Pi}\mathbf{q}_{\mathrm{e}} \in \mathcal{R}^2$ and force orientation $\theta_f \in \mathcal{S}^1$ data is specified at each pixel along the image-space trajectory, therefore, waypoints are initially chosen at each pixel location. The applied force f is selected at each waypoint. The initial waypoints are evenly distributed based on the image resolution. To obtain a minimal set of waypoints, the waypoints are first re-sampled and then redundant waypoints are removed.

Applied Force Selection

Force control is used to maintain a set applied force onto the workpiece when machining. The minimum corner radius imposed in the path causes the tool to lose contact with the part at sharp internal corners. At sharp external corners, the observed feedrate is zero, causing the tool to dwell at a single point and round the corner. Therefore, the applied force f is removed when contact is not expected or when at sharp external corners. To estimate if the tool should be in contact with the part, the Euclidean distance between each tool contact point $^{\Pi}\mathbf{p}_{\mathrm{contact}}$ and the detected foreground mask is referenced. The tool is estimated to be at a sharp external corner when the observed feedrate is near zero. With the waypoints fully described from equation (6.3), redundant waypoints are removed.

Re-Sampling Waypoints

Waypoints are re-sampled to define a spacing interval. The re-sampling factor is the ratio between the current pixel spacing and the desired waypoint spacing. The re-sampling ratio is converted to a fraction $\frac{\text{num}}{\text{den}}$, where the numerator is the up-sampling factor and the denominator is the down sampling factor. All waypoint data is up sampled, lowpass filtered at the cut-off frequency ω_c, then down sampled. The cut-off frequency ω_c is calculated from the Nyquist frequency ω_n as,

$$\omega_c = \frac{\omega_n}{\max(\text{num}, \text{den})}. \tag{6.4}$$

Removal of Redundant Waypoints

Waypoints are linearly interpolated between to obtain a trajectory. Waypoints along linear sections of a path are redundant. To identify redundant points, the derivative of the orientation θ_f is taken using a backwards difference approximation and waypoints are removed in areas with low rates of change. The remaining waypoints describe the image-space trajectory. Image E of Fig. 6.4 presents the image-space waypoints, which are next mapped into the robot's workspace.

6.1.5 Define Workspace Waypoints

The output of the PIST algorithm is a series of waypoints in the robot's workspace for the end effector to follow. The image-space waypoints from equation (6.3) are now mapped into line waypoints from equation (5.1) in the PCAT algorithm. The following subsection describes the selection of the waypoint data. First, the end effector position ${}^{\text{w}}\mathbf{p}_{\text{ee}}$ is determined by scaling the pixel path onto the plane in the workspace. Next, the force pose ${}^{\text{w}}\mathbf{R}_{\text{f}}$ is established using the force orientation data. The end effector pose ${}^{\text{w}}\mathbf{R}_{\text{ee}}$ is selected, referencing the force pose. The feedrate v and

force magnitude F are the same as previously specified in image-space.

End Effector Position

Waypoints in the orthophoto are linearly scaled onto points on the plane in the workspace and then transformed into the workspace coordinate frame. The combination of a transformation and scaling is performed using a similarity transformation $\mathbf{S} \in Sim(3)$ of the form,

$$ {}^{b}\mathbf{S}_{a} = \begin{bmatrix} {}^{b}\mathbf{R}_{a} & {}^{b}\mathbf{t}_{a} \\ 0 & s^{-1} \end{bmatrix}, \tag{6.5} $$

where $s \in \mathcal{R}^1$ is the scaling factor. The end effector position in image-space ${}^{\Pi}\mathbf{q}_{\text{ee}} \in \mathcal{R}^2$ is mapped into the workspace R^3 with,

$$ \begin{bmatrix} {}^{\text{w}}\mathbf{p}_{ee} \\ 1 \end{bmatrix} = {}^{\text{w}}\mathbf{S}_{\Pi} \begin{bmatrix} \begin{bmatrix} {}^{\Pi}\mathbf{q}_{\text{ee}} \\ 0 \end{bmatrix} \\ 1 \end{bmatrix}, \tag{6.6} $$

where the scaling factor s is the ratio between the image size in pixels and the measured size of the detected plane.

Force and End Effector Pose

The force orientation at each pixel $\theta_{\text{f}} \in \mathcal{S}^1$ can be expressed as a rotation matrix in the plane by assuming that the rotation is about the z axis with,

$$ {}^{\text{w}}\mathbf{R}_{\text{f}} = {}^{\text{w}}\mathbf{R}_{\Pi}\ \mathbf{R}_{z}(\theta_{\text{f}})\ \mathbf{R}_{y}(\theta_{\text{chfr}})\ \mathbf{R}_{x}(\theta_{\text{AOA}}). \tag{6.7} $$

The rotation ${}^{\text{w}}\mathbf{R}_{\Pi}$ transforms the force pose on the planar surface into the world coordinate frame, and $\mathbf{R}_{y}(\theta_{\text{chfr}})$ and $\mathbf{R}_{x}(\theta_{\text{AOA}})$ apply a chamfer and AOA to the force pose. Figure 6.1 displays the resulting waypoints around the circular contour with the end effector position and force pose displayed. The force pose x axis now describes the direction for the robot to apply force onto the contour. The end effector

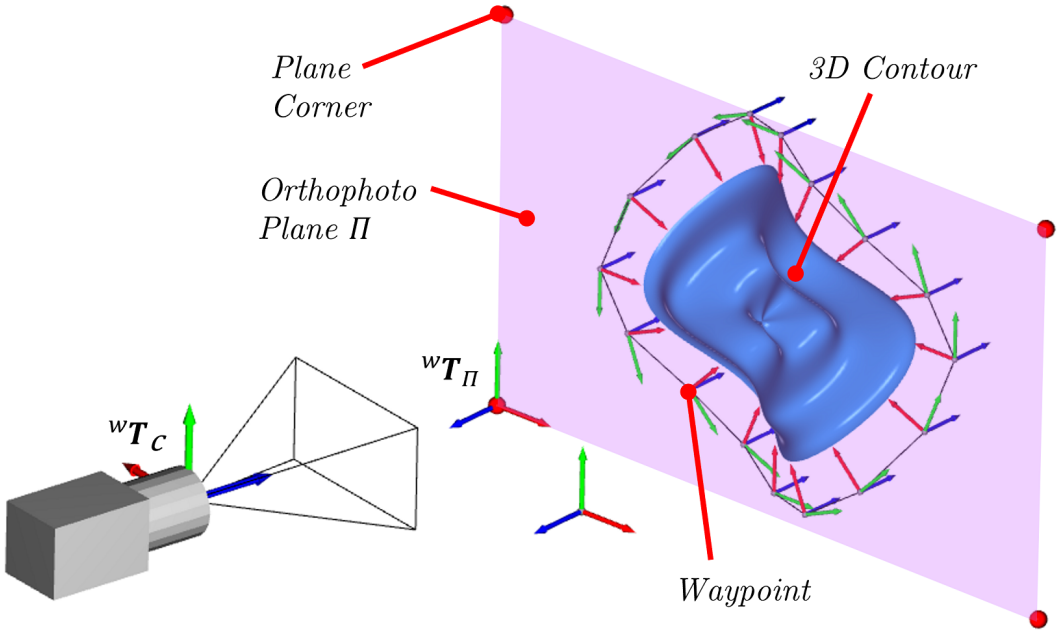


Figure 6.6: Workspace of the 3D PIST algorithm. See Fig. 6.7 C for the depth image of the 3D contour shown.

pose is selected referencing the force pose according to Section 5.1.5 for the PCAT algorithm, allowing for the design of chamfering paths. The workspace waypoints are now completely described for the 2D application, but the system can be extended to 3D.

6.2 Three-Dimensional PIST Algorithm

The 3D PIST algorithm uses orthorectification to represent a 3D surface in a 2D image, in which image-space path planning is performed. The image-space waypoints are then mapped back onto the 3D surface, to describe the trajectory. The benefit of the 3D PIST algorithm over traditional CAD/CAM software was that it still does not require a CAD model of the workpiece. Figure 6.6 displays the 3D PIST algorithm workspace with a path planned around a circular 3D surface with variations in the z dimension.

6.2.1 Orthorectification

Orthorectification is the process of removing the effects of perspective and surface elevation in an image, to create a to-scale representation of the 3D surface. Section 2.1.4 describes the 3D orthorectification procedure. The 3D PIST algorithm uses 3D orthorectification to obtain an image-space representation of a 3D surface. Figure 6.7 A displays an image taken from the camera's perspective in Fig. 6.6 and Fig. 6.7 B displays the generated orthophoto. The orthophoto is a perpendicular perspective that accounts for the camera's perspective and the variations in the z dimension of the surface shown in Fig. 6.7 C. With orthorectification, it is clear that the surface in Fig. 6.6 is a circular contour in the orthophoto image plane. The original PIST algorithm is used to perform image space path planning around the circular contour in the orthophoto, exactly as displayed in Fig. 6.4. The image-space waypoints created are projected into the workspace onto the 3D surface, by adjusting the end effector position.

End Effector Position

The end effector position in image-space ${}^{\Pi}\mathbf{q}_{\text{ee}} \in \mathcal{R}^2$ is mapped onto the 3D surface in the workspace R^3 with,

$$\begin{bmatrix} {}^{\text{w}}\mathbf{p}_{ee} \\ 1 \end{bmatrix} = {}^{\text{w}}\mathbf{S}_{\Pi} \begin{bmatrix} \begin{bmatrix} {}^{\Pi}\mathbf{q}_{\text{ee}} \\ \mathbf{I}_{\text{d}}[\,{}^{\Pi}\mathbf{q}_{\text{ee}}\,] \end{bmatrix} \\ 1 \end{bmatrix}. \tag{6.8}$$

The end effector position selection is similar to equation (6.6) except the z dimension is $\mathbf{I}_{\text{d}}[{}^{\Pi}\mathbf{q}_{\text{ee}}]$ which is the depth dimension in pixel units referenced from the depth image $\mathbf{I}_{\text{d}}$ created in the orthorectification procedure. Figure 6.6 displays the outputted waypoints with the end effector position and force pose around the 3D surface. The PIST algorithm and its 3D expansion are designed for contour following for robotic

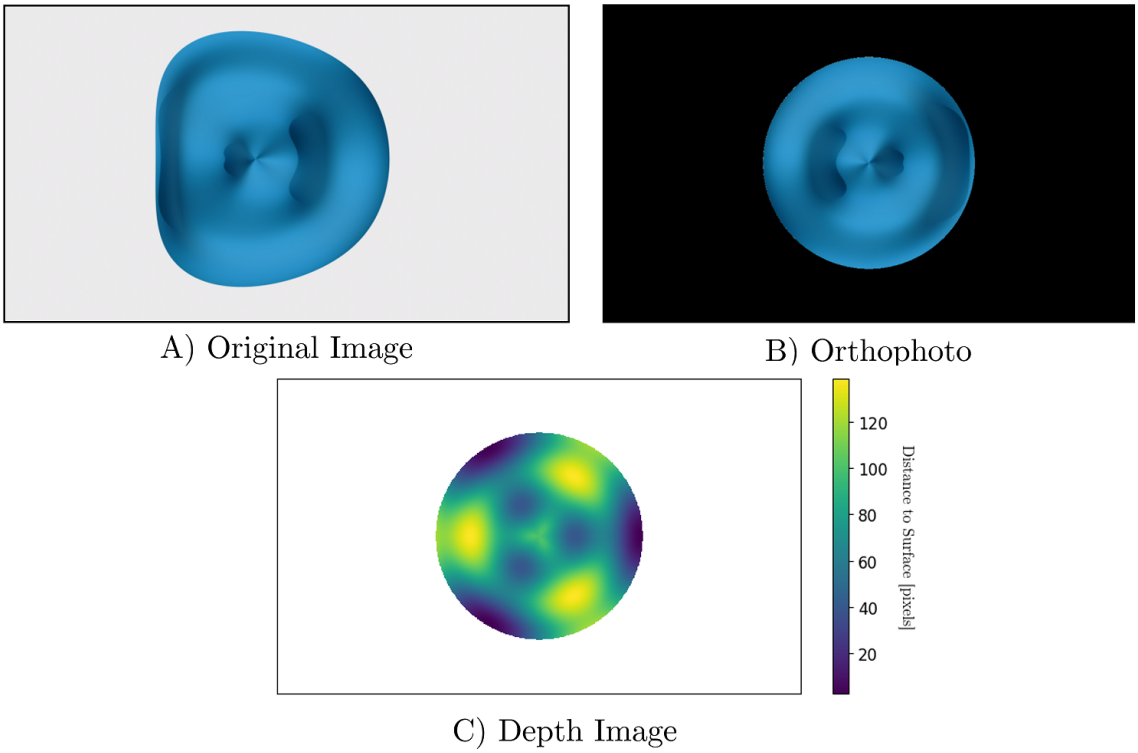
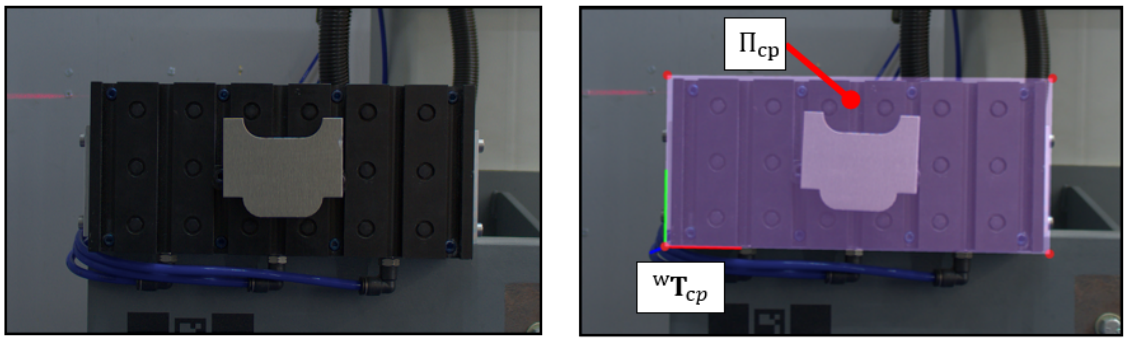


Figure 6.7: A) Image taken from the cameras perspective in Fig. 6.6 and B) the generated orthophoto and C) the depth image $\mathbf{I}_{\mathrm{d}}$

machining, which are implemented on the RoboDeb platform for deburring.

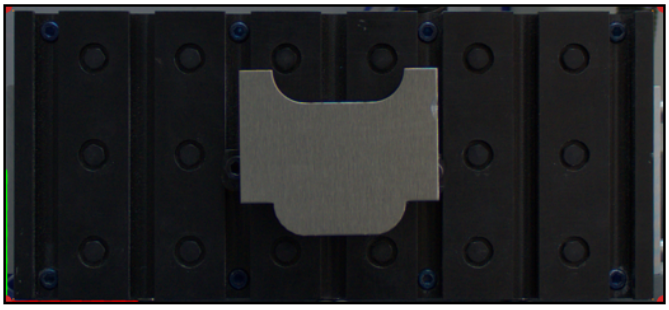
6.3 PIST Algorithm Implementation

The PIST algorithm designs planar contour following paths, which is applied to deburring the periphery of sheet metal parts on the RoboDeb platform. Figure 6.8 A is an image taken of a test part for trajectory design. The first step of the PIST algorithm is to use planar homography to transform the perspective of the camera into a perpendicular view of the clamping plane. The homography matrix between the image plane and clamping plane ${}^{\mathrm{cp}}\mathbf{H}_{\mathrm{I}}$ is calculated using the DLT method. The known correspondences are between the four corners of the clamping plane projected onto the original image and the four desired corners of the orthophoto. The four corners of the clamping plane in the image is illustrated in Fig. 6.8 B drawn as red



A) Camera's Perspective

B) Features of Interest



C) Orthophoto

Figure 6.8: A) Example image of a test part taken by the calibrated monocular camera. B) features of interest detected on the example image C) warped perspective, perpendicular to the clamping plane

dots. Figure 6.8 C displays the results of the perspective transformation. Figure 6.9 A is extracted from the orthophoto using the background subtractor from Zivkovic et al. [58] with static images of the scene without the part present. The background model is then compared to the image taken with the part present, and the dynamic pixels are masked off. Figure 6.9 displays the remainder of the image-space path planning procedure as per Section 6.1.2.

The image-space waypoints are projected into the workspace and are linearly interpolated between. Figure 6.10 displays the generated waypoints described as axes at the end effector positions with the force poses.

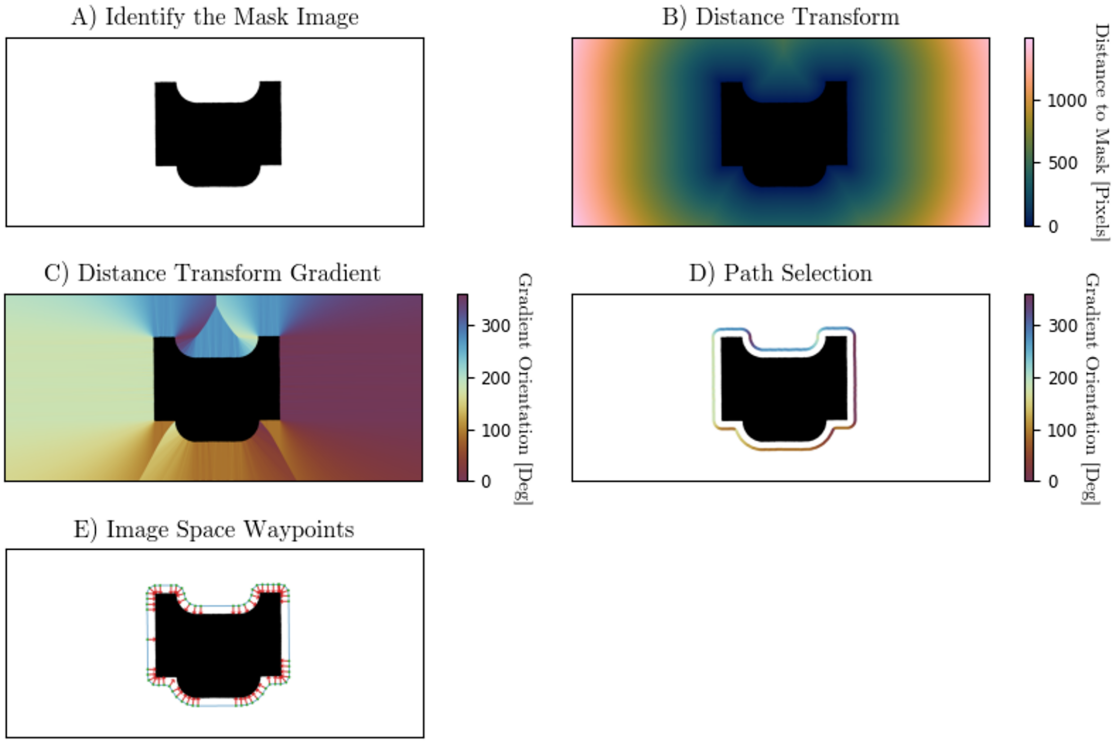


Figure 6.9: See description for Fig. 6.4

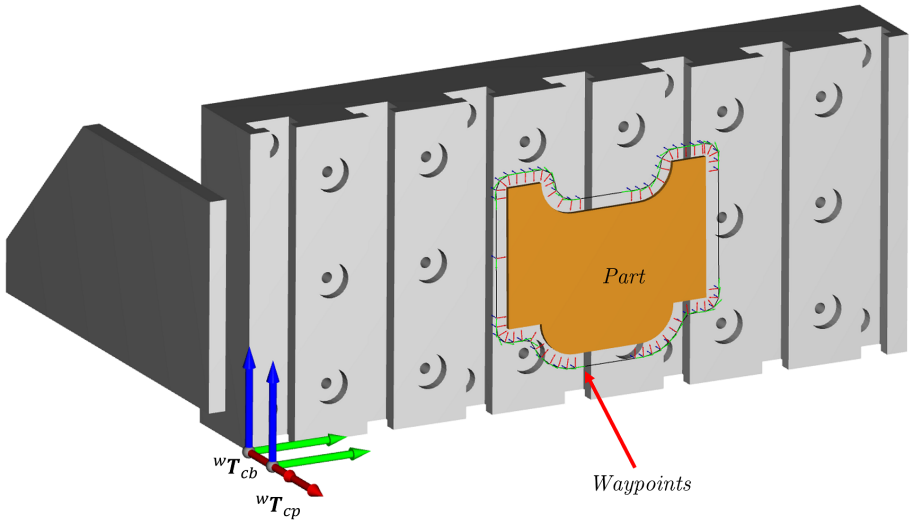


Figure 6.10: Output waypoints of the PIST algorithm displayed as axes at the end effector position and force pose. The waypoints are coarsely spaced for visual purposes.

6.4 The 3D PIST Algorithm Implementation

The 3D PIST algorithm is validated as a proof of concept on the RoboDeb Platform. The validation test will consist of machining the periphery of a test part displayed in Fig. 6.11. An orthophoto used for 3D trajectory planning is obtained referencing an image and point cloud model of the test part.

6.4.1 Orthophoto

Figure 6.12 B displays an orthophoto created using the point cloud model shown in Fig. 6.13 and the image displayed in Fig. 6.12 A. In the orthorectification procedure previously described in Section 2.1.4, it was assumed that there was a continuous model of the 3D surface, and the depth image for orthorectification was obtained by projecting the orthophoto grid points onto the surface. Here, the laser scanner obtains a discretized point cloud. Creating the depth image is slightly differently from Section 2.1.4, and performed by measuring the distance from the points to the orthophoto plane. The depth image is created by first transforming the point cloud into the orthophoto's image-space frame using the similarity transformation ${}^{\mathrm{w}}\mathbf{S}_{\mathrm{cp}}$. The orthophoto image-space frame shares the same origin as the clamping plane coordinate frame ${}^{\mathrm{w}}\mathbf{T}_{\mathrm{cp}}$, at a scaling factor s to convert from pixel units to mm. The z dimension of the 3D points are now the distances between the points to the orthophoto plane, in pixel units. The depth image is populated by scaling the pixel intensity by the as z distance at each x y point in the point cloud with,

$$I_{\mathrm{d}}[\ {}^{\mathrm{cp}}\mathbf{p}_{\mathrm{pc}}[1:2]\ ] = {}^{\mathrm{cp}}\mathbf{p}_{\mathrm{pc}}[3]. \tag{6.9}$$

Figure 6.12 Image A displayes the resulting depth image. Note that there are significant vacancies in the depth image, as the point cloud is sparsely sampled compared

to the orthophoto image. The sparse depth image is interpolated by convolving the image with a maximum kernel, performed using OpenCV's dilate function [64]. The dilate function results in a zero-order hold approximation in a local area around each sampled pixel. Figure 6.12 D displays the result of the dilation. The depth image provides the z dimension for each grid point in the orthophoto, required to perform the remainder of the orthorectification procedure. Figure 6.12 Image B displayes the resulting orthophoto. In the orthophoto, a contour following path is defined in image-space.

6.4.2 Image-space Path Planning

The traditional PIST algorithm is used to isolate the workpiece and design an image-space trajectory. Figure 6.14 displays the results of image-space path planning, which outputs a series of waypoints. Figure 6.12 displays the image-space waypoints projected back into the robot's workspace around the point cloud model of the workpiece.

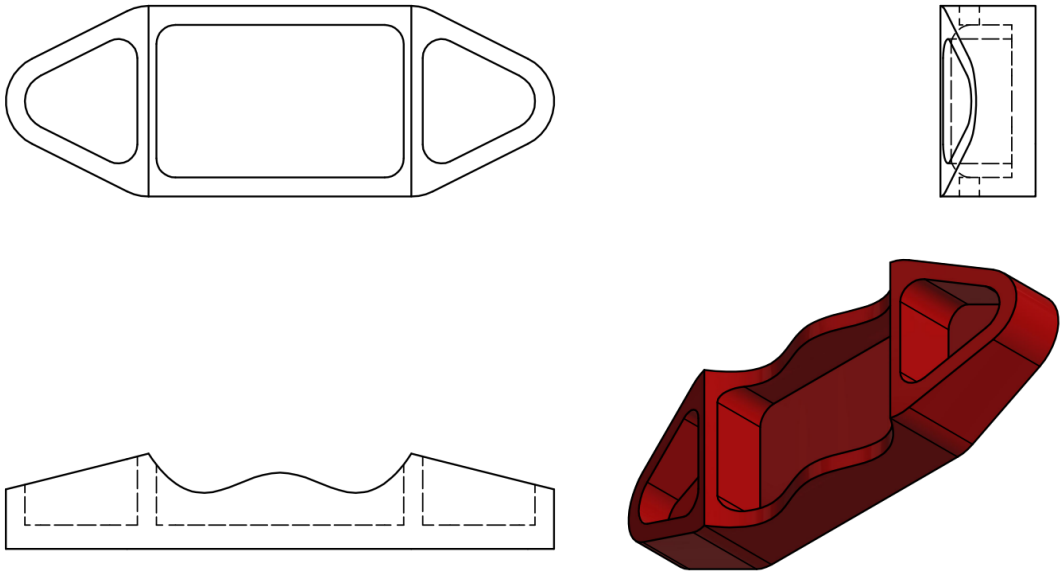


Figure 6.11: Geometry of the 3D test part for validation of the 3D PIST algorithm

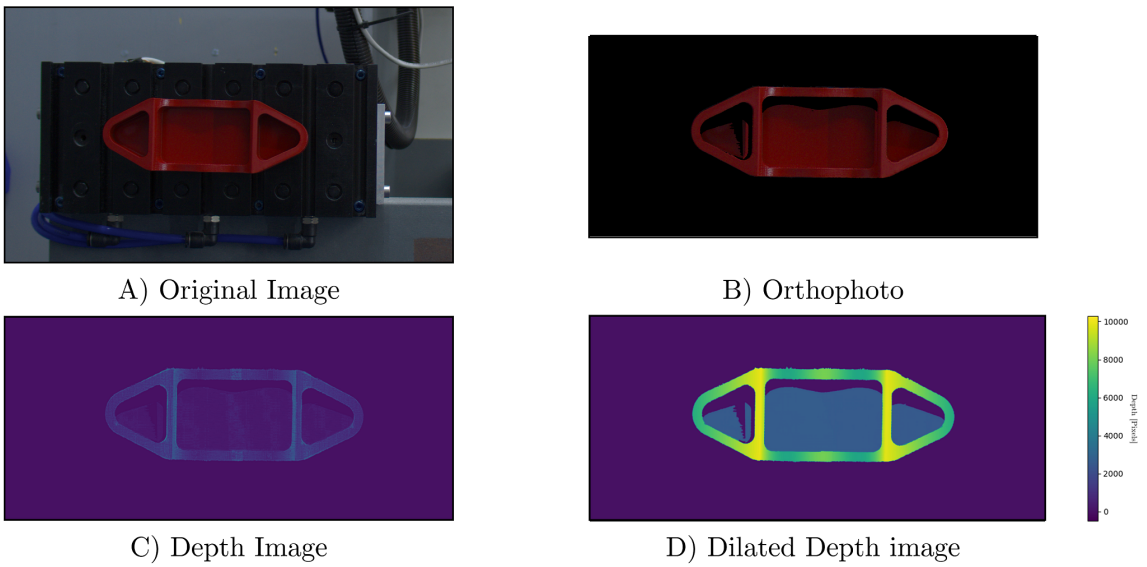


Figure 6.12: Orthorectification on an example image of a 3D part. Image A) is the original image, B) is the resulting orthophoto, C) is the depth image created from the pointcloud data and D) is the processed depth image with vacant locations filled.

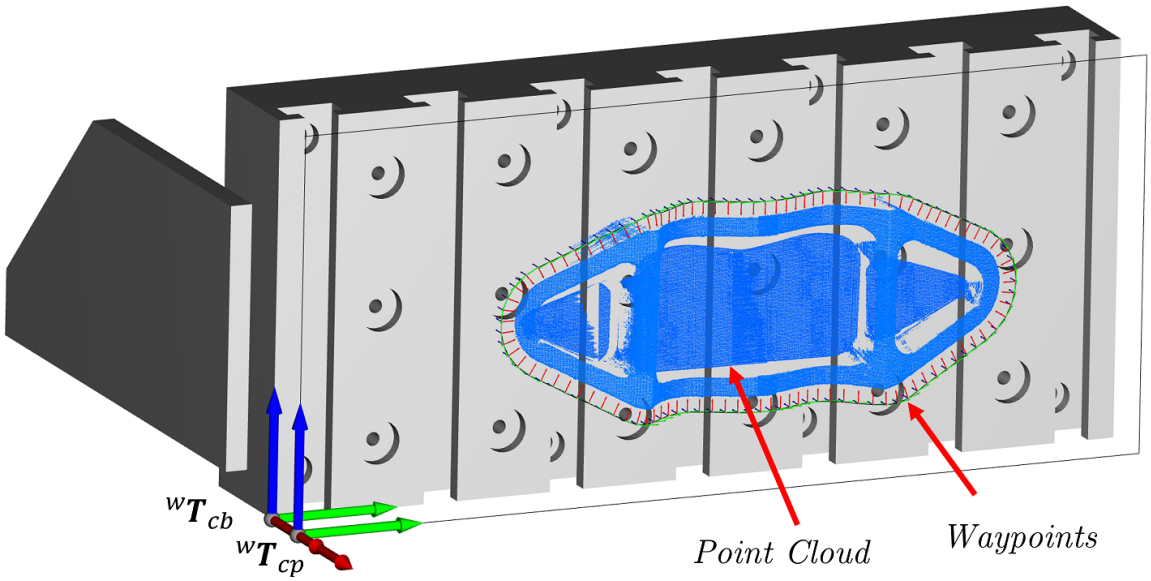


Figure 6.13: Pointcloud obtained of the 3D test part with the resulting output waypoints from the 3D PIST algorithm.

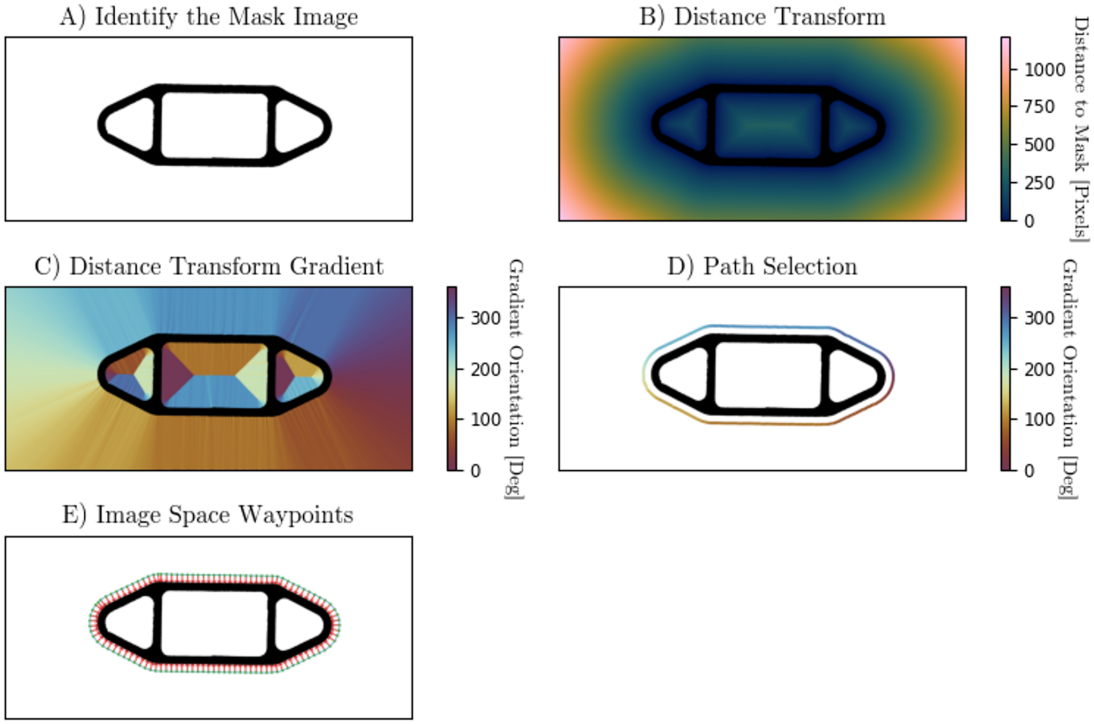


Figure 6.14: See description for Fig. 6.4

6.5 Summary

This chapter proposed the PIST algorithm for designing planar contour following paths for robotic machining using computer vision. The PIST algorithm used a monocular camera to obtain an image of a planar workpiece to track. Path planning was performed in a perpendicular perspective using the distance transform and its gradient to select an offset path with a known normal direction to the workpiece. The normal direction is used to select the applied force direction for machining with a force control scheme. The image-space path was exported into a set of waypoints and mapped into the robot's workspace for playback. The 3D PIST algorithm proof of concept was presented using orthorectification to create a 2D image representation of the 3D surface. In the orthophoto, image-space path planning was performed identically to the PIST algorithm to create a set of waypoints for playback. The PIST algorithm and its 3D extension is implemented on the RoboDeb platform to perform robotic deburring on machined parts. The results of robotic deburring using the PIST algorithm is presented in Chapter 7 of this thesis.

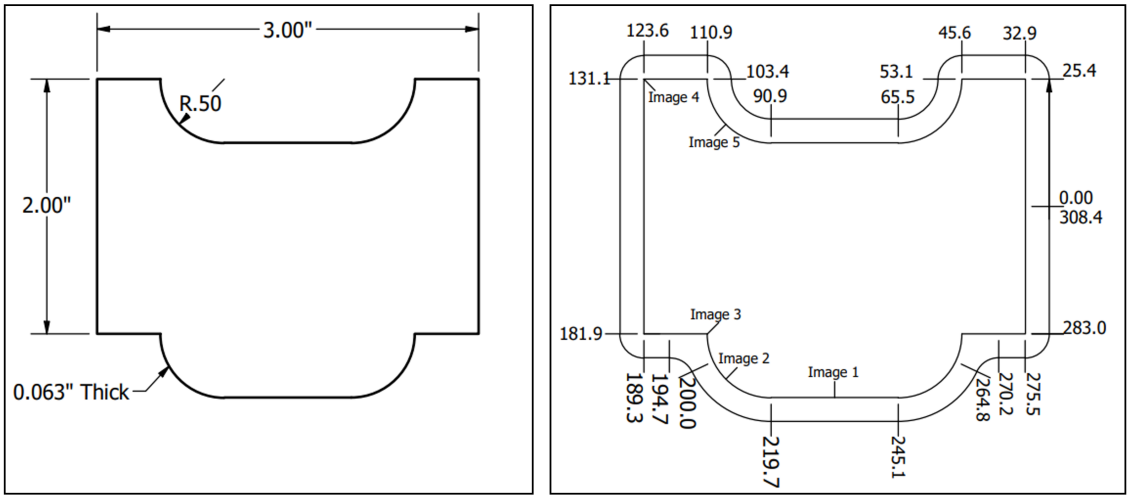
Chapter 7

Deburring Experimentation

The experimentation presented in the following chapter will apply the trajectory planners developed in this thesis to robotic deburring using the RoboDeb platform. The goal of the experimentation is to validate the PCAT and PIST algorithms and determine the preferred approached for deburring in an industrial setting. The experimentation begins with Section 7.1 in which the PIST and PCAT algorithms are utilized to deburr sheet metal parts. In Section 7.2 the 3D PIST algorithm is used to machine the periphery of a 3D test part.

7.1 Planar Experimentation

The three planar trajectory planning approaches investigated were the PCAT algorithm with laser localization, PCAT algorithm with camera localization, and the PIST algorithm using a single camera. Each trajectory planning approach was used to deburr three test parts at separate locations on the vacuum clamp. Figure 7.1 A displays the geometry of the test part used for each test. The PCAT and PIST algorithms designed comparable offset trajectories with similar force and feedrate



A) Test Part Geometry B) Distance Along Deburring Path

Figure 7.1: A) The test part geometry and dimensions in inches and B) the cumulative distance around the offset path in mm. B) also includes the locations images were taken at in Fig. 7.2.

modulation. Figure 7.1 B displays the offset path with reference to the cumulative distance around the part (in mm).

The RoboDeb Platform used for deburring was described in Chapter 4. Admittance control was used to maintain a 2 N contact force on the part to achieve a consistent surface finish. Deburring was performed with an abrasive $3/8 inch$ dia. flap wheel tool at an observed feedrate of 5 mm/s. The flap wheel tool was selected as it is compliant, complementing the admittance controller used. The flap wheel tool eroded as it was used to expose fresh abrasive material, preventing the aluminum debris from gumming up the tool. Each trajectory planning approach used a single tool, indexed in the end effector z axis, after each test to expose a fresh location on the tool. The Robodeb platform with the test part loaded is displayed in Chapter 4, Fig. 4.2. The procedure for deburring with the PCAT algorithm was as follows.

1. Place the test part in the workspace.
2. Plan a local deburring trajectory offline using the CAD data. Displayed in

Chapter 5, Fig. 5.2.

3. Obtain a 3D point cloud of the test part by either:
   (a) Scanning the part with the laser on the end effector of the robot. Chapter 4, Fig. 4.5 displays an example point cloud obtained with the laser scanner.
   (b) Capturing the part location with the monocular camera external to the robot. Chapter 4, Fig. 4.8 displays an example point cloud obtained with the monocular camera.
4. Fit a CAD file to the point cloud with an ICP algorithm. Chapter 5, Fig. 5.8 displays the test part registration.
5. Transform the local trajectory into the workspace based on the part localization. Chapter 5, Fig. 5.9 displays the workspace trajectory from the PCAT algorithm.
6. Perform the deburring trajectory.

The PIST algorithm further simplifies trajectory planning by detecting the part using the monocular camera instead of using CAD data. The procedure for the PIST algorithm was as follows.

1. Place the test part in the workspace.
2. Obtain an image of the part on the vacuum clamp. Chapter 6, Fig. 6.8 A displays an example image of the test part used for planning.
3. Generate the trajectory using the PIST algorithm. Chapter 6, Fig. 6.9 displays the image-space path planning procedure and Fig. 6.10 displays the output waypoints.
4. Perform the deburring trajectory.

With the tests performed, the ideal trajectory planning candidate for robotic deburring is investigated based on the achieved results.

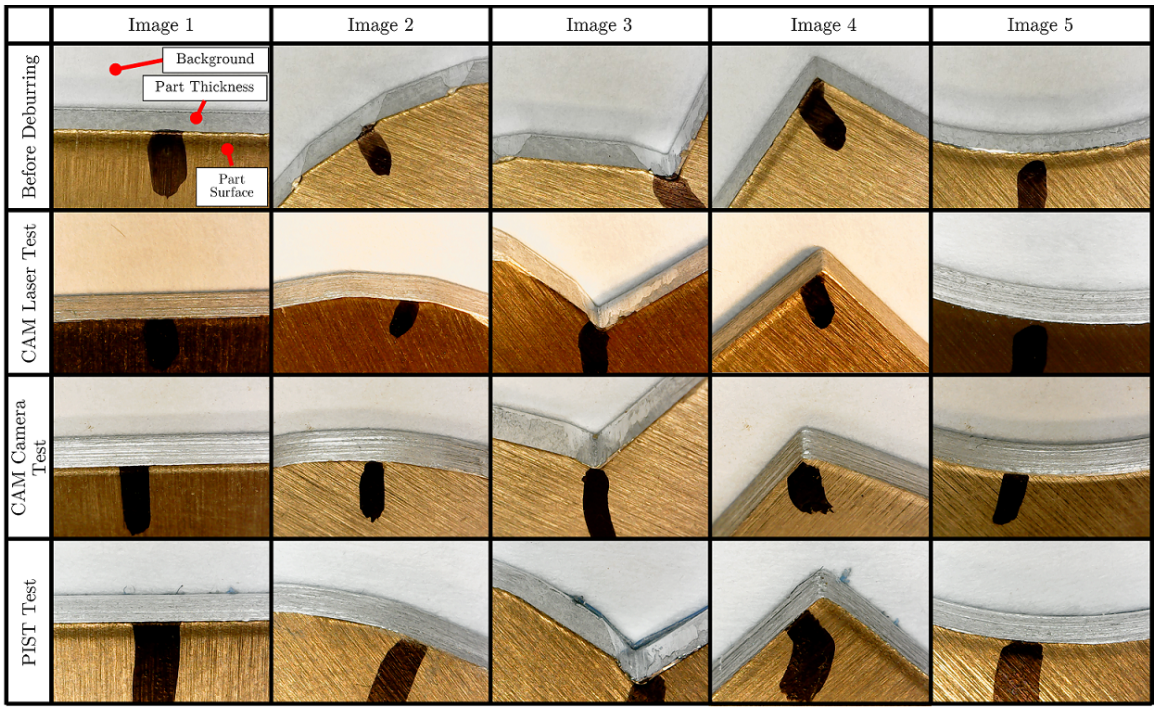


Figure 7.2: Images of the surface finish before and after deburring for all trajectory planning approaches. See Fig. 7.1 for image locations on the part.

7.1.1 Results Summary

The goal of the experimentation was to validate the PIST and PCAT algorithms and determine the preferred trajectory planner for robotic deburring. Figure 7.3 displays the results, which includes the force and position error for each test plotted against the distance travelled along the path. Figure 7.1 provides a reference for the tool position and image location on the part. For the three methods, figure 7.2 displays the surface finish of the deburred part at key positions. The metrics that the trajectory planners were evaluated on are: the position error in the detected path, the error in the applied force, the process time, and the part finish. All trajectory planners successfully deburred the sheet metal parts with admittance control, with indistinguishable resulting surface finishes.

Table 7.1: Positional Error results summary

	PCAT Laser Localization			PCAT Camera Localization			PIST		
	Test 1	Test 2	Test 3	Test 1	Test 2	Test 3	Test 1	Test 2	Test 3
max	0.981	0.964	1.144	0.826	1.176	1.144	1.218	1.201	1.098
mean	0.620	0.720	0.773	0.360	0.546	0.774	0.793	0.833	0.734
range	0.808	0.557	0.859	0.964	1.364	0.859	1.105	0.774	1.086
STD	0.207	0.153	0.268	0.309	0.292	0.266	0.242	0.115	0.277

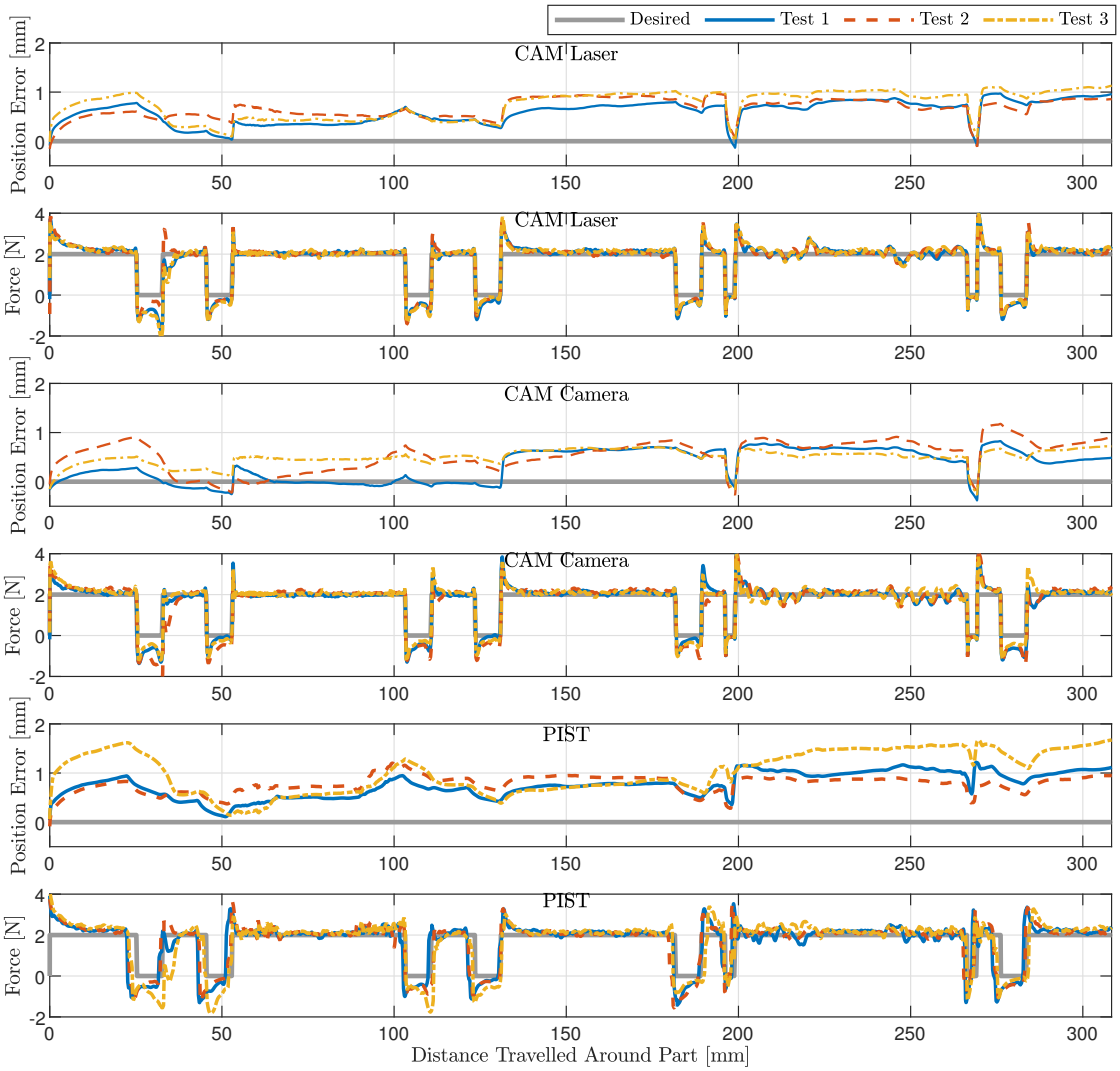


Figure 7.3: The positional error and applied force along the deburring path for the CAM tests with the laser localization (above), with the camera localization (middle), and the PIST tests (below). All force data displayed was low pass filtered at 5 Hz.

7.1.2 Detected Path Position Error

Admittance control was used to improve the edge tracking accuracy of the robot by admitting a positional error in the pre-defined path to maintain a set contact force. The positional accuracy of each trajectory planner was evaluated by measuring the tool's positional error, as it deviated from the defined path to contact the part. Figure 7.3 contains the positional error along the path for each trajectory planner. The max, mean and range of the positional error while the force was applied is displayed in Table 7.1. The error range and standard deviation (STD) were the best indications of the path accuracy, as it was not affected by inaccuracies in the initial tool diameter, which differed amongst tests. The average range/STD of the PCAT with laser localization, PCAT with camera localization and PIST tests were 0.74/0.209, 1.06/0.289, and 0.99/0.211 mm, respectfully. The positional accuracy reported was affected by the compliance and wear of the flap wheel tool, thus the actual positional accuracy of each trajectory planner is understated. The tool wear reduced the tool radius and increased the positional error as each test progressed. The tool wear is observed in Fig. 7.3 by the increasing positional accuracy as each test proceeds. Based on a first order fit of the positional error, it was estimated that the tool wears on average 0.5 mm for each test. The key outcomes from the positional error results are:

- The positional accuracy for all trajectory planning approaches was sufficient for satisfactory deburring with admittance control.
- The laser scanner proved to be more accurate than the camera system used as it provided a 0.28 mm narrower range and 0.08 mm lower standard deviation.
- The PIST tests and the PCAT tests with camera localization used the same camera and the PIST algorithm achieved an equivalent error range, and smaller standard deviation. The similar positional results indicated that the part de-

tection and path design in image-space did not introduce any significant errors, compared to path planning with respect to an accurate CAD model.

The positional accuracy was deemed to be the least important metric, as it did not directly affect the part finish when deburring with admittance control.

7.1.3 Applied Force Error

The applied force onto the part has a significant impact on the part finish, as presented in Bone et al. [67–70] who correlated surface roughness to measured force. Figure 7.3 also displays the magnitude of the applied force along the path. There were two minor differences observed between the trajectory planners performance in the applied force. The first difference was in the magnitude in the spikes in the applied force at sharp corners (occurred at 32.9, 53.1, 110.9, 131.1, 189.3, 200.0, 270.2,283.0 mm). The spikes were 5% larger for the PCAT tests with camera localization than laser localization. The second difference occurred on the PIST algorithm tests at internal arc features at $55-65$ mm and $90-100$ mm where there were minor fluctuation in the applied force. The key outcomes from the applied force error results are:

- The admittance controller successfully maintained the set contact force of 2 N for all trajectory planners.
- With admittance control, a large change in the positional error caused a spike in the applied force. The camera localization had lower accuracy than laser localization and therefore larger changes in the positional errors at corners, causing larger force spikes.
- The PIST algorithm discretized arcs into multiple linear segments, which reduced the positional accuracy at nonlinear features and caused fluctuations in the applied force.

At the external arcs at $200 - 220$ mm and $245 - 265$ mm there were oscillations in the applied force, displayed on both trajectory planners. Figure 7.2 displays close-up images of the test part before and after deburring, and Image 2 was taken at the external arc between $200 - 220$ mm. Before deburring, there were prominent burr features at the external arcs which caused fluctuations in the applied force. The applied force differences between trajectory planners were minor and therefore not reflected in the surface finish.

7.1.4 Process Time

All deburring tests required the robot to be in operation for 100 s. The PCAT algorithm required the part to be localized in the workspace. The laser localization required the robot to perform a 20 s scanning path increasing the process time by 20%, while camera localization and the PIST tests did not. The PIST algorithm further reduced the setup time as it did not require a predefined DXF model of the workpiece. The faster process time and setup time makes the PIST algorithm beneficial for automating deburring on a production line.

7.1.5 Part Finish

Figure 7.2 displays close-up images of the parts edges before and after deburring to display the surface finish. Figure 7.1 provides a reference for the image location on the part. The PCAT and PIST algorithms were able to remove burrs along the part and achieve a smooth and consistent surface finish at all accessible locations. At sharp internal corners, the tool cannot contact the part and the surface finish was untouched. At sharp external corners, the force was removed, and the corners were not rounded. With admittance control, the discrepancy in the positional accuracy and minor fluctuations in the applied force did not alter the surface finish. The surface finish between all trajectory planners were indistinguishable. The benchmark for

trajectory planning for robotic deburring is using CAD/CAM software with a laser scanner for localization. The monocular camera provided adequate positional accuracy to the laser scanner, at a fraction of the cost, without a scanning pass performed by the robot. The initial cost and scanning time associated with the laser scanner was not justified as the small improvement in accuracy did not significantly improve the final deburring results. The PIST algorithm provided equivalent surface finish results, with the benefits of not requiring pre-emptive knowledge of the workpiece and adapting to manufacturing imperfections. The PIST algorithm provided equivalent positional accuracy compared to the PCAT algorithm with camera localization, without referencing a CAD model. Thus, the PIST algorithm was the preferred approach for designing deburring trajectories for sheet metal parts.

7.2 Three Dimensional Experimentation

The following section details the experimentation performed to verify the proof of concept 3D PIST algorithm by machining the periphery of a 3D test part. Most off-the-shelf CAD/CAM software use a CAD model to design 3D trajectories, while the 3D PIST algorithm uses a an observed point cloud model and image. The 3D PIST algorithm is a proof of concept and future work will optimize the trajectory planner. The experiment procedure is detailed in Section 7.2.1 and the results are presented in Section 7.2.2.

7.2.1 Procedure

The 3D deburring experimentation is again performed on the RoboDeb platform using the same tool and process parameters. The 3D test part used for machining was designed to emulate a typical pocket milled structural component. The 3D test part geometry is displayed in Chapter 6, Fig. 6.11, which is 3D printed black and painted

red. When machined, the red paint is removed to highlight the machined locations. The procedure for the 3D PIST algorithm was as follows.

1. Place the test part in the workspace.
2. Obtain an image of the part on the vacuum clamp. Chapter 6, Fig. 6.12 A displays an example image of the 3D test part.
3. Obtain a 3D point cloud of the test part by scanning the part with the laser scanner. Chapter 6, Fig. 6.13 displays an example point cloud of the 3D test part.
4. Obtain the orthophoto and depth image. Chapter 6, Fig. 6.12 B displays an example orthophoto created from the image and point cloud data.
5. Generate the trajectory using the PIST algorithm. Chapter 6, Fig. 6.14 displays the image-space path planning procedure on the 3D test part.
6. Perform the deburring trajectory.

7.2.2 Results Summary

Figure 7.5 displays the results which detail the position error for each contour against the distance travelled. Figure 7.6 displays an image of the part before and after machining. Figure 7.4 provides a reference for the contour location and tool position along the part. The 3D PIST algorithm was evaluated based on the position error in the detected path, the error in the applied force, and the machining result. The 3D PIST algorithm successfully tracked the contour of the part and accounted for out-of-plane variations in the flange height, with comparable positional and applied force results to the original PIST algorithm.

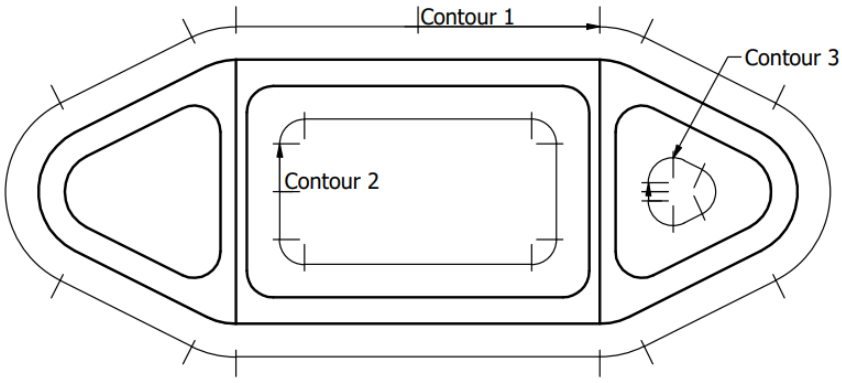


Figure 7.4: Part position as the tool travels around the part in mm and the locations images were taken at in Fig. 7.6 .

7.2.3 Detected Path Position Error

Figure 7.3 displays the positional accuracy of the 3D PIST algorithm as the tool deviated from the defined trajectory to contact the part. The maximum, mean, STD and range of the positional error was 0.73, 0.16, 0.22 and 1.07 mm respectively for all contours combined. The positional error results are similar to the original PIST results and are sufficient for machining using the admittance controller.

7.2.4 Applied Force Error

Figure 7.3 also displays the magnitude of the applied force along the trajectory before and after a lowpass filter at a 5 Hz cut-off frequency. The 3D PIST algorithm closely maintained the set contact force of 2 N for adequate machining results.

7.2.5 Machining Result

Figure 7.6 displays the test part before and after machining. The tool path is identified as locations where machining removed the red paint. The 3D PIST algorithm successfully tracked the contour of the part and accounted for out-of-plane variations

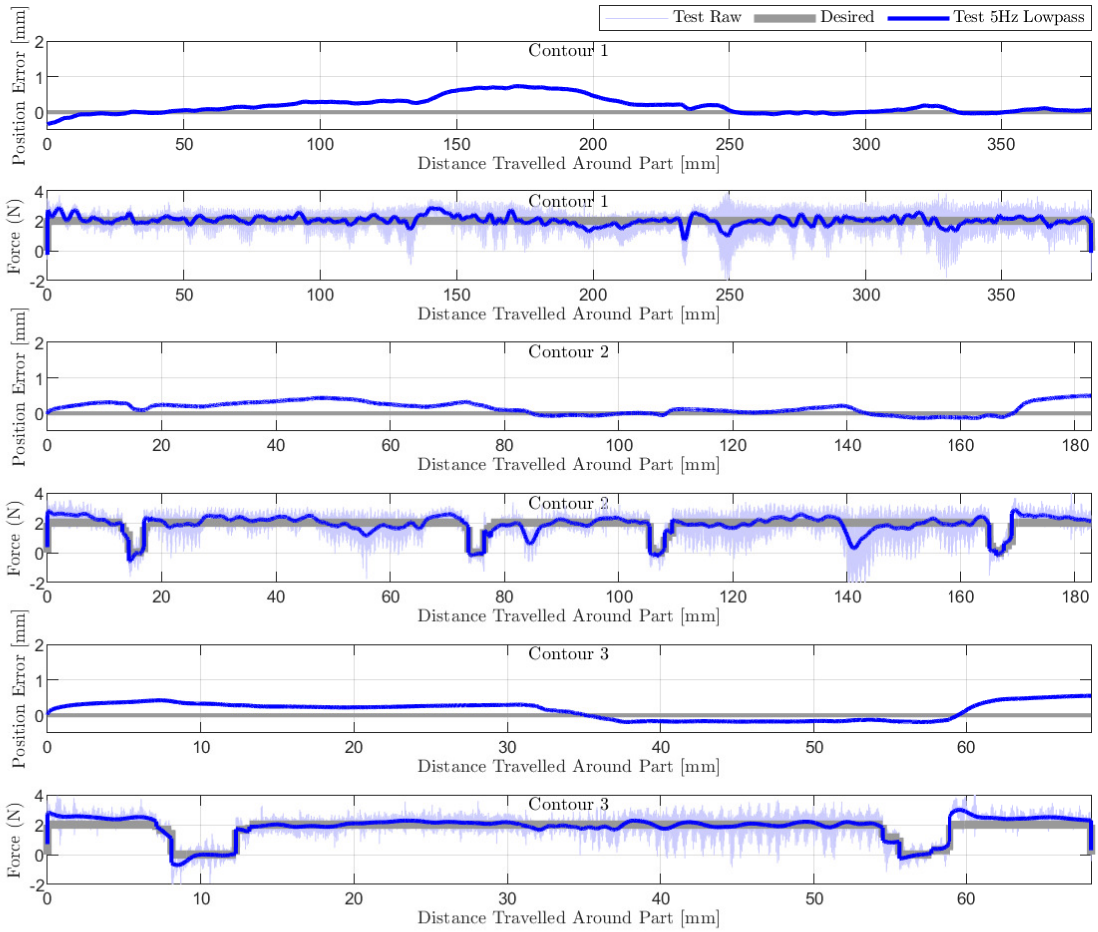


Figure 7.5: The positional error and applied force along the machining trajectory for the CAM tests (above) and the PIST tests (below).

in the flange height. The successful machining validates the 3D PIST algorithm as a proof of concept.

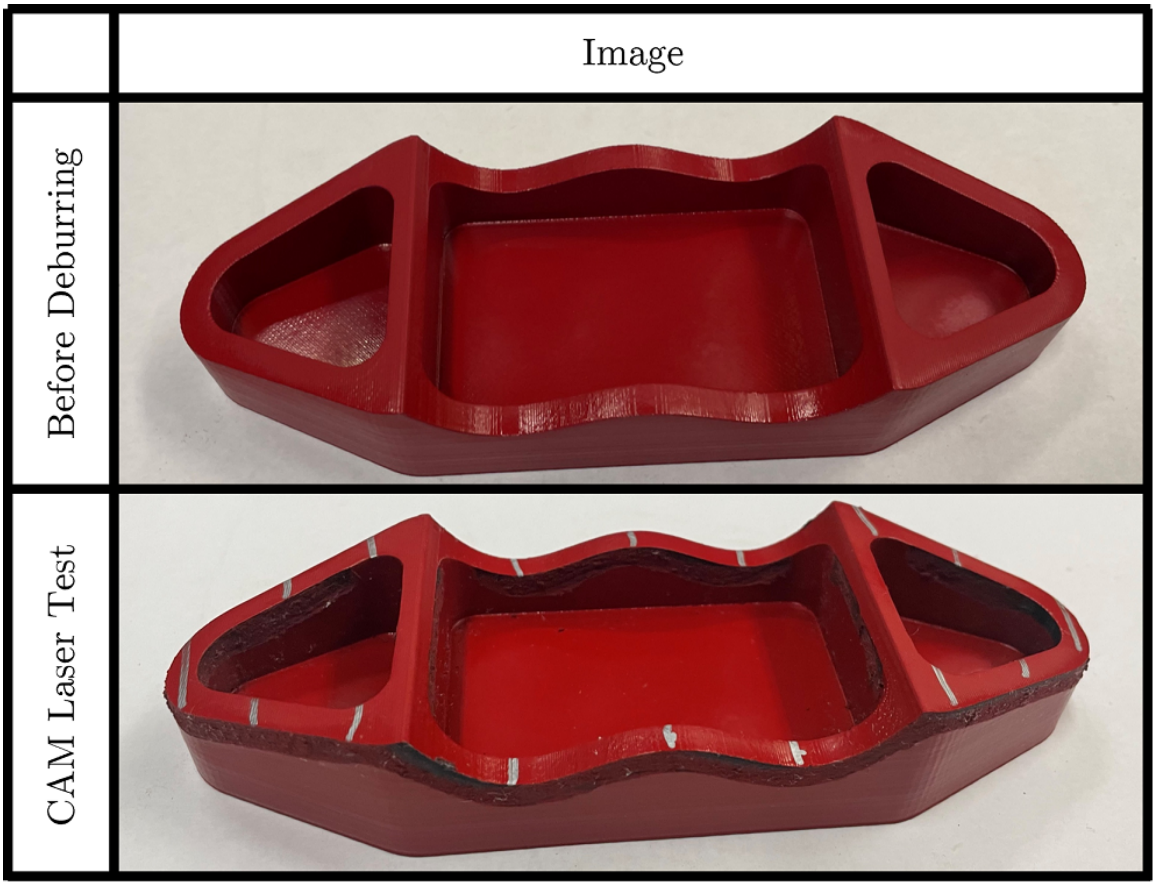


Figure 7.6: Images of the 3D test part before and after paint removal.

7.3 Summary

This chapter described the experimentation performed on the Robodeb platform using the PCAT and PIST trajectory planners for validation. The PCAT and PIST algorithms were used to design deburring trajectories around the periphery of planar sheet metal parts. The accuracy of each trajectory planner was evaluated using the range and standard deviation of the positional error along the trajectory. The average range/STD of the PCAT with laser localization, PCAT with camera localization and PIST tests were 0.74/0.209, 1.06/0.289, and 0.99/0.211 mm, respectfully. The laser scanner proved to be more accurate than the camera system as it provided a 0.2840 narrower range. The PIST tests and the PCAT tests with camera localization, used the same camera and achieved an equivalent error range, indicating that image-space path planning did not introduce significant errors in the PIST algorithm. All trajectory planners successfully deburred the sheet metal parts with admittance control, with indistinguishable resulting surface finishes. The preferred localization method for the PCAT algorithm was using the camera over the laser scanner for its lower cost and process time. The PIST algorithm proved to be the preferred candidate for robotic deburring, as it provided identical machining results, without an expensive laser scanner or a CAD model of the part. The 3D PIST algorithm successfully machined the periphery of a test part with varying flange heights, proving its viability for future 3D applications.

Chapter 8

Conclusion

The objectives of this theses were to:

1. Develop the PCAT algorithm for planar contour following with force control using CAD data for trajectory design.
2. Develop the PIST algorithm to design equivalent contour following trajectories using computer vision.

8.1 PCAT Algorithm

In chapter 4, the PCAT algorithm was developed using a CAD/CAM approach to establish a benchmark for planar contour following trajectory planning. A DXF file was used to define the geometry of the workpiece. The PCAT algorithm read the DXF file and generated an offset path with a known normal direction to apply force. The local path was transformed into the RoboDeb's workspace using a localization procedure. Localization was performed by obtaining a point cloud model using the laser scanner or monocular camera and fitting a CAD model to the observed point cloud. The tool's pose along the designed path could be changed to create deburring

or chamfering paths. The designed workspace trajectory was expressed as a series of waypoints.

8.2 PIST Algorithm

In chapter 5, the PIST algorithm was developed which used computer vision to design planar contour following trajectories. The PIST algorithm used a monocular camera to provide the spatial information required for path design and created equivalent trajectories to the PCAT algorithm. The PIST algorithm's novelties were the use of a perspective transformation to obtain a to scale view of the planar workpiece and the image-space path planning procedure. Image-space path planning was performed by identifying the workpiece and using the distance transform and its gradient to select an offset path with a normal direction. The PIST algorithm was extended into 3D space as a proof of concept. The 3D PIST algorithm used orthorectification to obtain a perpendicular, to scale, perspective of the 3D workpiece for image-space path planning. The benefit of the 3D PIST algorithm was that it still did not require a CAD model of the workpiece. The planar and 3D PIST algorithms were applied to the RoboDeb platform for robotic deburring, using the fixed camera system for planar planning, and both the camera and the laser scanner for 3D planning.

8.3 Deburring Experimentation

In chapter 6, experimentation was performed on the RoboDeb platform for validation. Deburring was performed using the PIST algorithm, and the PCAT algorithm. For the PCAT algorithm, the laser scanner and monocular camera were both used to capture a point cloud for localization and their accuracy was compared. The metrics that the trajectory planners were evaluated on are: the position error in the detected path, the error in the applied force, the process time, and the part finish. The key

results of the experimentation are:

- The laser scanner proved to be slightly more accurate than the monocular camera.
- The PIST tests and the PCAT tests with camera localization, used the same camera and achieved equivalent positional error results, indicating that the image-space trajectory planning did not introduce errors in the designed trajectory.
- With admittance control, the discrepancy in the positional accuracy between camera and laser localization only had a minor effect on the applied force.
- The laser localization increased the process time by 20% due to the required scanning pass.
- All PIST and PCAT tests successfully deburred the parts and exhibited an identical surface finishes.

The initial cost and scanning time associated with the laser scanner was not justified, therefore the monocular camera was preferred for localization. The PIST algorithm was the preferred approach for planar trajectory planning, as it used a single low-cost camera and provided comparable accuracy to the PCAT algorithm. The greatest benefit of the PIST algorithm was that it created trajectories without CAD data and could adapt to imperfections perceived on the part. The 3D PIST algorithm was used to machine a test part with flanges of varying heights for validation. The successful results prove that there are further applications of the PIST algorithm for 3D trajectory planning in future work.

8.4 Future Work

This thesis introduced the novel PIST algorithm, which can be improved to address future research in industrial applications. The following are suggestions for

future improvements:

1. The PIST algorithm can be improved by defining both line and arc waypoints, similar to the PCAT algorithm. The current PIST algorithm discretizes the trajectory into a series of line waypoints which are linearly interpolated between. This linear approximation caused inaccuracies in the trajectory which would be improved by identifying arc waypoints.
2. The PIST planner was created for planar contour following but is easily extended to surface path planning. The offset path is created using the distance transform, similar to Xu et al. [65] for surface path planning in pocket milling applications. The current PIST algorithm as is, can create surfacing paths by defining multiple offset paths on the part surface. Future work would be to validate and refine the PIST algorithm for surface path planning.
3. In 3D contour following or surfacing trajectories, it may be beneficial to keep the tool normal to the part's surface by changing the end effector pose. The normal direction to the part must be estimated along the path and the end effector z axis would be changed accordingly. Point cloud processing libraries like open3D [62] already have functions to estimate the normal direction to a point cloud or CAD model, as it is required for ICP registration with the point-to-plane metric. Figure 8.1 displays the x y and z components of the estimated normal directions to the point cloud model of the 3D test part. Future work would reference this data to change the end effector pose.
4. The 3D PIST algorithm currently uses the laser scanner to obtain a point cloud model of the workpiece to create an orthophoto. An improvement would be to remove the laser scanner and use the stereo camera system alone to generate a depth image for orthorectification. The issue with the current stereo camera setup is that there is not enough texture in the scene to generate a surface with stereo matching. An improvement would be to add structured light to the scene

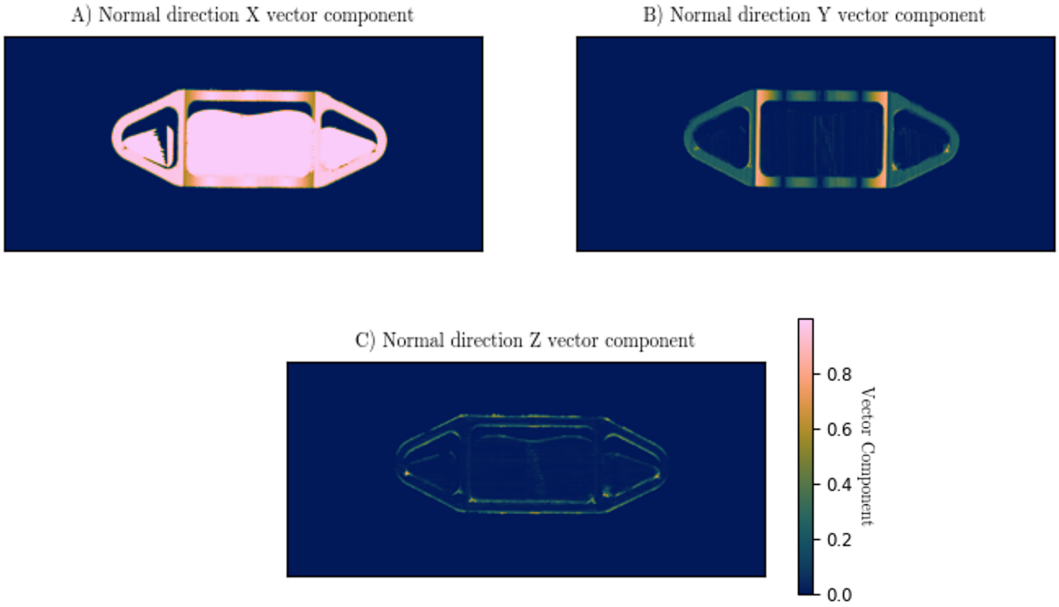


Figure 8.1: The estimated normal directions of the 3D test part's point cloud model. The normals are expressed as images indicating the A) x B) y and C) z components of the unit normal vectors.

to aid in stereo matching.

Bibliography

[1] I. F. Onstein, O. Semeniuta, and M. Bjerkeng, "Deburring Using Robot Manipulators: A Review," *2020 3rd International Symposium on Small-Scale Intelligent Manufacturing Systems, SIMS 2020*, 2020. doi: 10.1109/SIMS49386.2020.9121490

[2] H. Kosler, U. Pavlovčič, M. Jezeršek, and J. Možina, "Adaptive Robotic Deburring of Die-Cast Parts with Position and Orientation Measurements Using a 3D Laser-Triangulation Sensor," *Strojniski Vestnik/Journal of Mechanical Engineering*, vol. 62, no. 4, pp. 207–212, 2016. doi: 10.5545/sv-jme.2015.3227

[3] A. Brunete, E. Gambao, J. Koskinen, T. Heikkilä, K. B. Kaldestad, I. Tyapin, G. Hovland, D. Surdilovic, M. Hernando, A. Bottero, and S. Anton, "Hard material small-batch industrial machining robot," *Robotics and Computer-Integrated Manufacturing*, vol. 54, no. March 2017, pp. 185–199, 2018. doi: 10.1016/j.rcim.2017.11.004

[4] A. Verl, A. Valente, S. Melkote, C. Brecher, E. Ozturk, and L. T. Tunc, "Robots in machining," *CIRP Annals*, vol. 68, no. 2, pp. 799–822, 2019. doi: 10.1016/j.cirp.2019.05.009

[5] S. Lloyd, R. Irani, and M. Ahmadi, "A numeric derivation for fast regressive modeling of manipulator dynamics," *Mechanism and Machine Theory*, vol. 156,

p. 104149, 2021. doi: https://doi.org/10.1016/j.mechmachtheory.2020.104149

[6] S. Lloyd, R. A. Irani, and M. Ahmadi, "Fast and robust inverse kinematics of serial robots using halleys method," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 2768–2780, 2022. doi: 10.1109/TRO.2022.3162954

[7] G. Miller, "Pilot study for cutting force model of robotic deburring by," 2020.

[8] M. Dehghani, R. A. Mckenzie, R. A. Irani, and M. Ahmadi, "Robot-mounted sensing and local calibration for high-accuracy manufacturing," *Robotics and Computer-Integrated Manufacturing*, vol. 79, no. April 2022, p. 102429, 2023. doi: 10.1016/j.rcim.2022.102429

[9] H. Zhang, H. Chen, N. Xi, G. Zhang, and J. He, "On-line path generation for robotic deburring of cast aluminum wheels," *IEEE International Conference on Intelligent Robots and Systems*, pp. 2400–2405, 2006. doi: 10.1109/IROS.2006.281679

[10] K. M. Murphy, R. J. Norcross, and F. M. Proctor, "CAD directed robotic deburring," *Proceedings of the second international symposium on robotics and manufacturing research, education, and applications, Albuquerque, NM*, 1988.

[11] H. C. Song, B. S. Kim, and J. B. Song, "Tool path generation based on matching between teaching points and CAD model for robotic deburring," *IEEE/ASME International Conference on Advanced Intelligent Mechatronics, AIM*, pp. 890–895, 2012. doi: 10.1109/AIM.2012.6265921

[12] J. Hu, A. M. Kabir, S. M. Hartford, S. K. Gupta, and P. R. Pagilla, "Robotic deburring and chamfering of complex geometries in high-mix/low-volume production applications," *IEEE International Conference on Automation Science and Engineering*, vol. 2020-Augus, pp. 1155–1160, 2020. doi: 10.1109/CASE48305.2020.9217042

[13] R. Béarée, J. Y. Dieulot, and P. Rabaté, "An innovative subdivision-ICP registration method for tool-path correction applied to deformed aircraft parts machining," *International Journal of Advanced Manufacturing Technology*, vol. 53, no. 5-8, pp. 463–471, 2011. doi: 10.1007/s00170-010-2875-0

[14] A. Kuss, M. Drust, and A. Verl, "Detection of Workpiece Shape Deviations for Tool Path Adaptation in Robotic Deburring Systems," *Procedia CIRP*, vol. 57, pp. 545–550, 2016. doi: 10.1016/j.procir.2016.11.094

[15] E. Villagrossi, C. Cenati, N. Pedrocchi, M. Beschi, and L. Molinari Tosatti, "Flexible robot-based cast iron deburring cell for small batch production using single-point laser sensor," *International Journal of Advanced Manufacturing Technology*, vol. 92, no. 1-4, pp. 1425–1438, 2017. doi: 10.1007/s00170-017-0232-2

[16] K. Micallef, , G. Fang, , and M. Dinham, "Automatic Seam Detection and Path Planning in Robotic Welding," *Robotic Welding, Intelligence and Automation*, pp. 23–32, 2011. doi: https://doi.org/10.1007/978-3-642-19959-2_3

[17] H. N. M. Shah, M. Sulaiman, A. Z. Shukor, Z. Kamis, and A. A. Rahman, "Butt welding joints recognition and location identification by using local thresholding," *Robotics and Computer-Integrated Manufacturing*, vol. 51, no. December 2017, pp. 181–188, 2018. doi: 10.1016/j.rcim.2017.12.007

[18] W. C. Chang, M. Y. Cheng, and H. J. Tsai, "Image feature command generation of contour following tasks for SCARA robots employing Image-Based Visual Servoing—A PH-spline approach," *Robotics and Computer-Integrated Manufacturing*, vol. 44, pp. 57–66, 2017. doi: 10.1016/j.rcim.2016.08.002

[19] M. Jinno, M. Uenohara, J. Oaki, and K. Tatsuno, "Teaching-less robot system for finishing workpieces of various shapes using force control and computer vision,"

IEEE International Conference on Intelligent Robots and Systems, vol. 1, pp. 573–578, 1999. doi: 10.1109/iros.1999.813065

[20] B. Solvang, P. Korondi, G. Sziebig, and N. Ando, "SAPIR: Supervised and adaptive programming of industrial robots," *INES 2007 - 11th International Conference on Intelligent Engineering Systems, Proceedings*, pp. 281–286, 2007. doi: 10.1109/INES.2007.4283712

[21] B. Solvang, G. Sziebig, and P. Korondi, "Vision based robot programming," *Proceedings of 2008 IEEE International Conference on Networking, Sensing and Control, ICNSC*, pp. 949–954, 2008. doi: 10.1109/ICNSC.2008.4525353

[22] S. Pagano, R. Russo, and S. Savino, "A vision guided robotic system for flexible gluing process in the footwear industry," *Robotics and Computer-Integrated Manufacturing*, vol. 65, no. March, p. 101965, 2020. doi: 10.1016/j.rcim.2020.101965

[23] A. Z. Richard Hartley, *Multiple View Geometry in Computer Vision*, 2nd ed. New York: Cambridge University Press, 2000. ISBN 9780521540513

[24] E. Malis, G. Chesi, and R. Cipolla, "2 1/2 D visual servoing with respect to planar contours having complex and unknown shapes," *International Journal of Robotics Research*, vol. 22, no. 10-11, pp. 841–853, 2003. doi: 10.1177/027836490302210004

[25] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000. doi: 10.1109/34.888718

[26] M. J. Tsai, H. W. Lee, and N. J. Ann, "Machine vision based path planning for a robotic golf club head welding system," *Robotics and Computer-Integrated Manufacturing*, vol. 27, no. 4, pp. 843–849, 2011. doi: 10.1016/j.rcim.2011.01.005

[27] Z. Wang, W. Wu, X. Xu, and D. Xue, "Recognition and location of the internal corners of planar checkerboard calibration pattern image," *Applied Mathematics and Computation*, vol. 185, no. 2, pp. 894–906, 2007. doi: 10.1016/j.amc.2006.05.210

[28] J. He, J. Xia, X. Xu, and Q. Zhang, "Automatic corner detection and localization for camera calibration," *Proceedings - IEEE 2011 10th International Conference on Electronic Measurement and Instruments, ICEMI 2011*, vol. 4, pp. 312–315, 2011. doi: 10.1109/ICEMI.2011.6038004

[29] R. S. D. Scharstein and R. Zabih, "A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms," *Proceedings IEEE Workshop on Stereo and Multi-Baseline Vision*, pp. 131–140, 2001. doi: 10.1109/SMBV.2001.988771.

[30] M. Dinham, "Autonomous Weld Joint Detection and Localisation Using Computer Vision in Robotic Arc Welding," Ph.D. dissertation, University of Western Sydney, 2013.

[31] MICRO-EPSILON, *scanCONTROL 2900 Quick Reference*. Optronic GmbH, Lessingstr. 14, D-01465 Dresden-Langebrück, Germany: MICRO-EPSILON.

[32] P. J. B. McKay and N. D., "A method for registration of 3-D shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992. doi: 10.1109/34.121791

[33] Z. Zhang, "Iterative point matching for registration of free-form curves and surfaces," *International Journal of Computer Vision*, vol. 13, no. 2, pp. 119–152, 1994. doi: 10.1007/BF01427149

[34] S. Rusinkiewicz and M. Levoy, "Efficient variants of the ICP algorithm," *Proceedings of International Conference on 3-D Digital Imaging and Modeling, 3DIM*, pp. 145–152, 2001. doi: 10.1109/IM.2001.924423

[35] R. B. Rusu, N. Blodow, and M. Beetz, "Fast Point Feature Histograms (FPFH) for 3D Registration," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 3212–3217, 2009. doi: 10.1109/ROBOT.2009.5152473

[36] K. Low, "Linear Least-squares Optimization for Point-to-plane ICP Surface Registration," *Chapel Hill, University of North Carolina*, no. February, pp. 2–4, 2004. [Online]. Available: https://www.iscs.nus.edu.sg/$\sim$lowkl/publications/lowk_point-to-plane_icp_techrep.pdf

[37] P. Li, R. Wang, Y. Wang, and W. Tao, "Evaluation of the ICP Algorithm in 3D Point Cloud Registration," *IEEE Access*, vol. 8, pp. 68 030–68 048, 2020. doi: 10.1109/ACCESS.2020.2986470

[38] R. Schnabel, R. Wahl, and R. Klein, "Efficient RANSAC for point-cloud shape detection," *Computer Graphics Forum*, vol. 26, no. 2, pp. 214–226, 2007. doi: 10.1111/j.1467-8659.2007.01016.x

[39] C. Miyata, "Development of a Safety-Oriented Interaction Strategy and Compliant Limb Sensor System for a Gait Rehabilitation Robot," 2015. [Online]. Available: https://curve.carleton.ca/61da05f8-1079-48c1-be92-988ed48b5e07

[40] "Mastercam," 2022. [Online]. Available: https://www.mastercam.com/

[41] "Nx cam," 2022. [Online]. Available: https://www.plm.automation.siemens.com/global/en/products/manufacturing-planning/cam-software.html

[42] "Fusion 360," 2022. [Online]. Available: https://www.autodesk.com/products/fusion-360/overview?term=1-YEAR&tab=subscription

[43] N. Asakawa, K. Toda, and Y. Takeuchi, "Automation of chamfering by an industrial robot; For the case of hole on free-curved surface," *Robotics and Computer-Integrated Manufacturing*, vol. 18, no. 5-6, pp. 379–385, 2002. doi: 10.1016/S0736-5845(02)00006-6

[44] D. J. Tomizuka and M., "Learning Hybrid Force and Position Control of Robot Manipulators," *IEEE transactions on robotics and automation*, vol. 9, no. 4, pp. 423–431, 1993. doi: 10.1109/70.246053

[45] H. Kazerooni, "Automated robotic deburring using impedance control," *IEEE Control Systems Magazine*, pp. 21–25, 1988. doi: 10.1109/37.464

[46] H. C. Song and J. B. Song, "Precision robotic deburring based on force control for arbitrarily shaped workpiece using CAD model matching," pp. 85–91, 2013.

[47] F. Leali, M. Pellicciari, F. Pini, G. Berselli, and A. Vergnano, "An offline programming method for the robotic Deburring of aerospace components," *Communications in Computer and Information Science*, vol. 371, pp. 1–13, 2013. doi: 10.1007/978-3-642-39223-8_1

[48] R. Rajagopalan, R. M. Cheng, V. R. Ayyadevara, and G. Huard, "System architecture and edge tracking performance of an automated robotic deburring workcell," *IEEE International Symposium on Intelligent Control - Proceedings*, no. Figure 1, pp. 351–356, 1995. doi: 10.1109/isic.1995.525082

[49] H. Huang, Z. M. Gong, X. Q. Chen, and L. Zhou, "SMART robotic system for 3D profile turbine vane airfoil repair," *International Journal of Advanced Manufacturing Technology*, vol. 21, no. 4, pp. 275–283, 2003. doi: 10.1007/s001700300032

[50] A. M. Kabir, P. M. Bhatt, B. C. Shah, and S. K. Gupta, "Robotic Finishing of Geometrically Complex Parts," in *Manufacturing In The*

Era Of 4th Industrial Revolution: A World Scientific Reference (In 3 Volumes), mar 2018, pp. 315–336. ISBN 9789811222795. [Online]. Available: https://www.worldscientific.com/doi/abs/10.1142/9789811222849_0012

[51] S. Kumar, A. Kuss, U. Schneider, M. Drust, T. Dietz, J. R. D. Posada, and A. Verly, "Automatic programming and control for robotic deburring," *47th International Symposium on Robotics, ISR 2016*, pp. 688–695, 2016.

[52] W. L. Li, H. Xie, G. Zhang, S. J. Yan, and Z. P. Yin, "3-D Shape Matching of a Blade Surface in Robotic Grinding Applications," *IEEE/ASME Transactions on Mechatronics*, vol. 21, no. 5, pp. 2294–2306, 2016. doi: 10.1109/TMECH.2016.2574813

[53] M. Dinham and G. Fang, "Autonomous weld seam identification and localisation using eye-in-hand stereo vision for robotic arc welding," *Robotics and Computer-Integrated Manufacturing*, vol. 29, no. 5, pp. 288–301, 2013. doi: 10.1016/j.rcim.2013.01.004

[54] M. Kong, F. H. Shi, S. B. Chen, and T. Lin, "Recognition of the initial position of weld based on the corner detection for welding robot in global environment," *Lecture Notes in Control and Information Sciences*, vol. 362, pp. 249–255, 2007. doi: 10.1007/978-3-540-73374-4_29

[55] M. Dinham and G. Fang, "Detection of fillet weld joints using an adaptive line growing algorithm for robotic arc welding," *Robotics and Computer-Integrated Manufacturing*, vol. 30, no. 3, pp. 229–243, 2014. doi: 10.1016/j.rcim.2013.10.008

[56] Z. Lai, R. Xiong, H. Wu, and Y. Guan, "Integration of Visual Information and Robot Offline Programming System for Improving Automatic Deburring Process," *2018 IEEE International Conference on Robotics and Biomimetics, ROBIO 2018*, pp. 1132–1137, 2018. doi: 10.1109/ROBIO.2018.8665148

[57] F. Leo Princely and T. Selvaraj, "Vision assisted robotic deburring of edge burrs in cast parts," *Procedia Engineering*, vol. 97, pp. 1906–1914, 2014. doi: 10.1016/j.proeng.2014.12.344

[58] Z. Zivkovic, "Improved adaptive Gaussian mixture model for background subtraction," *Proceedings - International Conference on Pattern Recognition*, vol. 2, pp. 28–31, 2004. doi: 10.1109/icpr.2004.1333992

[59] D. Nakhaeinia, P. Payeur, and R. Laganiere, "Adaptive robotic contour following from low accuracy RGB-D surface profiling and visual servoing," *Proceedings - Conference on Computer and Robot Vision, CRV 2014*, pp. 48–55, 2014. doi: 10.1109/CRV.2014.15

[60] M. Pedersen, 2022. [Online]. Available: https://www.mathworks.com/matlabcentral/fileexchange/66632-dxftool

[61] B. Braden, "The Surveyor's Area Formula," *The College Mathematics Journal*, vol. 17, no. 4, p. 326, 1986. doi: 10.2307/2686282

[62] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," *arXiv:1801.09847*, 2018.

[63] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. doi: 10.1038/s41592-019-0686-2

[64] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[65] K. Xu, Y. Li, and B. Xiang, "Image processing-based contour parallel tool path optimization for arbitrary pocket shape," *International Journal of Advanced Manufacturing Technology*, vol. 102, no. 5-8, pp. 1091–1105, 2018. doi: 10.1007/s00170-018-3016-4

[66] S. Suzuki and K. A. Be, "Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985. doi: 10.1016/0734-189X(85)90016-7

[67] G. Bone, M. Elbestawi, L. Liu, and N. Sinha, "Adaptive Robotic Deburring," *IFAC Proc. Vol.*, vol. 23, no. 1, pp. 313–318, 1990. doi: 10.1016/s1474-6670(17)52739-1

[68] G. M. Bone, M. A. Elbestawi, R. Lingarkar, and L. Liu, "Force control for robotic deburring," *J. Dyn. Syst. Meas. Control. Trans. ASME*, vol. 113, no. 3, pp. 395–400, 1991. doi: 10.1115/1.2896423

[69] Gary M. Bone and Mohamed A. Elbestawi, "Sensing and Control for Automated Edge Deburring," *IEEE Transactions on Industrial Electronics*, vol. 41, no. 2, pp. 137–146, 1994. doi: 10.1109/41.293873

[70] G. M. Bone and M. A. Elbestawi, "Dual-sensor based robotic deburring," Tech. Rep. 3, 1996.